Faculty of Computing and Data Science

Alexandria University

RENTY

Graduation Project



Yasser Ashraf Mohammed

Hamza Hussain Youssef Omran

Ahmad Khalifa Abdulraouf

Moaz Moustafa AbdulHamid

Moamen Ahmed Mohammed

AbdulRahman Hesham Ragab

Supervised By

Prof. Dr. Mohammed Khamis

Abstract

The rapid growth of the sharing economy and e-commerce platforms has created unprecedented opportunities for peer-to-peer resource utilization, yet this expansion brings significant challenges in personalization, security, operational efficiency, and supply-demand optimization. Generic regional classifieds platforms struggle to deliver relevant recommendations to users navigating extensive product catalogs, fail to detect sophisticated fraudulent activities that evolve continuously, and lack analytical frameworks to identify market imbalances that hinder platform growth. These challenges result in suboptimal user experiences, financial losses, and inefficient resource allocation across product categories. Furthermore, high-value asset sharing requires profound trust mechanisms that these generic listing platforms inherently lack, creating a barrier to adoption in emerging markets where institutional trust infrastructure remains nascent.

This graduation project presents **Renty**, an intelligent, cross-platform (Web and Mobile) peer-to-peer rental ecosystem that addresses these critical challenges through the integration of artificial intelligence, data science components, and robust operational workflows. The platform facilitates secure rentals primarily focused on sports tourism in Egypt—particularly in regions such as Dahab, Sinai, and the Red Sea coast—and general high-value items, offering a viable, commission-based business model. Renty serves local users who wish to try sports without purchasing expensive gear and sports tourists who prefer not to travel with heavy equipment, bridging a significant gap in the Egyptian marketplace.

To personalize the user experience, Renty incorporates a hybrid recommendation system built upon the LightFM framework that combines collaborative filtering with content-based approaches. Trained on 55,666 transactions across 17,293 customers and 130 product categories, the system achieved a test AUC of 0.8421, representing a 12.3% improvement over baseline performance. The hybrid approach effectively addresses the cold-start problem inherent in dynamic catalogs by representing users and items as the weighted sum of their feature embeddings—including demographic profiles, behavioral metrics, and textual descriptions processed via TF-IDF analysis—enabling meaningful recommendations even for newly registered users and freshly listed items. The WARP (Weighted Approximate-Rank Pairwise) loss function was employed to optimize for the top of the recommendation list, and a meticulously tuned L2 regularization penalty reduced the overfitting gap by 80.8% while increasing Precision@10 by 28.8%.

Security is fortified through an unsupervised anomaly detection module employing the Isolation Forest algorithm, designed to identify fraudulent behavioral patterns—such as account takeovers, velocity abuse through automated rapid-fire transactions, and financial mismatches suggesting stolen payment credentials—without requiring labeled training examples. The model analyzes a behavioral velocity vector encompassing temporal patterns, velocity metrics, financial ratios, and geographic mismatches, generating a risk score between 0.0 and 1.0. Transactions scoring above the configurable threshold

(default 0.75) are logged to dedicated fraud tables and trigger an immediate checkout lock, requiring administrative review before the user may proceed. This unsupervised approach is particularly well-suited to the platform's context, where labeled fraud data is unavailable for a nascent marketplace.

Operational efficiency is achieved via a comprehensive Gap Analysis framework and KPI dashboards, providing platform administrators with real-time visibility into supply-demand imbalances and overall platform health. The Gap Analysis framework computes normalized gap scores for every product subcategory by aggregating active inventory against observed user demand—derived from search logs and rejected rental requests—and classifies gaps into four severity tiers (Low, Moderate, High, Critical) to guide targeted interventions ranging from passive monitoring to emergency supplier recruitment campaigns.

Crucially, AI modules are integrated as supporting components to enhance decision-making, not as standalone systems. The core of Renty is a transactional engine driven by strict state machines governing the full rental lifecycle through nine distinct states—from requested through confirmed, pending_hub_evaluation, active, returned, completed, cancelled, extended, to disputed—with each transition enforced at the database level through PostgreSQL ENUMs, triggers, and check constraints. Trust and security in peer-to-peer transactions are enforced through physical "Renty Points" (hubs) for expert inspections with a continuity rule requiring the same expert for pre- and post-rental verification, escrow payment handling that holds all funds until successful completion, and clear dispute resolution protocols governed by a robust PostgreSQL relational database and an ASP.NET Core backend API.

The mobile application, developed using Flutter, acts as an optimized on-the-go interface for renters and owners, ensuring real-time push notifications via Firebase Cloud Messaging (FCM) and SignalR, GPS-based discovery through interactive map interfaces, and quick photo uploads for listing creation and verification. The web application, built with Angular, serves as the primary, fully-featured platform for all user roles, while providing administrators, support agents, and experts with comprehensive operational interfaces including KPI monitoring, fraud moderation panels, gap analysis heatmaps, and expert hub consoles.

By uniting these technical and operational components into a scalable, layered architecture—separating presentation, application logic, data persistence, and intelligence layers—Renty provides a comprehensive, data-driven solution to modern e-commerce challenges, demonstrating practical engineering capabilities suitable for production deployment while maintaining academic rigor across both semesters of development.

Acknowledgment

The successful completion of this graduation project represents the culmination of extensive effort, dedication, and invaluable support from numerous individuals whose contributions deserve recognition and gratitude.

First and foremost, we express our deepest appreciation to our project supervisor, Dr. Mohammed Khamis, whose guidance, expertise, and unwavering support throughout this journey proved instrumental in shaping both the technical direction and academic rigor of this work. The insightful feedback, constructive criticism, and encouragement provided during countless supervision sessions helped us navigate complex challenges and maintain focus on delivering a comprehensive solution that meets both academic standards and practical applicability. Dr. Khamis's emphasis on architectural correctness, documentation depth, and real-world feasibility pushed our work beyond the typical scope of a graduation project, and the resulting system reflects his high standards.

We extend our sincere gratitude to the faculty members of the Faculty of Computing and Data Science, Alexandria University, who equipped us with the foundational knowledge and technical skills essential for undertaking a project of this scope and complexity. The coursework in machine learning, data structures, software engineering, database systems, and distributed computing provided the theoretical framework upon which we built our practical implementation. Their dedication to teaching and commitment to student success created an environment conducive to learning and innovation.

Our appreciation extends to the authors of the open-source libraries and frameworks leveraged throughout this project, particularly the developers of Angular, Flutter, ASP.NET Core, Entity Framework, PostgreSQL, SignalR, LightFM, scikit-learn, Pandas, and related tools that accelerated our development timeline and enabled us to focus on domain-specific challenges rather than reinventing foundational algorithms. The spirit of knowledge sharing embodied by the open-source community exemplifies the collaborative nature of technological advancement.

We acknowledge the dedication and complementary expertise within our team, which collectively enabled the successful completion of this ambitious project within a constrained timeline. Yasser, Momen, and Abdurrahman led the development of the intelligent components of the system, specifically the recommendation system and the fraud detection module, encompassing data preprocessing, feature engineering, collaborative filtering techniques, and anomaly detection. Additionally, Yasser single-handedly spearheaded the complete development of the cross-platform mobile application. Hamza, Ahmed, and Moaz led the backend API and web dashboard frontend, system architecture design, KPI formulation, gap analysis algorithms, business analytics, and real-time data integration. Their efforts ensured that complex analytical insights and robust operational logic were effectively translated into intuitive, user-facing interfaces and decision-support tools.

This graduation project represents not merely the completion of academic requirements but the realization of our collective vision to apply theoretical knowledge toward solving real-world challenges in the rapidly evolving e-commerce domain. The lessons learned, skills developed, and relationships forged during this journey will undoubtedly influence our future endeavors in technology and innovation.

Table of Contents

Table of Contents

Cover Letter....1

Abstract....2

Acknowledgment....3

Table of Contents....5

List of Figures....15

List of Tables....44

Abbreviations....62

Symbols....65

Chapter 1 – Introduction....66

1.1 Motivation....66

1.2 Problem Statement....67

1.3 Objective....68

1.4 Problem Complexity....69

1.5 Constraints....70

1.6 Standards....71

1.7 Feasibility Study and Business Canvas....71

1.8 Addressing Supervisor Feedback....72

Chapter 2 – Background....73

2.1 E-Commerce and Peer-to-Peer Rental Platforms....73

2.1.1 Evolution of Digital Marketplaces....73

2.1.2 Characteristics of Rental Transactions....73

2.1.3 Market Dynamics and Network Effects....74
2.2 Recommender Systems....74
2.2.1 Fundamental Concepts and Approaches....74
2.2.2 Matrix Factorization Techniques....75
2.2.3 The LightFM Framework....75
2.2.4 Evaluation Metrics for Recommender Systems....76
2.3 Anomaly Detection and Fraud Prevention....76
2.3.1 Fundamentals of Anomaly Detection....76
2.3.2 Supervised versus Unsupervised Approaches....77
2.3.3 Isolation Forest Algorithm....77
2.3.4 Feature Engineering for Fraud Detection....77
2.4 Business Analytics and Key Performance Indicators....78
2.4.1 Principles of Performance Measurement....78
2.4.2 Gap Analysis Frameworks....78
2.4.3 Analytical Data Pipelines....78
2.5 Software Architecture Patterns....78
2.5.1 Layered Architecture....78
2.5.2 Microservices-Inspired Patterns....79
2.5.3 Real-Time Communication Patterns....79
Chapter 3 – Related Work and Similar Systems....79
3.1 Egyptian Market Analysis....79
3.1.1 Dubizzle Egypt (OLX Egypt before)....79
3.2 Regional Platform Analysis....80

3.2.1 OpenSooq..... 80
3.2.2 Yalla Fel Nile (Emerging)..... 80
3.3 Global Platform Analysis..... 80
3.3.1 Airbnb..... 80
3.3.2 Fat Llama / Hurr..... 80
3.3.3 Turo..... 81
3.4 Competitive Differentiation Analysis..... 81
Chapter 4 – Analysis..... 81
4.1 Functional Requirements..... 82
4.1.1 Mobile Application (Renters & Owners)..... 82
4.1.2 Web Application (Administrators, Experts, and Support Agents)..... 88
4.2 Non-functional Requirements..... 91
4.3 UML Diagrams..... 92
Chapter 5 – System Design and Architecture..... 93
5.1 High-Level Architecture..... 93
5.2 System Integration and Concurrency..... 94
5.3 User Roles and RBAC Matrix..... 95
5.4 The Rental Lifecycle and State Machine..... 96
5.5 Trust Mechanisms and Escrow..... 97
5.6 Notification System Architecture..... 98
Chapter 6 – Database and Backend Architecture..... 99
6.1 Database Design Philosophy..... 99
6.2 Core Tables and Relationships..... 99

6.2.1 User Management..... 99
6.2.2 Item Management..... 99
6.2.3 Rental and Order Management..... 100
6.2.4 Trust and Verification..... 100
6.2.5 Communication..... 100
6.2.6 Support and Disputes..... 100
6.3 Advanced Database Constraints..... 101
6.3.1 Triggers..... 101
6.3.2 CHECK Constraints..... 101
6.3.3 Views..... 101
6.3.4 Indexing Strategy..... 101
6.4 Backend API Architecture..... 102
6.4.1 ASP.NET Core Endpoint Structure..... 102
6.4.2 Request Processing Pipeline..... 102
6.4.3 Real-Time Infrastructure (SignalR)..... 102
6.5 Environment and Configuration..... 103
Chapter 7 – Intelligent Enhancements (AI Modules)..... 104
7.1 Hybrid Recommendation System..... 104
7.1.1 Data Preparation and Exploration..... 104
7.1.2 Feature Engineering..... 104
7.1.3 Model Selection and Architecture..... 105
7.1.4 Training Configuration and Hyperparameter Tuning..... 106
7.1.5 Performance Results..... 106

7.1.6 Inference and API Integration.....107
7.2 Unsupervised Fraud Detection.....107
7.2.1 Motivation and Approach.....107
7.2.2 Feature Engineering.....107
7.2.3 Model Configuration.....108
7.2.4 Scoring and Decision Logic.....108
7.2.5 Integration with Rental Lifecycle.....109
7.3 Supply-Demand Gap Analysis.....109
7.3.1 Pipeline Overview.....109
7.3.2 Gap Score Computation.....109
7.3.3 Severity Classification.....110
7.3.4 Dashboard Integration.....110
7.3.5 Automation and Manual Trigger.....111
Chapter 8 – System Implementation.....112
8.1.2 Frontend Technologies (Web Dashboard).....114
8.1.3 Mobile Technologies.....114
8.1.4 AI/Analytics Technologies.....115
8.2 Development Environment.....116
8.3 Backend Implementation Highlights.....116
8.3.1 Authentication Flow.....116
8.3.2 State Machine Enforcement.....116
8.3.3 Escrow Implementation.....117
8.4 Mobile Application Implementation.....118

Chapter 9 – User Interface and Experience....120
9.1 Mobile Application and Web UX....120
9.1.1 Home Page and Discovery....120
9.1.2 Item Detail and Booking Flow....120
9.1.3 Active Rental Dashboard....121
9.1.4 Chat Interface....121
9.1.5 Profile and Settings....121
9.2 Web Dashboard UX....122
9.2.1 Admin Dashboard — KPI Overview....122
9.2.2 Gap Analysis Heatmap....122
9.2.3 Expert Hub Console....122
9.2.4 Support Ticket Management....122
9.3 Super Admin Interface....123
9.4 Support Agent Interface....123
9.5 Expert (Renty Point) Interface....124
Chapter 10 – Industrial Application and Business Model....125
10.1 Revenue Streams....125
10.2 The Complete Rental Lifecycle (Business Perspective)....125
10.2.1 Discovery and Negotiation....125
10.2.2 Rental Request and Date Management....125
10.2.3 Payment and Escrow....126
10.2.4 The Hub and Expert Evaluation (Trust Mechanic)....126
10.2.5 Two-Way Confirmation....126

10.2.6 Completion and Payout....127
10.3 Late Return Escalation....127
10.4 Rental Extensions....127
10.5 Cancellation Protection....128
10.6 Dispute Resolution....128
10.7 Wallet Operations....129
10.8 Edge Case Scenarios....129
10.8.1 Expert Unavailability....129
10.8.2 Payment Gateway Failure....129
10.8.3 Simultaneous Booking Race Condition....130
10.8.4 User Suspension During Active Rental....130
10.8.5 Item Reported During Active Rental....130
10.9 Platform Configuration....130
Chapter 11 – Testing and Validation....131
11.1 Testing Scope and Strategy....131
11.1.1 Testing Environment....131
11.2 Recommendation System Testing....132
11.2.1 Objective....132
11.2.2 Offline Evaluation Procedure....132
11.2.3 Overfitting and Stability Checks....132
11.2.4 Cold-Start Validation....132
11.2.5 Output Inspection....133
11.3 Gap Analysis Testing....133

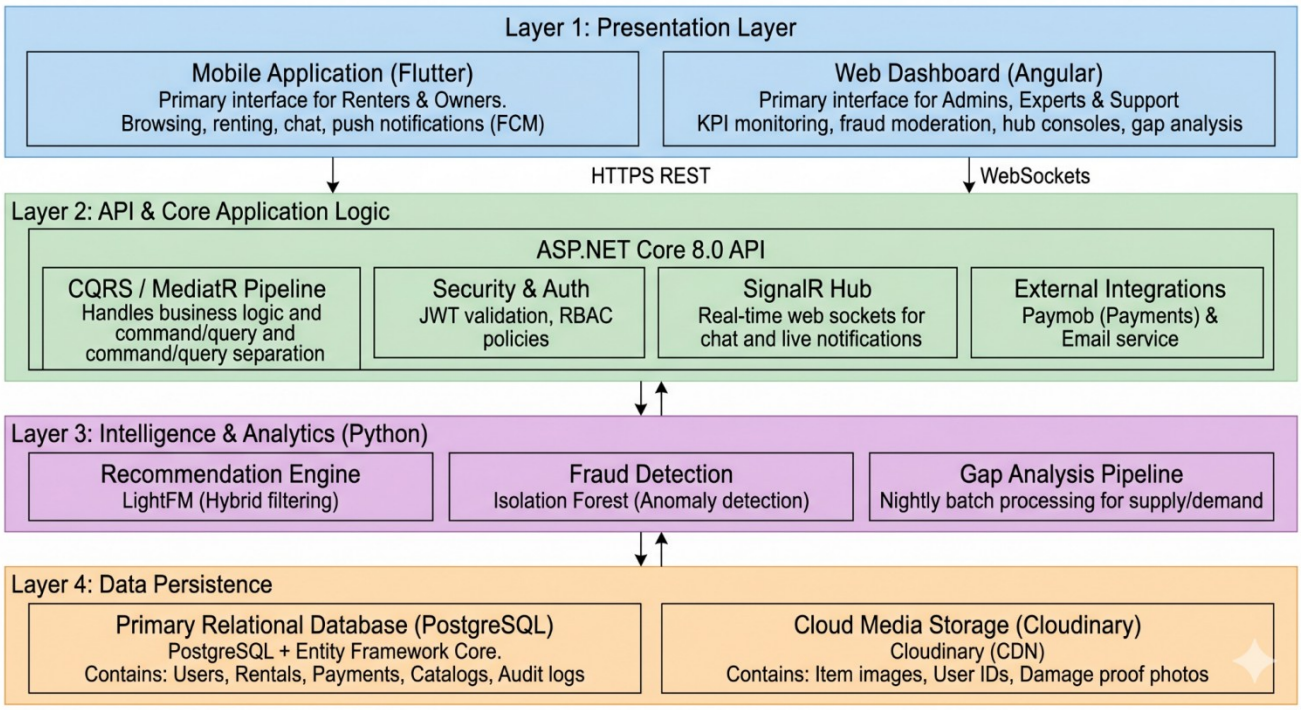
11.3.1 Pipeline Execution Testing....133
11.3.2 Automated Execution Testing....133
11.3.3 Manual Trigger Testing....134
11.3.4 Data Persistence and Retrieval....134
11.4 Integration Testing....134
11.4.1 Backend-Database Integration....134
11.4.2 Frontend-Backend Integration....135
11.4.3 End-to-End Rental Flow....135
11.5 Next-Semester Testing Enhancements (Completed)....136
Chapter 12 – Conclusion and Future Work....137
12.1 Summary of Achievements....137
12.2 Lessons Learned....138
12.3 Future Work....139
Chapter 13 – References....141
Appendix A – Database Schema Excerpts....144
A.1 Core ENUM Types....144
A.2 Self-Rental Prevention Trigger....144
A.3 Date Validation Constraints....145
A.4 Key Analytical Views....145
Appendix B – API Contract Reference....146
B.1 Authentication Endpoints....146
POST /api/v1/auth/register....146
POST /api/v1/auth/login....147

B.2 Item Management Endpoints....148
GET /api/v1/items....148
POST /api/v1/items....149
B.3 Rental Lifecycle Endpoints....150
POST /api/v1/rentals....150
PUT /api/v1/rentals/:id/confirm....151
PUT /api/v1/rentals/:id/cancel....151
B.4 Payment Endpoints....152
POST /api/v1/payments/checkout....152
POST /api/v1/payments/webhook....153
B.5 Real-Time WebSocket Events....153
Client → Server Events....153
Server → Client Events....153
B.6 Admin-Only Endpoints....154
GET /api/v1/admin/dashboard....154
PUT /api/v1/admin/items/:id/approve....154
PUT /api/v1/admin/items/:id/reject....154
PUT /api/v1/admin/fraud/:id/action....155
Appendix C – Notification Event Type Reference....155
Appendix D – Extended Entity Interaction Scenarios....156
D.1 Scenario: Renter Requests Rental for a High-Value Item....157
D.2 Scenario: Late Return with Deposit Exhaustion....158
D.3 Scenario: Dispute Resolution with Expert Evidence....159

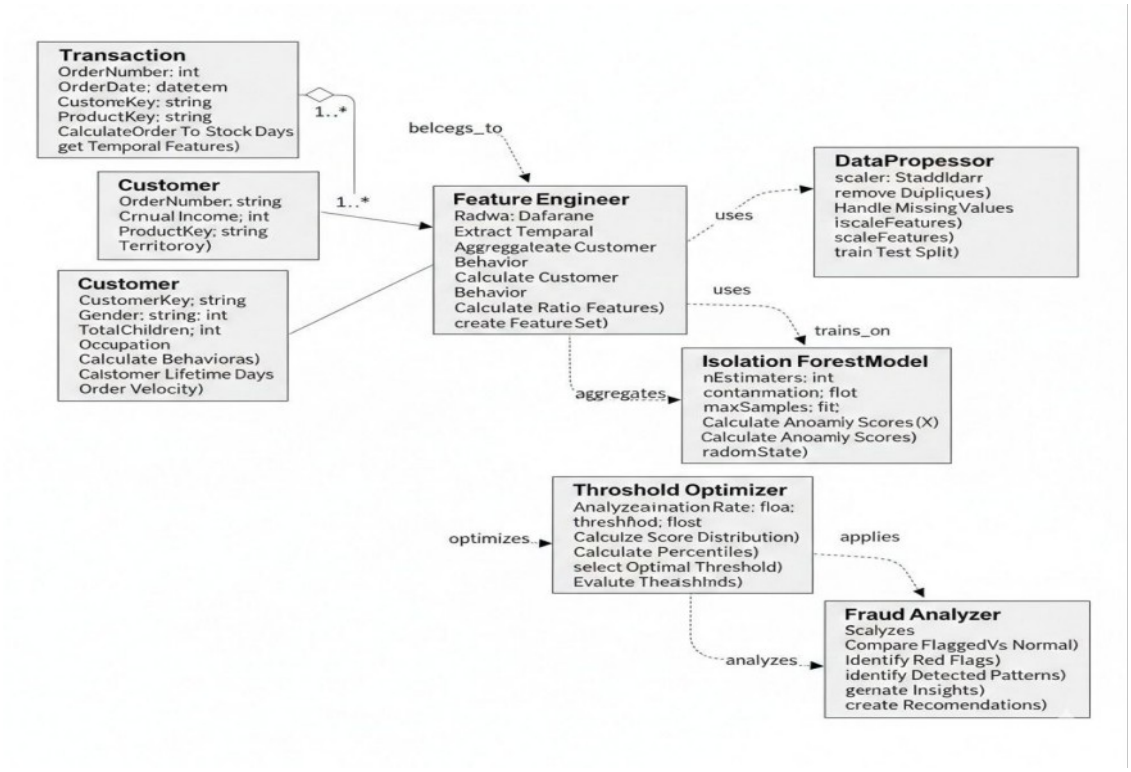
D.4 Scenario: Cancellation Under Strict Policy....159
D.5 Scenario: Simultaneous Booking Race Condition....160
D.6 Scenario: Owner Cancellation Abuse Prevention....160
D.7 Scenario: Expert Continuity Rule Violation....161
D.8 Scenario: Mobile App Offline-to-Online Sync....161
Appendix F – Detailed Non-Functional Requirements....162
F.1 Performance Requirements....162
F.1.1 Response Time Requirements....162
F.1.2 Throughput Requirements....163
F.1.3 Scalability Approach....163
F.2 Security Requirements....164
F.2.1 Authentication Security....164
F.2.2 Data Protection....164
F.2.3 Rate Limiting Configuration....165
F.3 Reliability and Availability....165
F.3.1 Data Integrity Guarantees....165
F.3.2 Error Handling Strategy....166
F.4 Maintainability Requirements....166
F.4.1 Code Organization....166
F.4.2 Documentation Standards....167
F.4.3 Version Control Practices....167
Appendix G – Glossary of Domain-Specific Terms....167

List of Figures

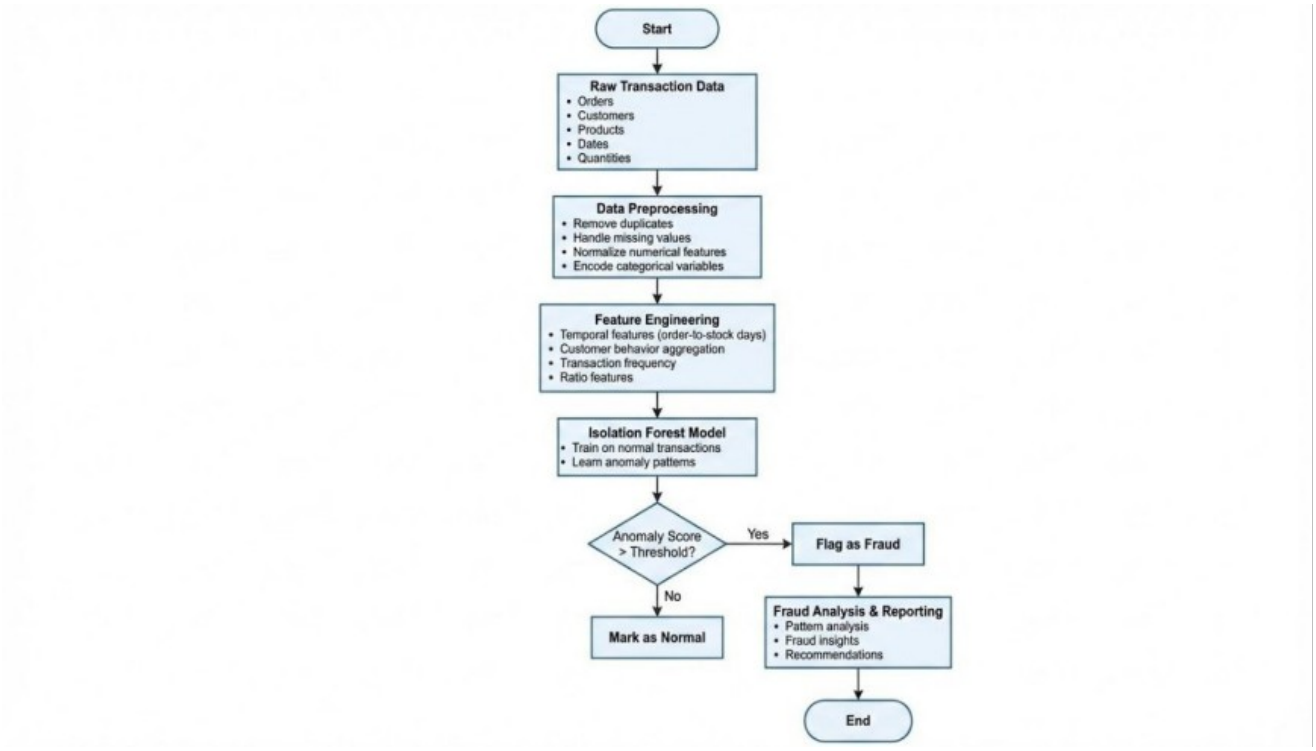
- **Figure 1:** High-Level System Architecture Diagram (Four-Layer Overview)



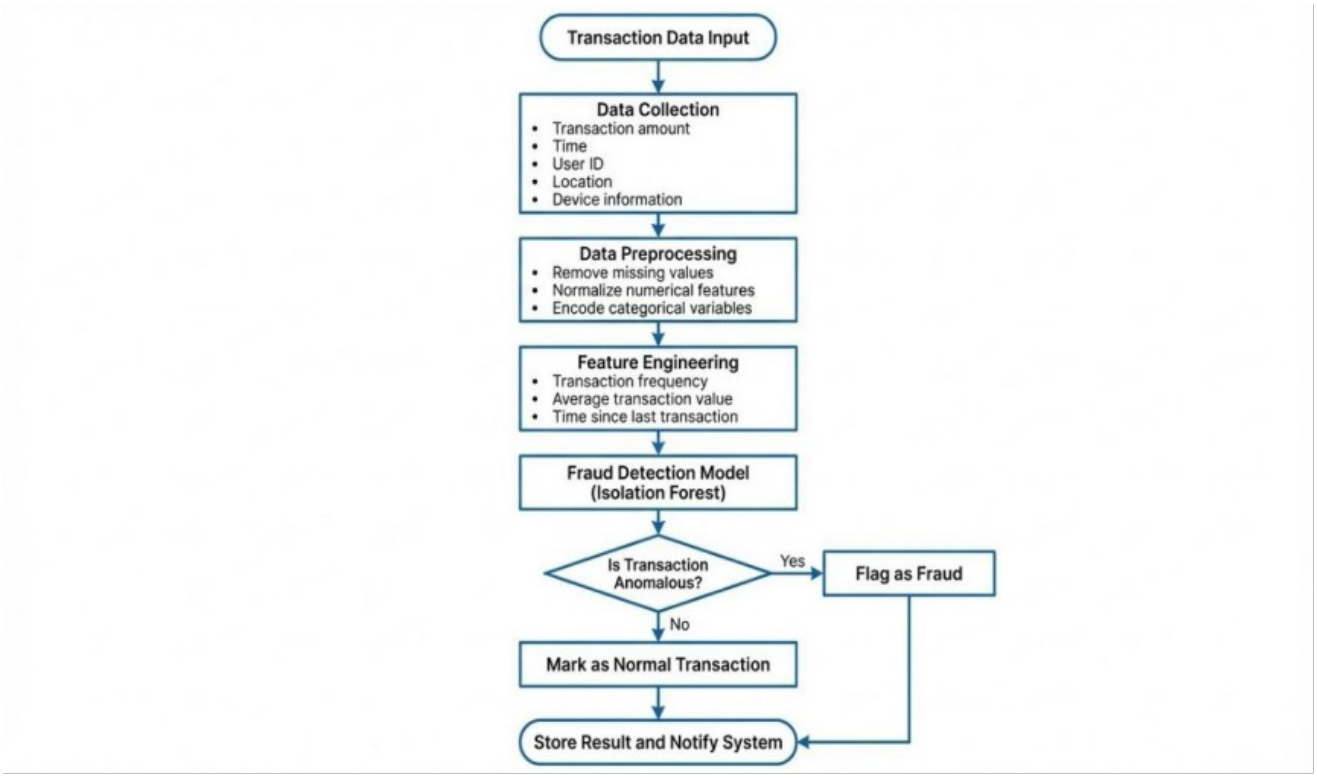
- **Figure 2:** Recommendation System Pipeline Diagram (Data → Features → LightFM → Predictions)



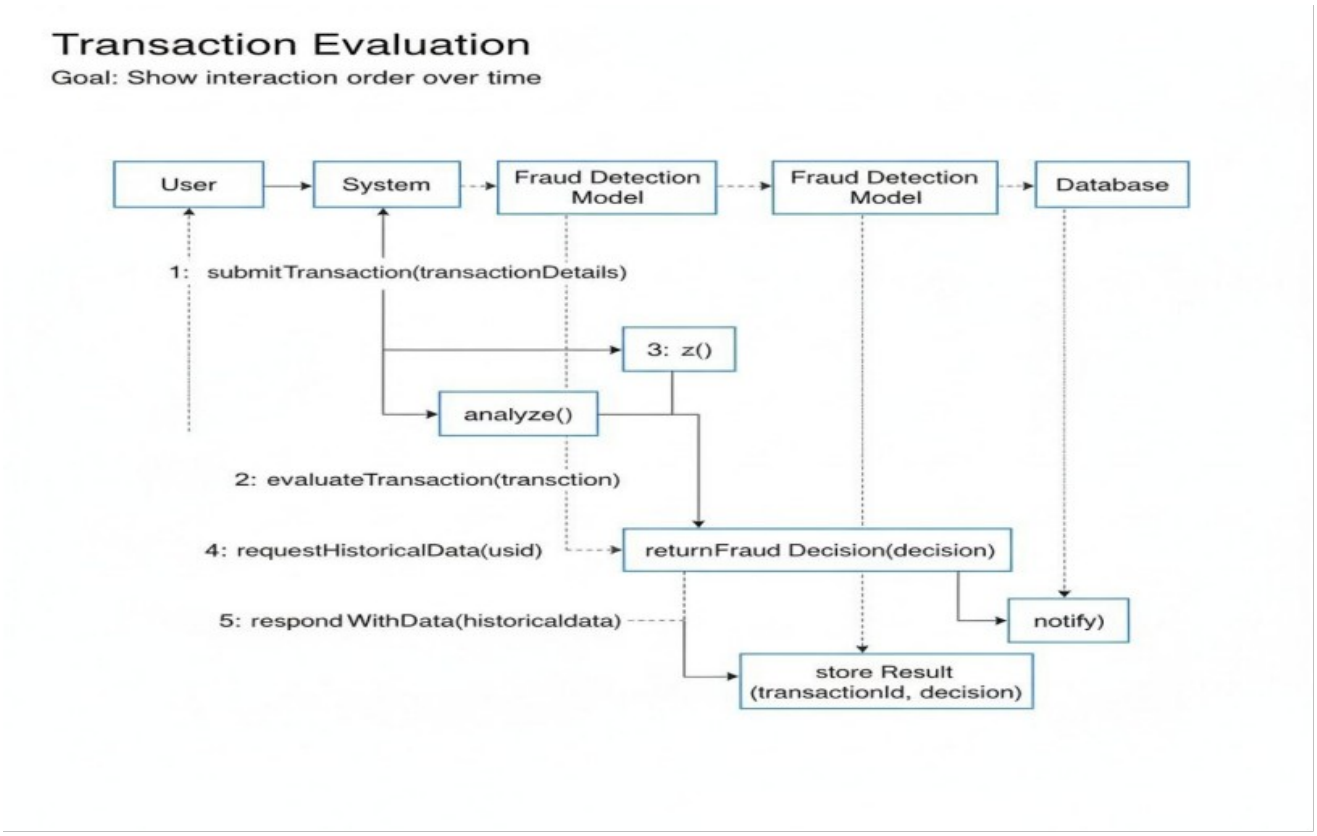
- **Figure 3:** Fraud Detection UML Class Diagram



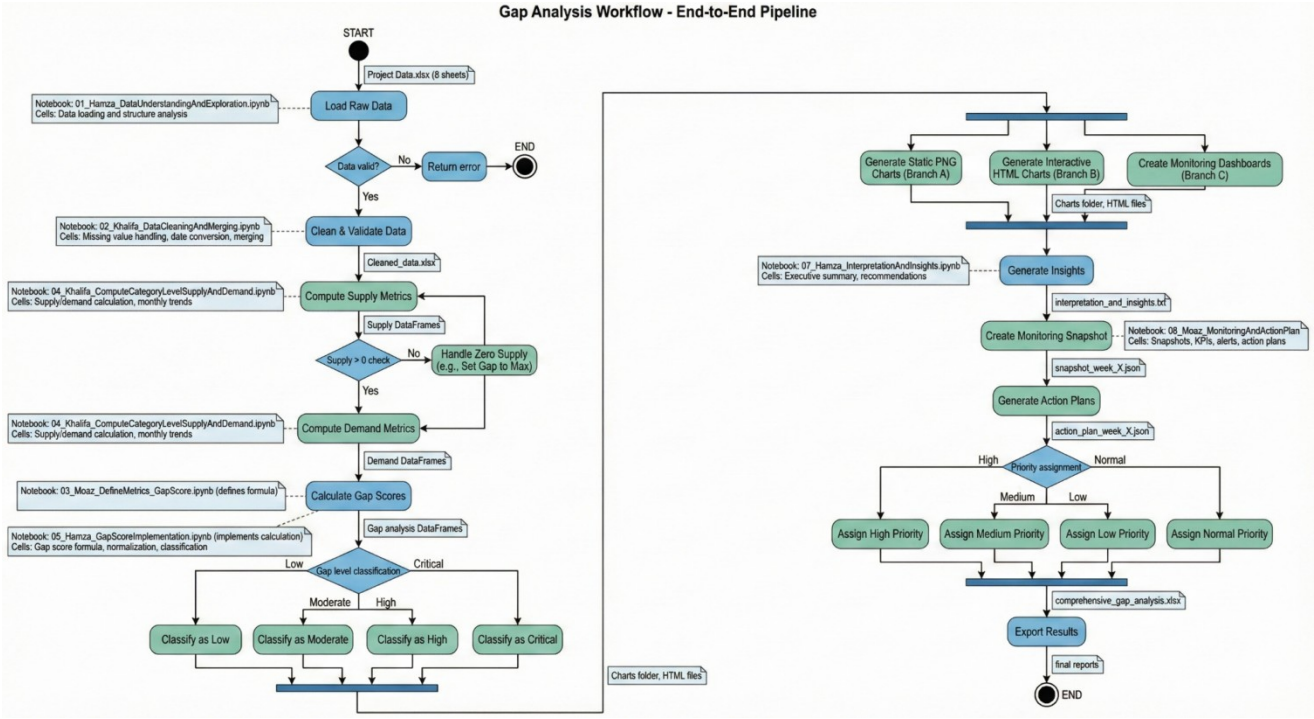
- **Figure 4:** Fraud Detection Activity Diagram



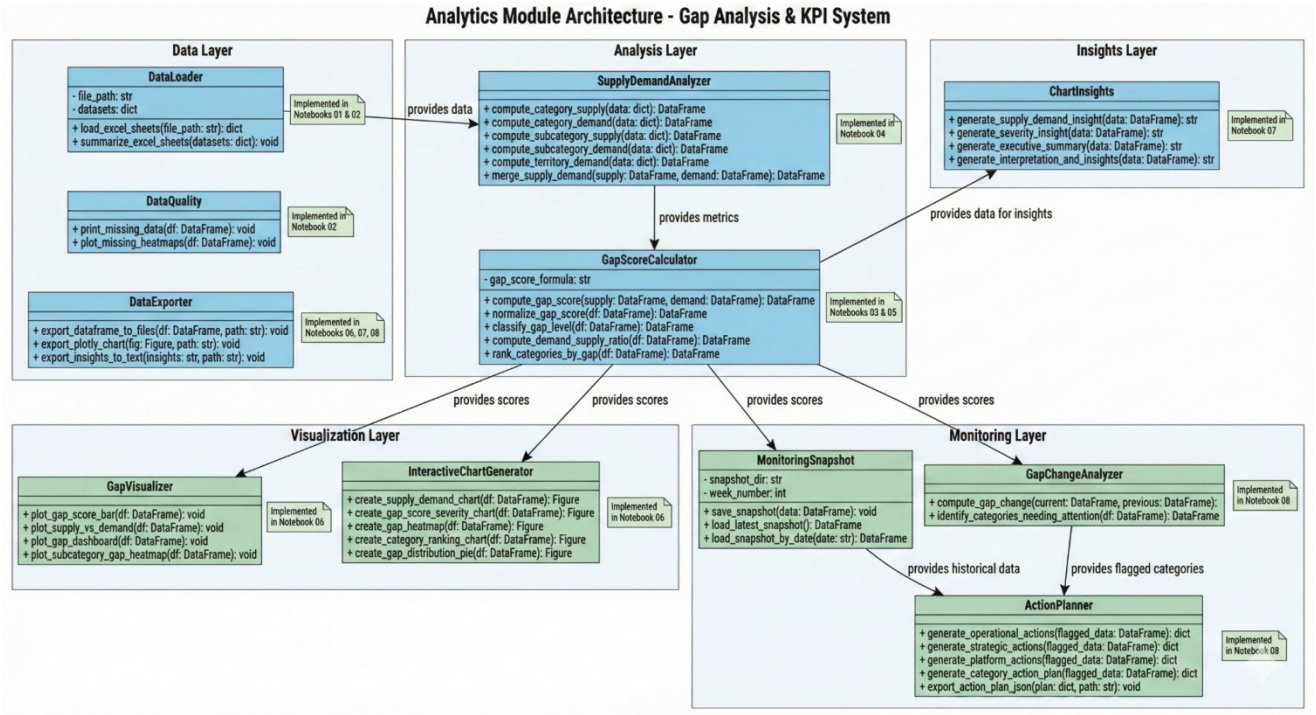
- **Figure 5:** Fraud Detection Data Flow Diagram



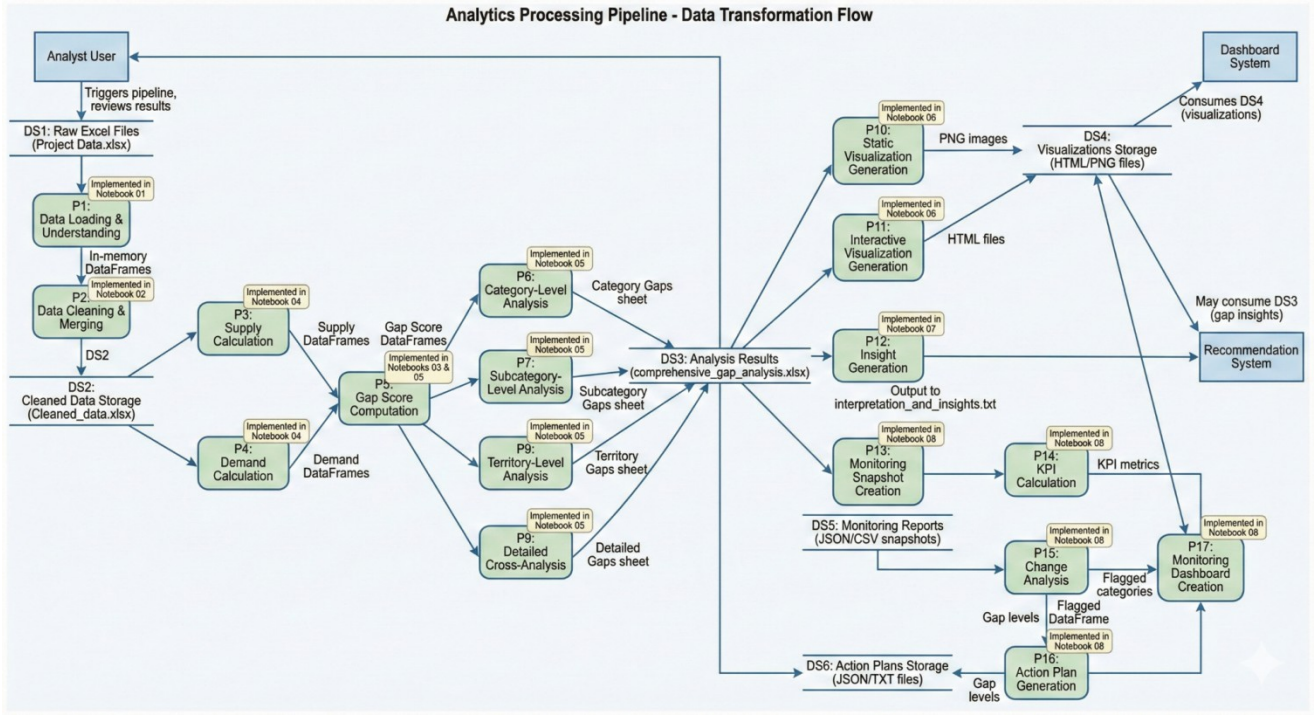
- **Figure 7:** Gap Analysis Activity Diagram



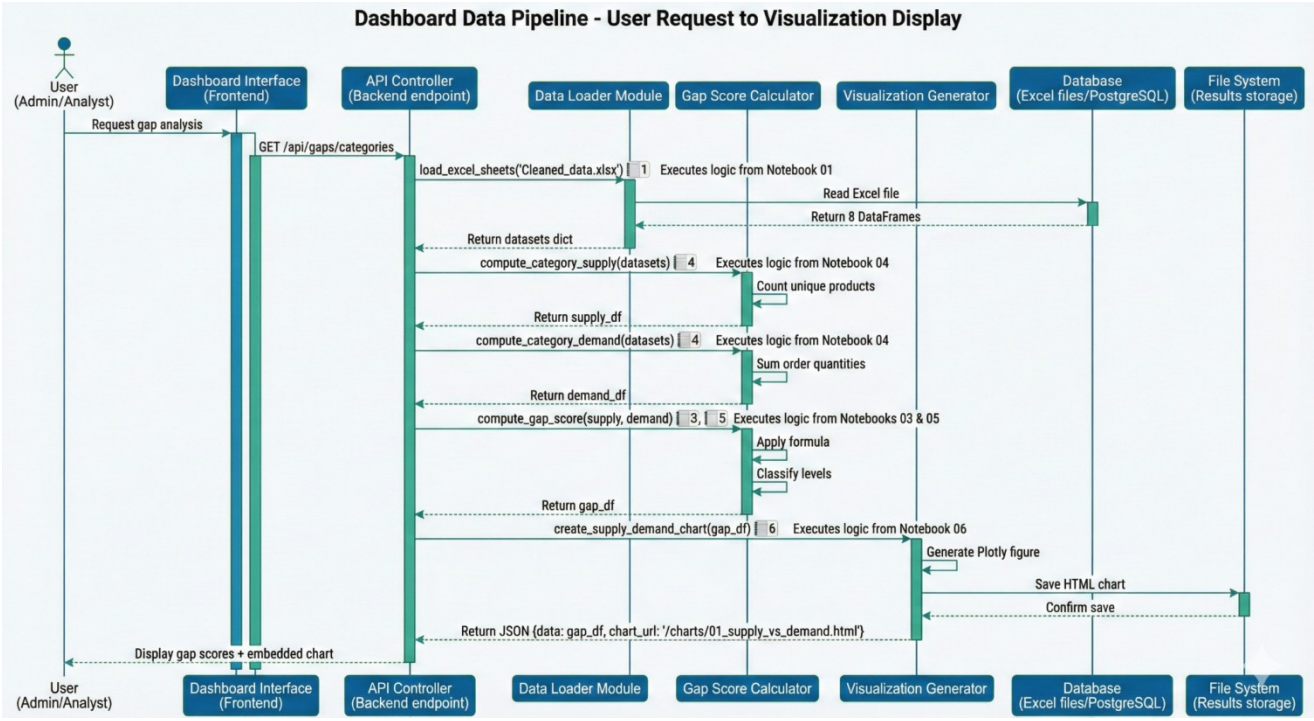
- **Figure 8:** Gap Analysis UML Class Diagram



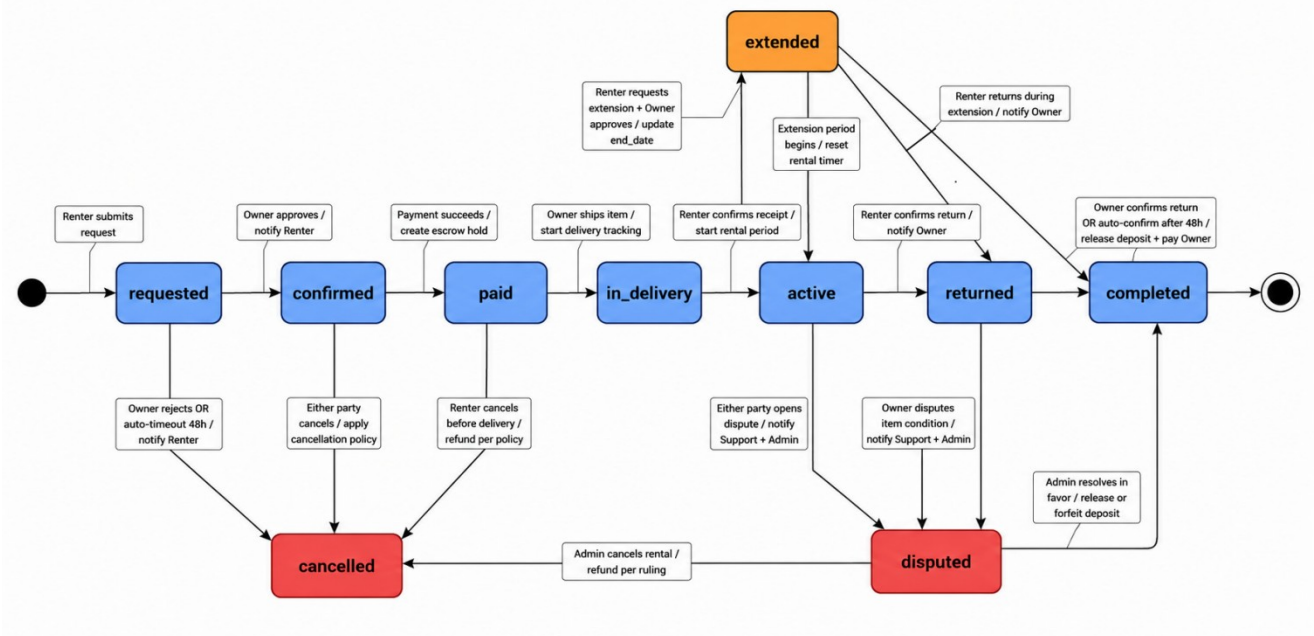
- **Figure 9:** Gap Analysis Data Flow Diagram



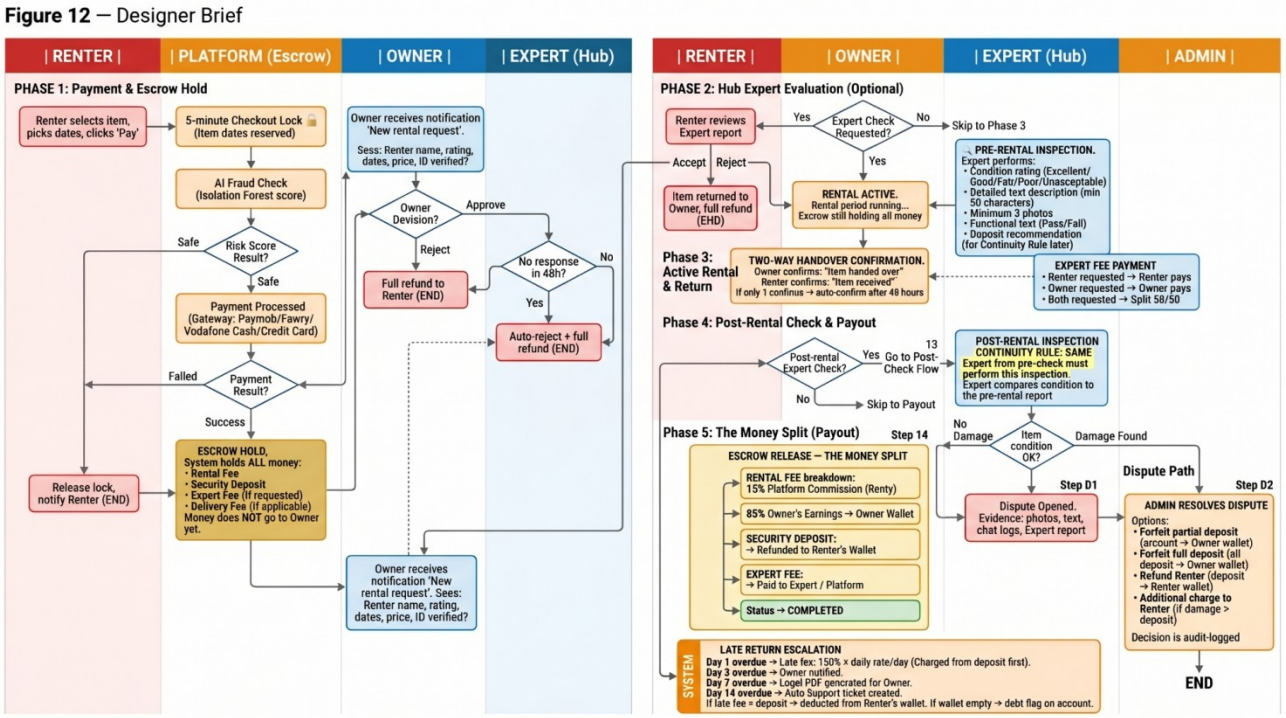
- **Figure 10:** Gap Analysis Sequence Diagram



- **Figure 11:** Entity State Machine for Rental Lifecycle (10-State Diagram)



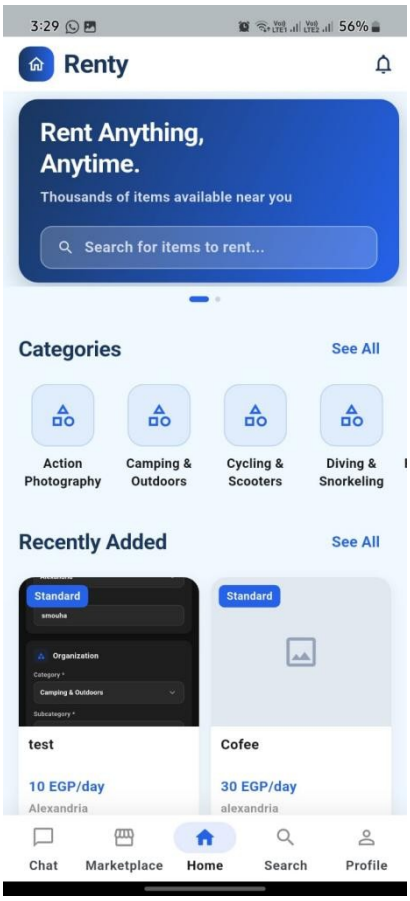
- **Figure 12:** Trust Workflow: Escrow and Renty Points Flow



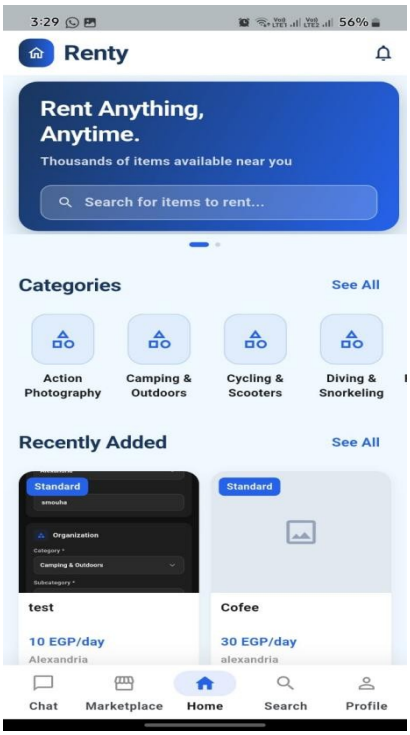
- **Figure 13:** System Database Schema (ERD — Full Relational Diagram)



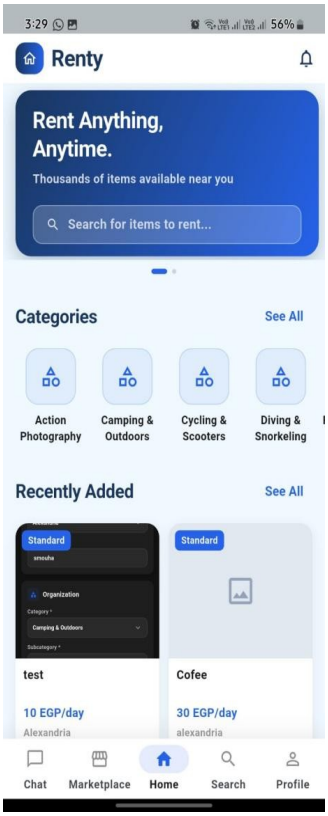
- **Figure 14:** Mobile Application — Home Page and Discovery Flow



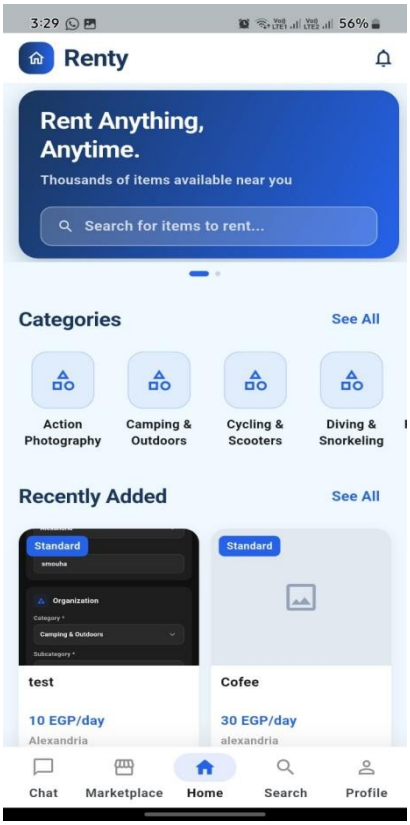
- **Figure 15:** Mobile Application — Item Details and Booking Flow [PLACEHOLDER_FIG_15]



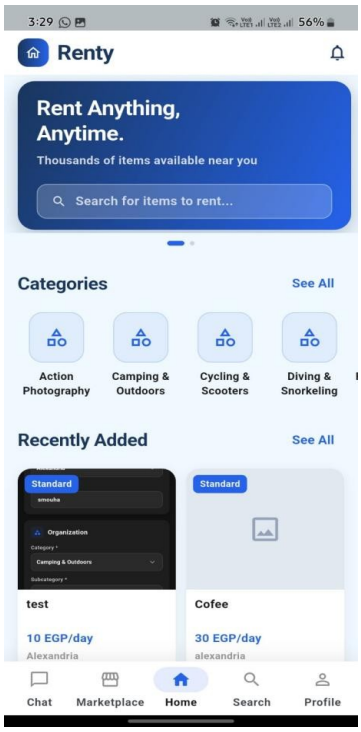
- **Figure 16:** Mobile Application — Active Rental Dashboard [PLACEHOLDER_FIG_16]



- **Figure 17:** Mobile Application — Chat Interface [PLACEHOLDER_FIG_17]



- **Figure 18:** Mobile Application — Identity Verification Flow [PLACEHOLDER_FIG_18]



- **Figure 19:** Web Dashboard — Admin KPI Overview [PLACEHOLDER_FIG_19]

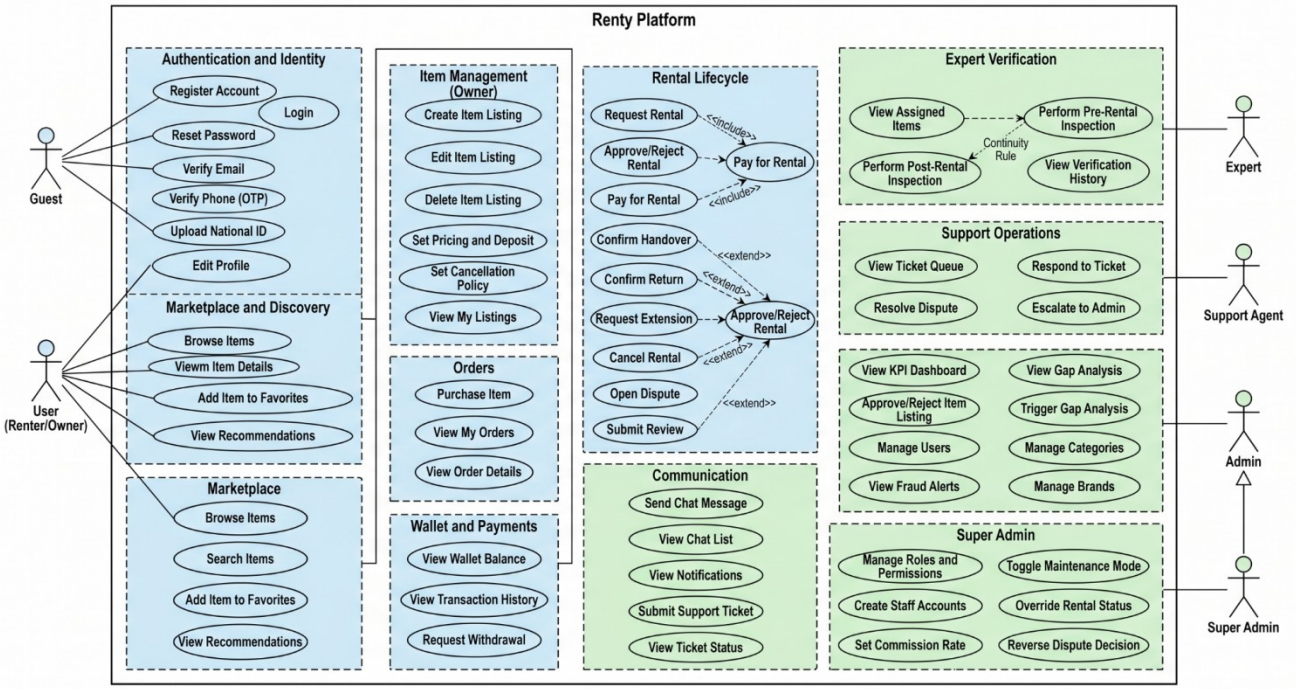


Figure 24 — Designer Brief: System Use Case Diagram (All Actors)

- **Figure 20:** Web Dashboard — Gap Analysis Heatmap [PLACEHOLDER_FIG_20]

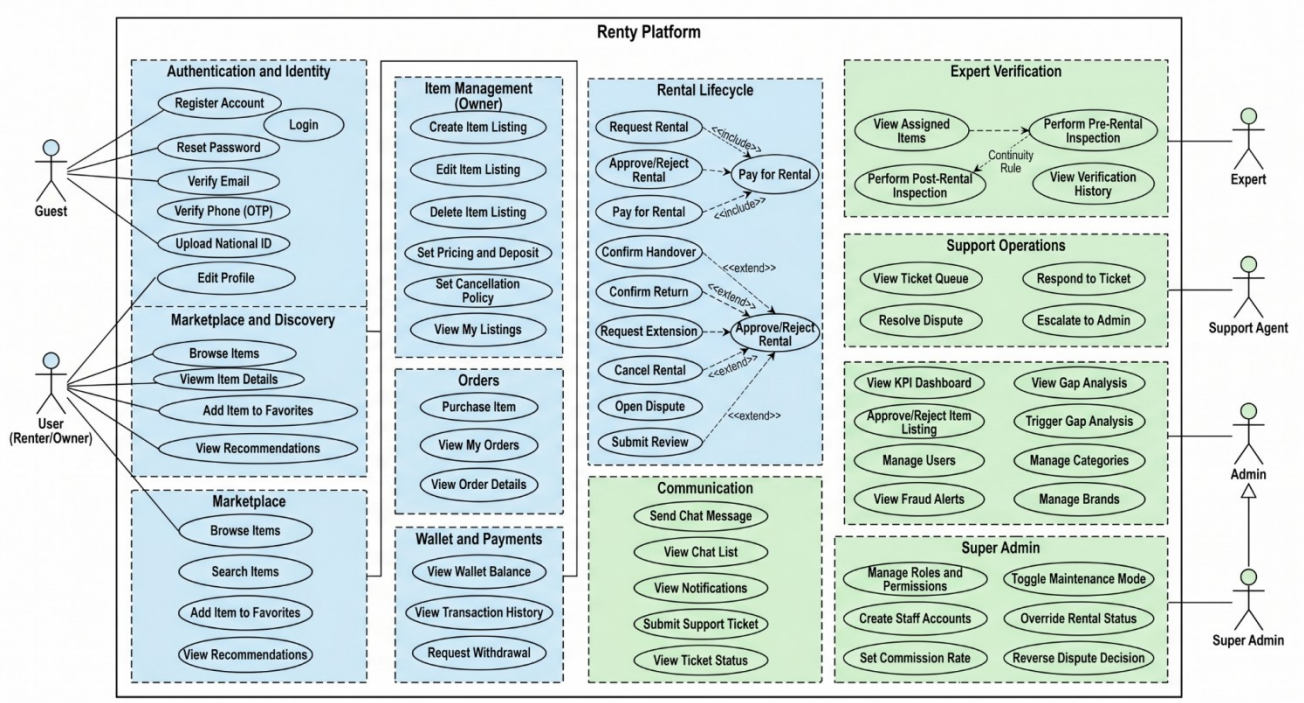
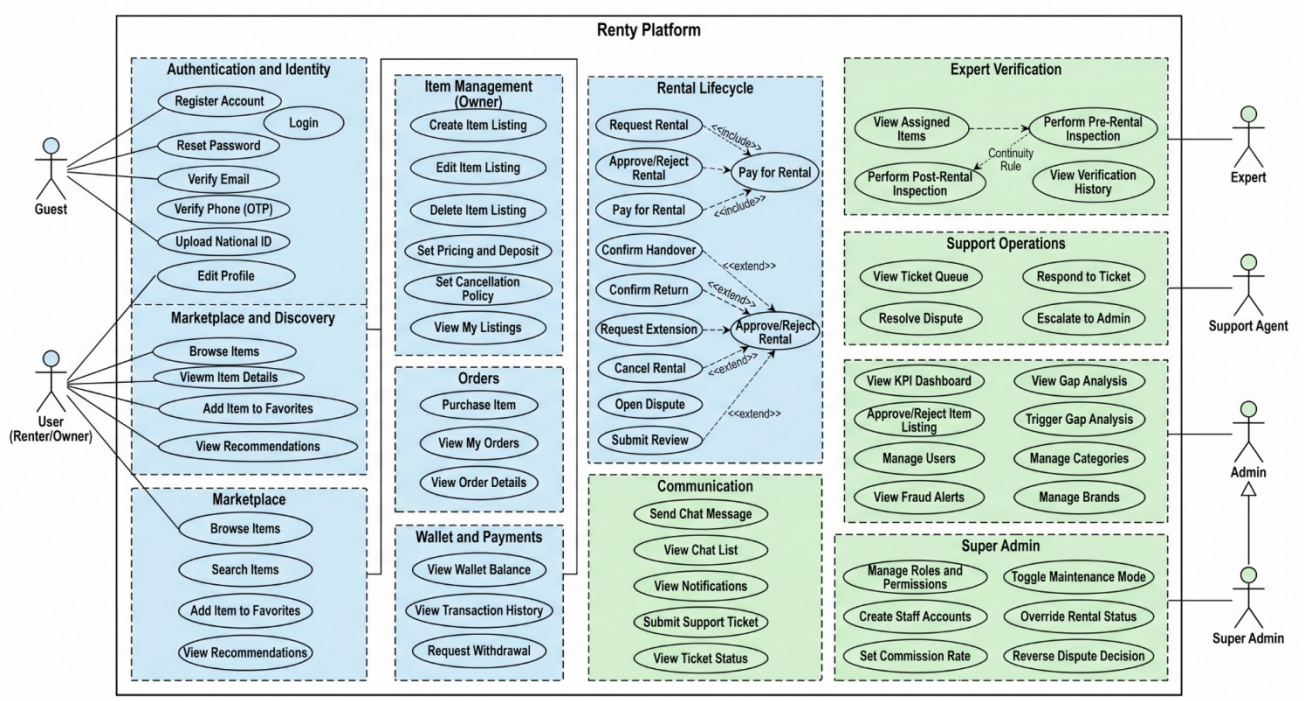


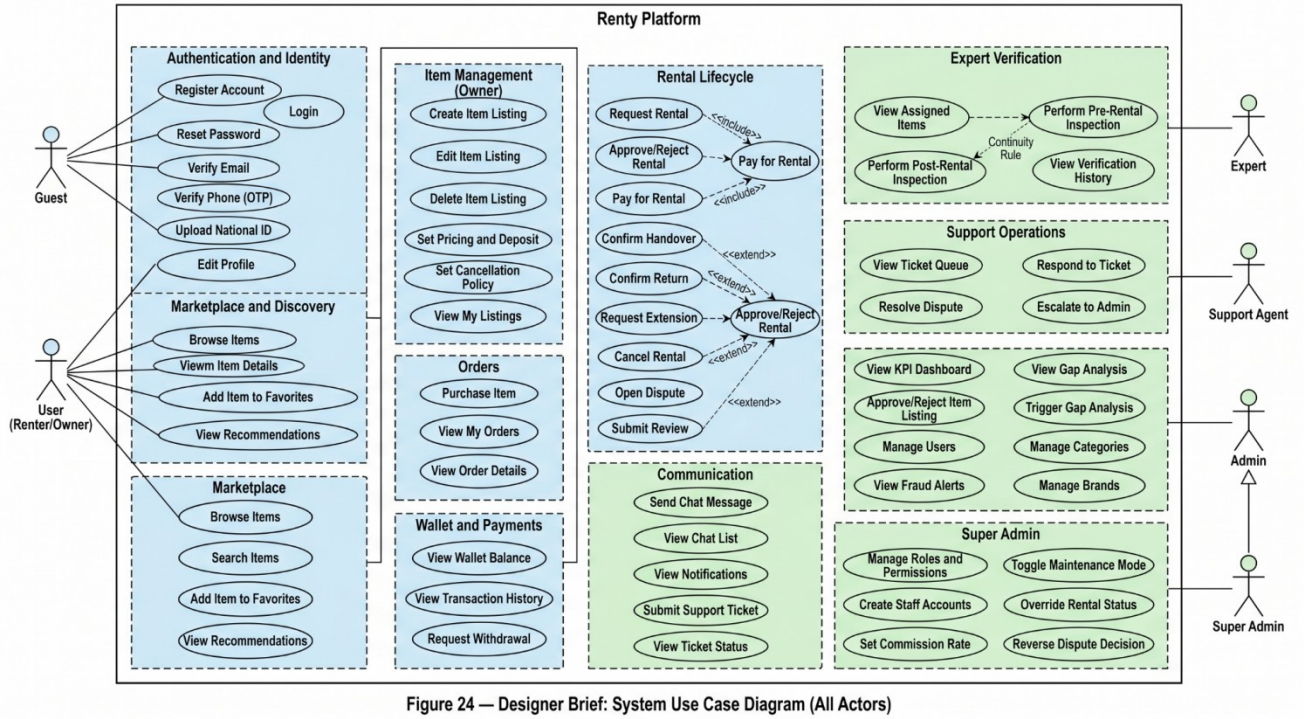
Figure 24 — Designer Brief: System Use Case Diagram (All Actors)

- **Figure 21:** Web Dashboard — Fraud Moderation Panel [PLACEHOLDER_FIG_21]



- Figure 24 — Designer Brief: System Use Case Diagram (All Actors)

- **Figure 22:** Expert Hub Console — Verification Interface [PLACEHOLDER_FIG_22]



- Figure 24 — Designer Brief: System Use Case Diagram (All Actors)

- **Figure 23:** Support Panel — Ticket Management Interface [PLACEHOLDER_FIG_23]

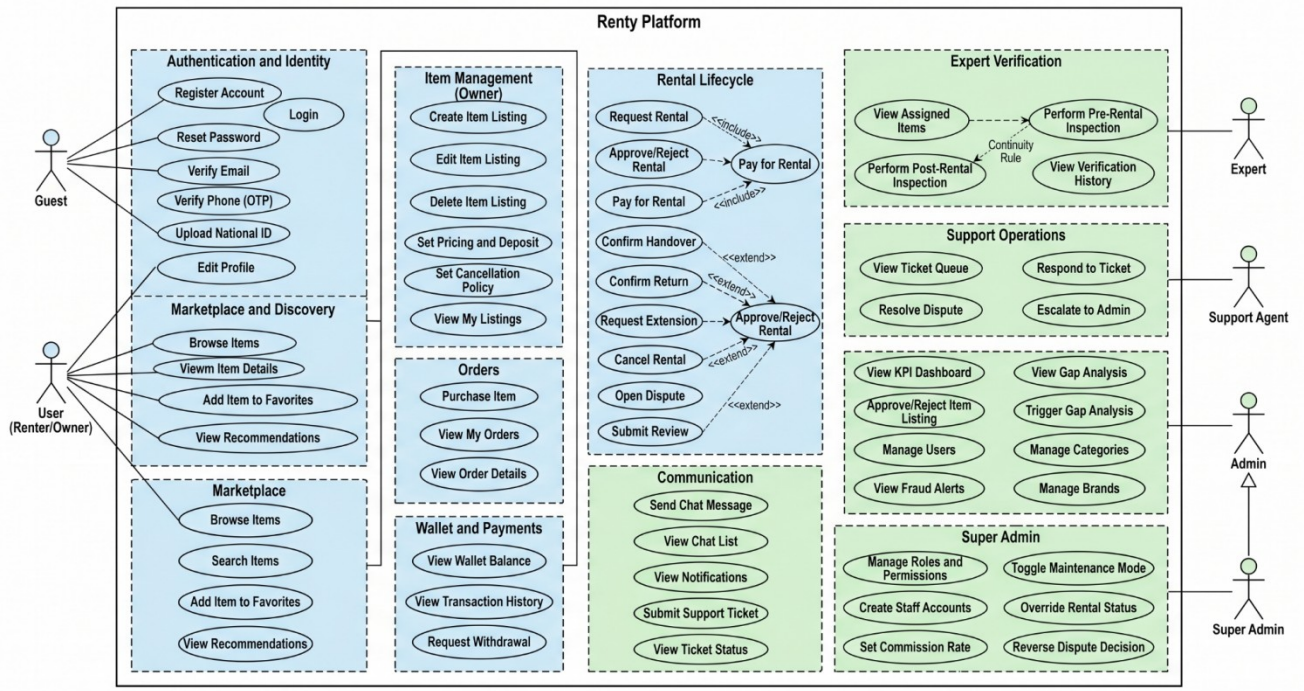


Figure 24 — Designer Brief: System Use Case Diagram (All Actors)

- **Figure 24:** System Use Case Diagram (All Actors)

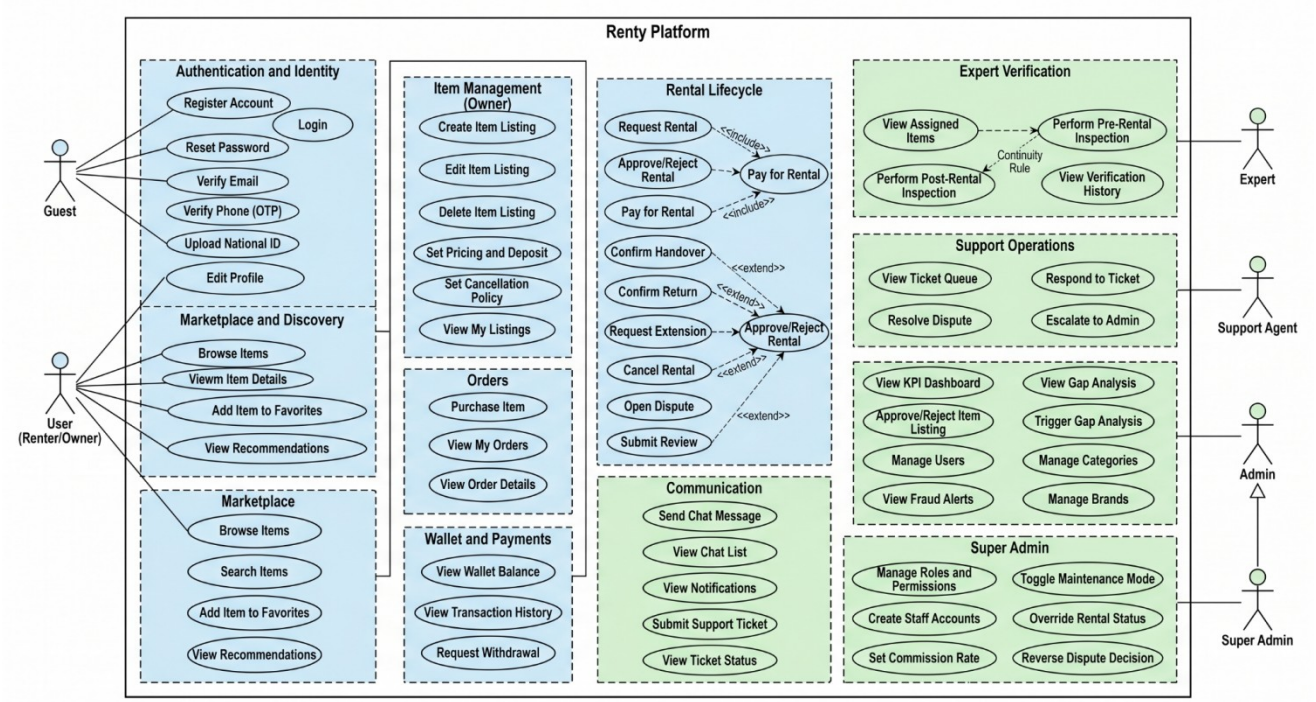


Figure 24 — Designer Brief: System Use Case Diagram (All Actors)

- **Figure 25:** System Activity Diagram — Complete Rental Flow

Figure 25a: Rental Flow — Discovery to Handover

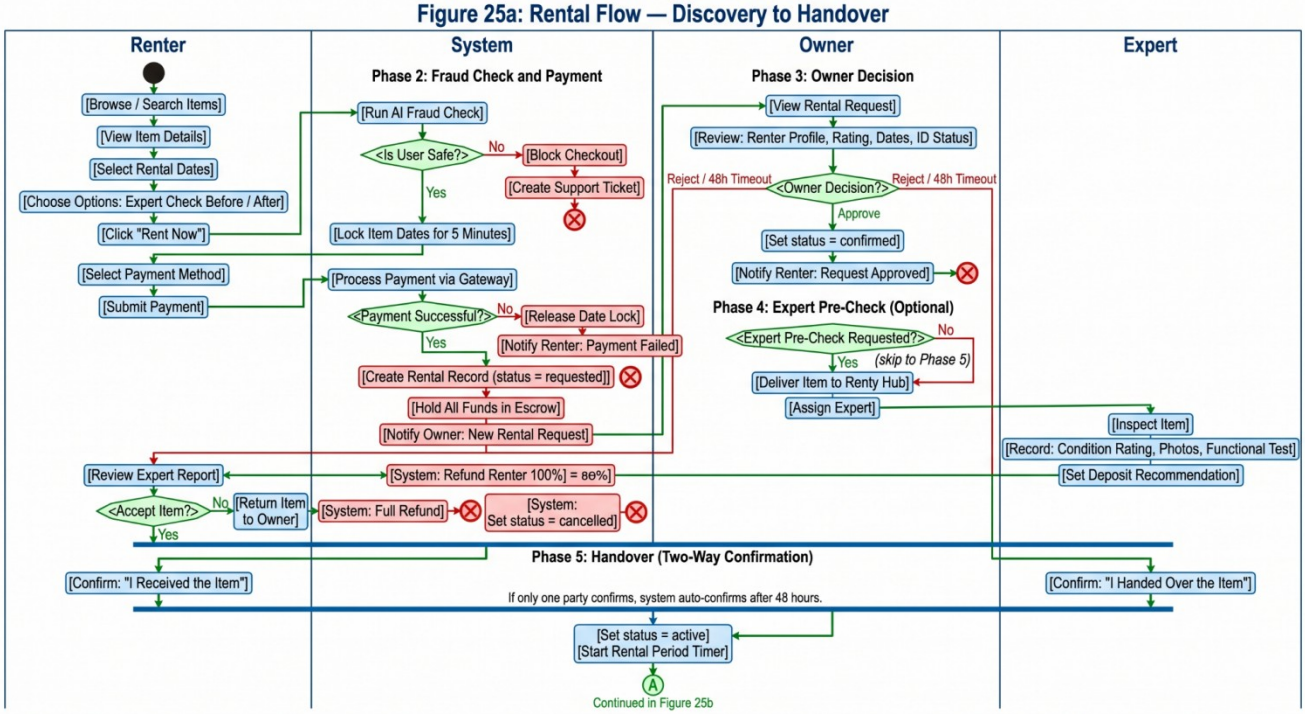
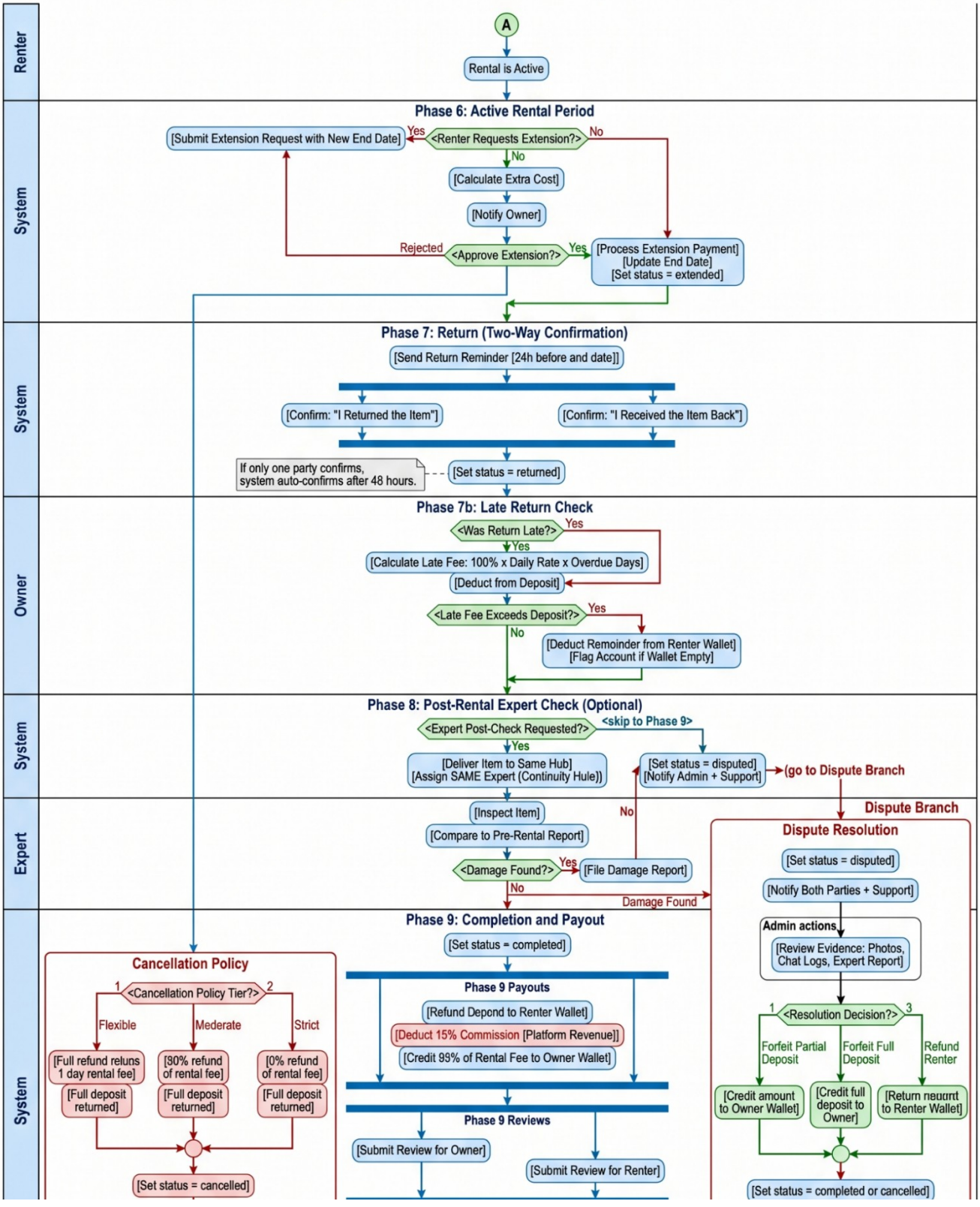
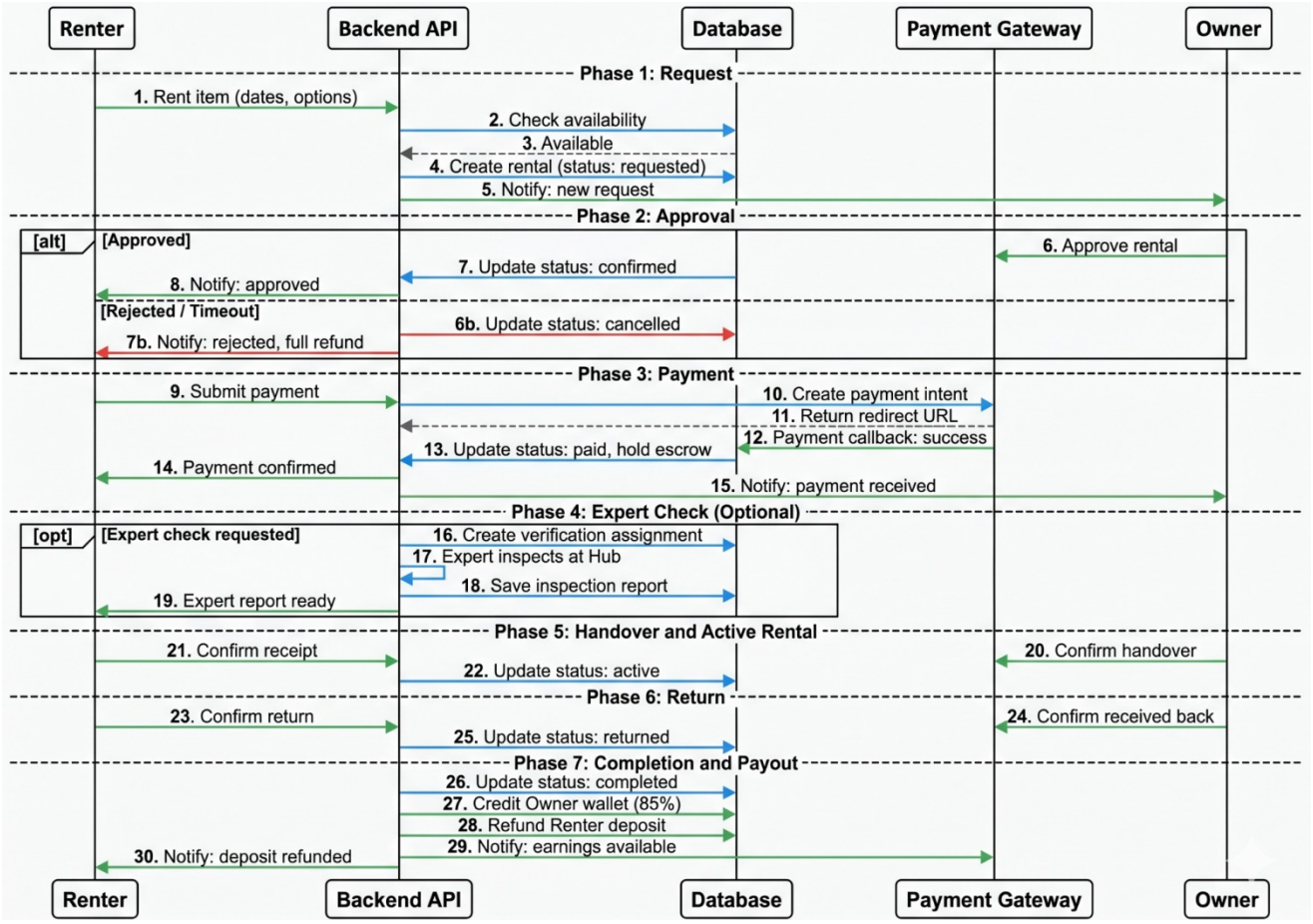


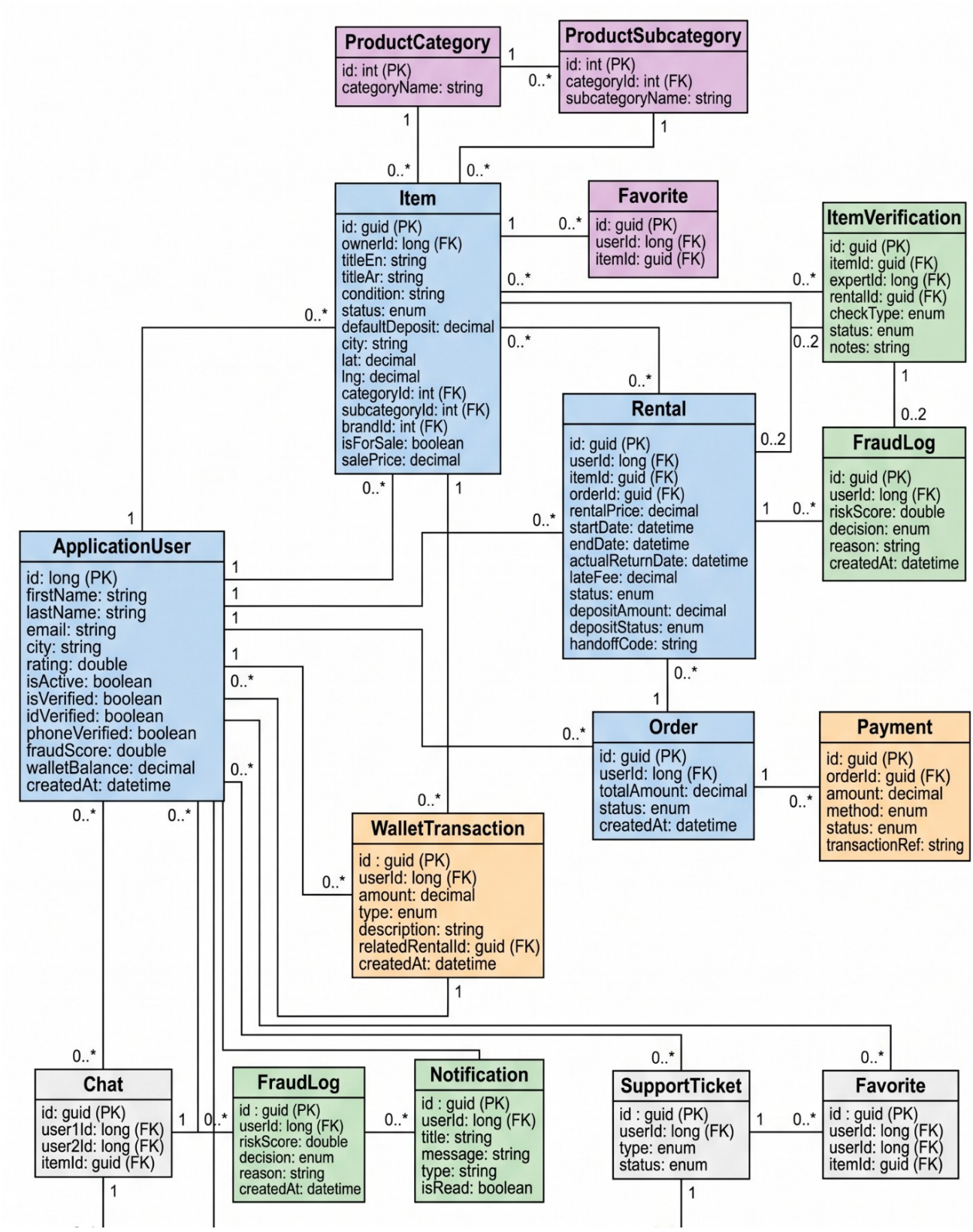
Figure 25b: Active Rental to Completion



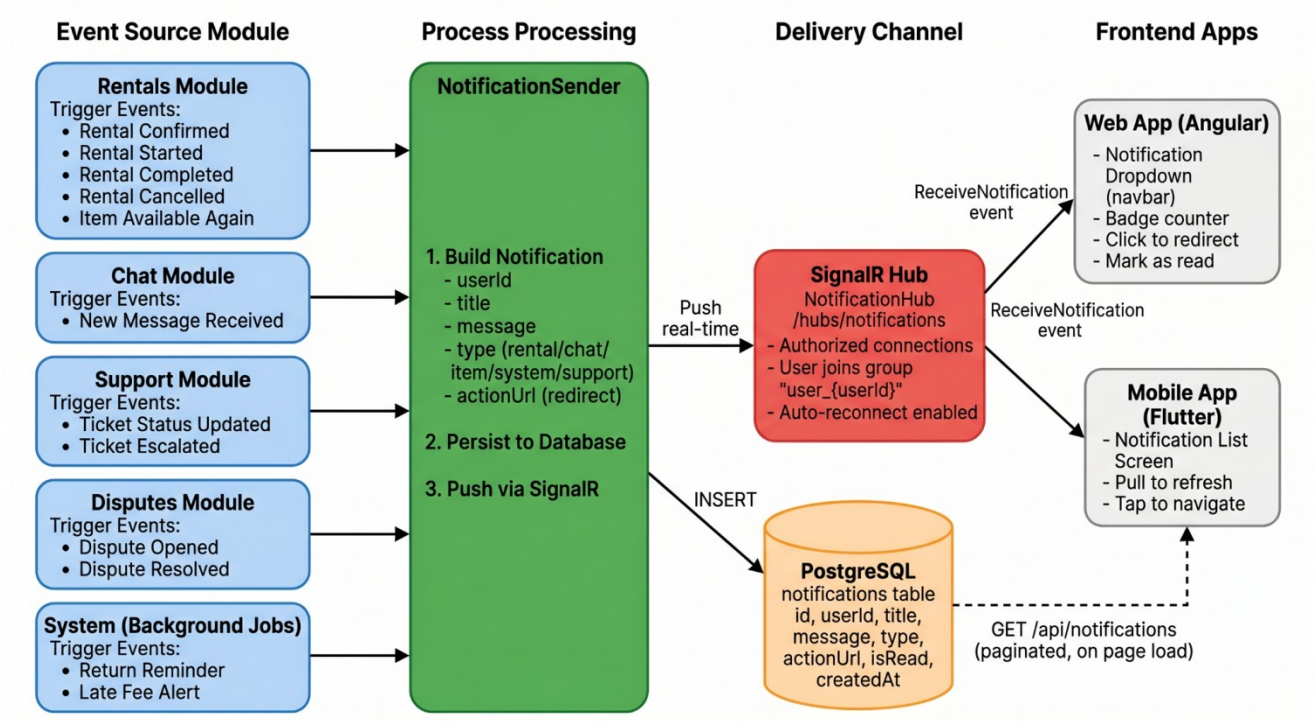
- **Figure 26:** System Sequence Diagram — Rental Request to Completion



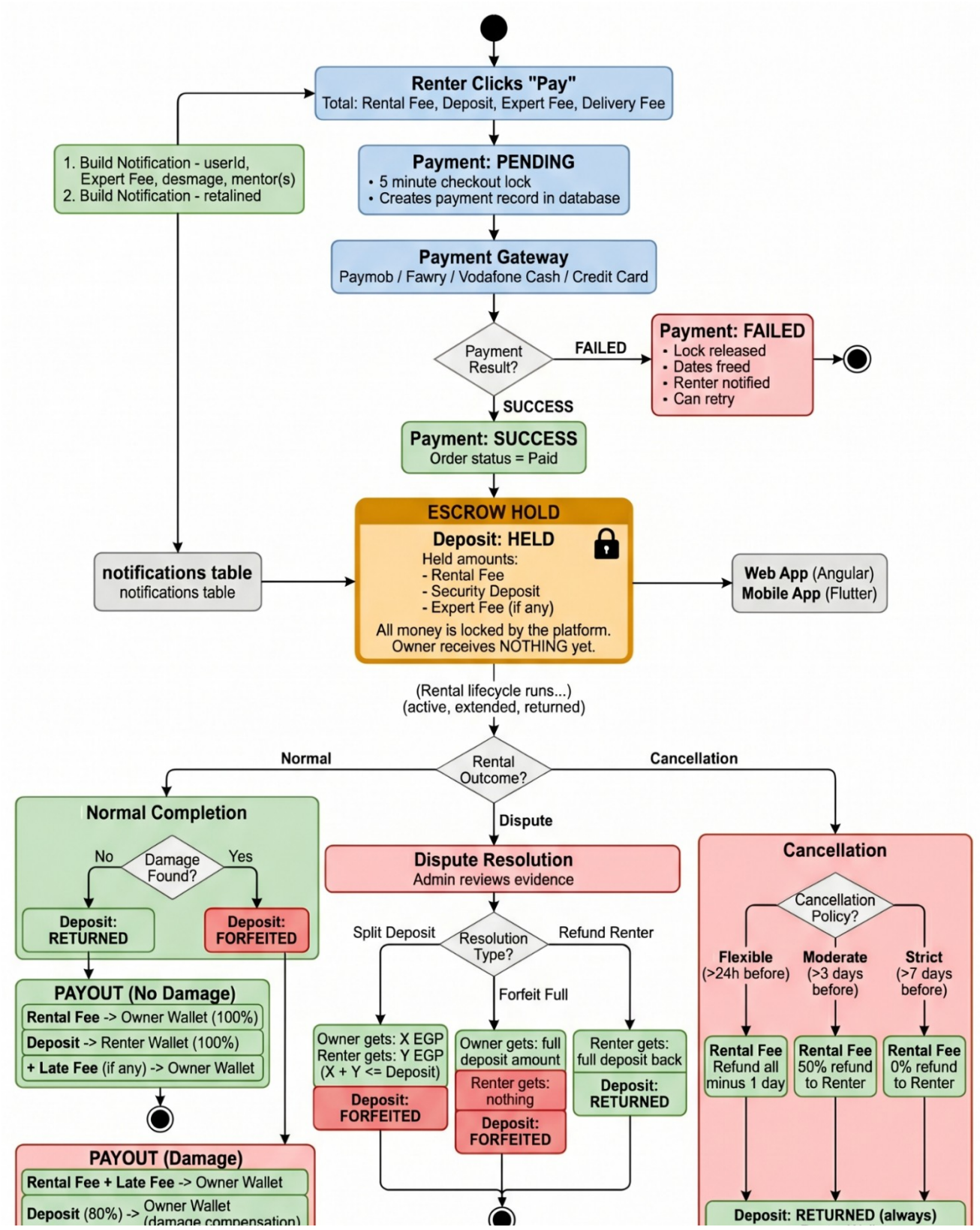
- **Figure 27:** System UML Class Diagram (Backend Models)


ProductCategory
id: int (PK)
categoryName: string
ProductSubcategory
id: int (PK)
categoryId: int (FK)
subcategoryName: string
Item
id: guid (PK)
ownerId: long (FK)
titleEn: string
titleAr: string
condition: string
status: enum
defaultDeposit: decimal
city: string
lat: decimal
lng: decimal
categoryId: int (FK)
subcategoryId: int (FK)
brandId: int (FK)
isForSale: boolean
salePrice: decimal
Favorite
id: guid (PK)
userId: long (FK)
itemId: guid (FK)
ItemVerification
id: guid (PK)
itemId: guid (FK)
expertId: long (FK)
rentalId: guid (FK)
checkType: enum
status: enum
notes: string
Rental
id: guid (PK)
userId: long (FK)
itemId: guid (FK)
orderId: guid (FK)
rentalPrice: decimal
startDate: datetime
endDate: datetime
actualReturnDate: datetime
lateFee: decimal
status: enum
depositAmount: decimal
depositStatus: enum
handoffCode: string
FraudLog
id: guid (PK)
userId: long (FK)
riskScore: double
decision: enum
reason: string
createdAt: datetime
ApplicationUser
id: long (PK)
firstName: string
lastName: string
email: string
city: string
rating: double
isActive: boolean
isVerified: boolean
idVerified: boolean
phoneVerified: boolean
fraudScore: double
walletBalance: decimal
createdAt: datetime
Order
id: guid (PK)
userId: long (FK)
totalAmount: decimal
status: enum
createdAt: datetime
Payment
id: guid (PK)
orderId: guid (FK)
amount: decimal
method: enum
status: enum
transactionRef: string
WalletTransaction
id : guid (PK)
userId: long (FK)
amount: decimal
type: enum
description: string
relatedRentalId: guid (FK)
createdAt: datetime
Chat
id: guid (PK)
user1Id: long (FK)
user2Id: long (FK)
itemId: guid (FK)
FraudLog
id : guid (PK)
userId: long (FK)
riskScore: double
decision: enum
reason: string
createdAt: datetime
Notification
id : guid (PK)
userId: long (FK)
title: string
message: string
type: string
isRead: boolean
SupportTicket
id : guid (PK)
userId: long (FK)
type: enum
status: enum
Favorite
id : guid (PK)
userId: long (FK)
userId: long (FK)
itemId: guid (FK)

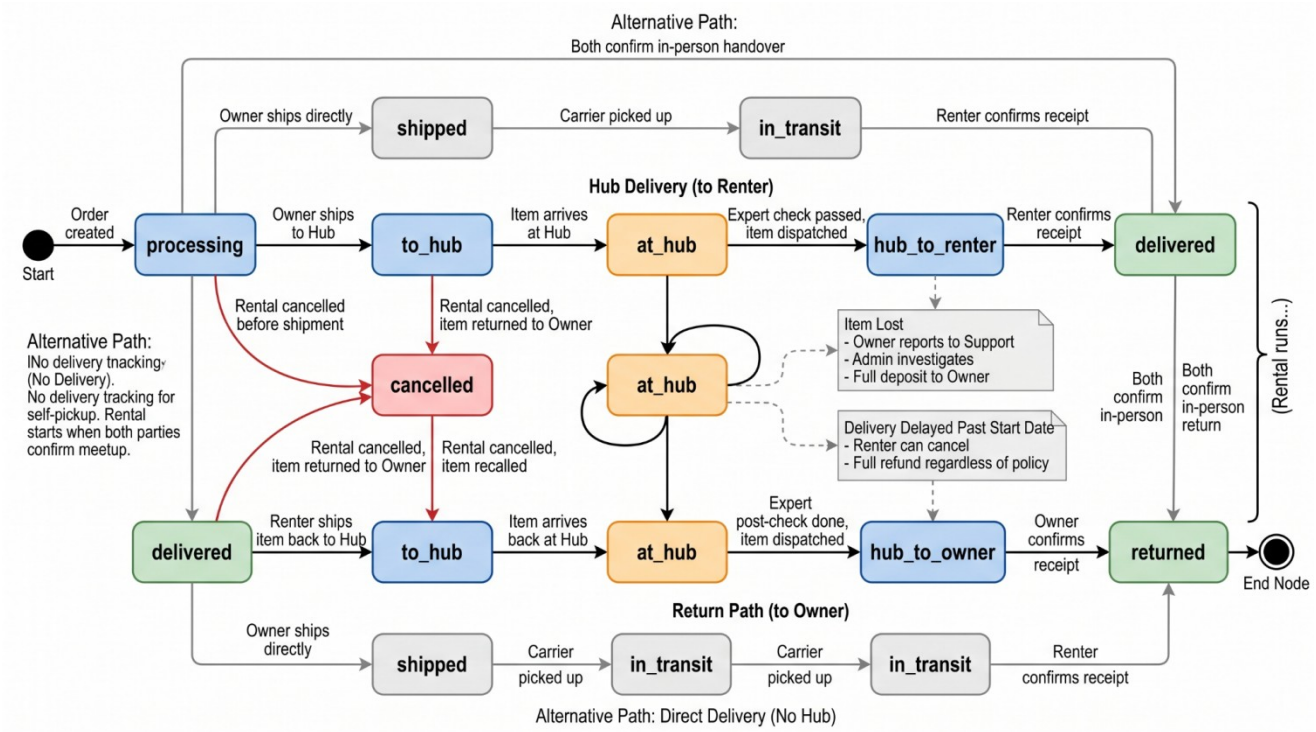
- **Figure 28:** Notification System Event Flow Diagram



- **Figure 29:** Payment and Escrow State Flow

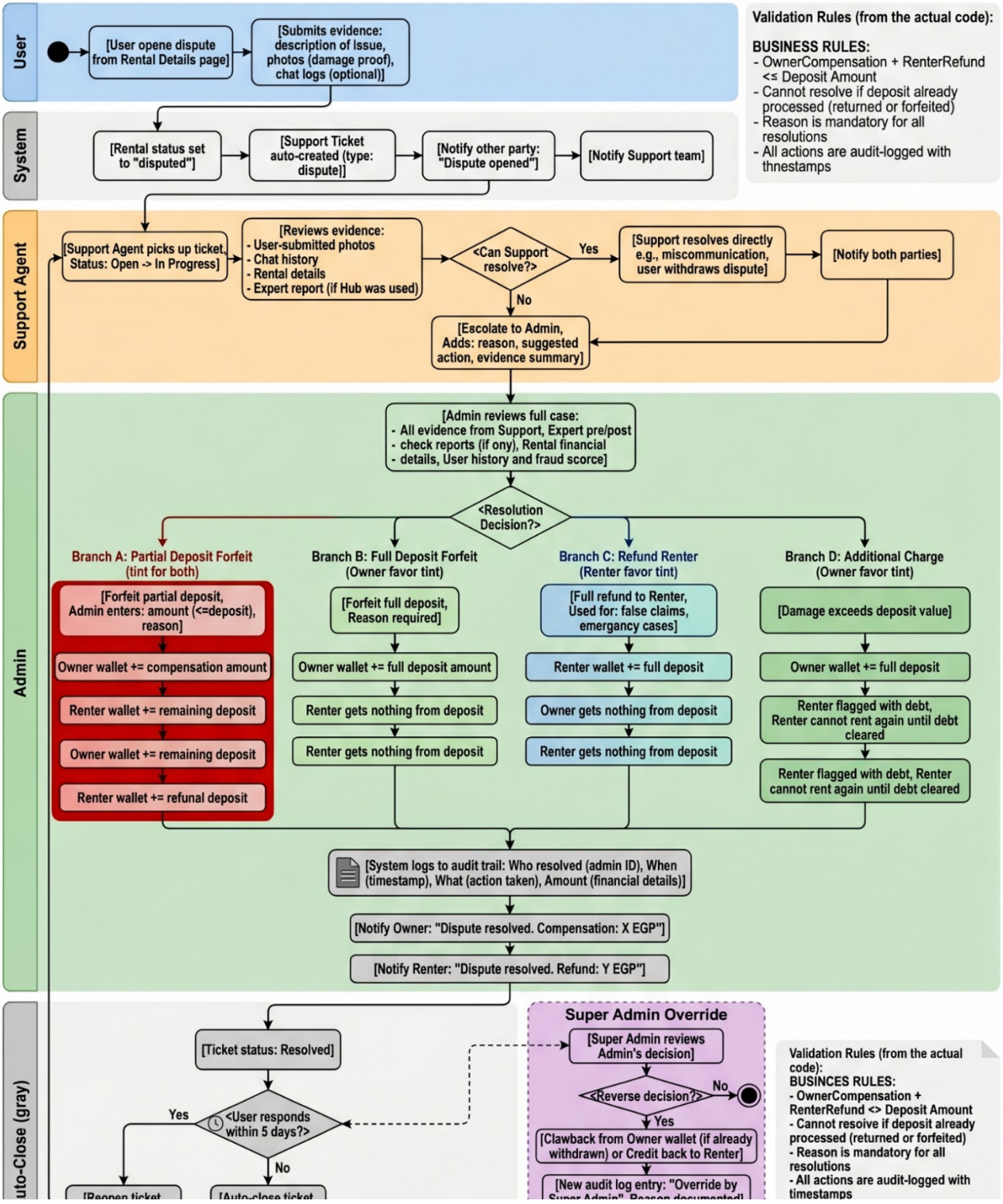

Renter Clicks "Pay"
Total: Rental Fee, Deposit, Expert Fee, Delivery Fee
1. Build Notification - userId, Expert Fee, desmage, mentor(s)
2. Build Notification - retalined
Payment: PENDING
• 5 minute checkout lock
• Creates payment record in database
Payment Gateway
Paymob / Fawry / Vodafone Cash / Credit Card
Payment Result?
FAILED
Payment: FAILED
• Lock released
• Dates freed
• Renter notified
• Can retry
SUCCESS
Payment: SUCCESS
Order status = Paid
ESCROW HOLD
Deposit: HELD
Held amounts:
- Rental Fee
- Security Deposit
- Expert Fee (if any)
All money is locked by the platform.
Owner receives NOTHING yet.
notifications table
notifications table
Web App (Angular)
Mobile App (Flutter)
(Rental lifecycle runs...)
(active, extended, returned)
Rental Outcome?
Normal
Dispute
Cancellation
Normal Completion
No
Damage Found?
Yes
Deposit: RETURNED
Deposit: FORFEITED
PAYOUT (No Damage)
Rental Fee -> Owner Wallet (100%)
Deposit -> Renter Wallet (100%)
+ Late Fee (if any) -> Owner Wallet
PAYOUT (Damage)
Rental Fee + Late Fee -> Owner Wallet
Deposit (80%) -> Owner Wallet
Dispute Resolution
Admin reviews evidence
Split Deposit
Resolution Type?
Refund Renter
Forfeit Full
Owner gets: X EGP
Renter gets: Y EGP
(X + Y <= Deposit)
Deposit: FORFEITED
Owner gets: full deposit amount
Renter gets: nothing
Deposit: FORFEITED
Renter gets: full deposit back
Deposit: RETURNED
Cancellation
Cancellation Policy?
Flexible (>24h before)
Moderate (>3 days before)
Strict (>7 days before)
Rental Fee Refund all minus 1 day
Rental Fee 50% refund to Renter
Rental Fee 0% refund to Renter
Deposit: RETURNED (always)

- **Figure 30:** Delivery Tracking State Machine

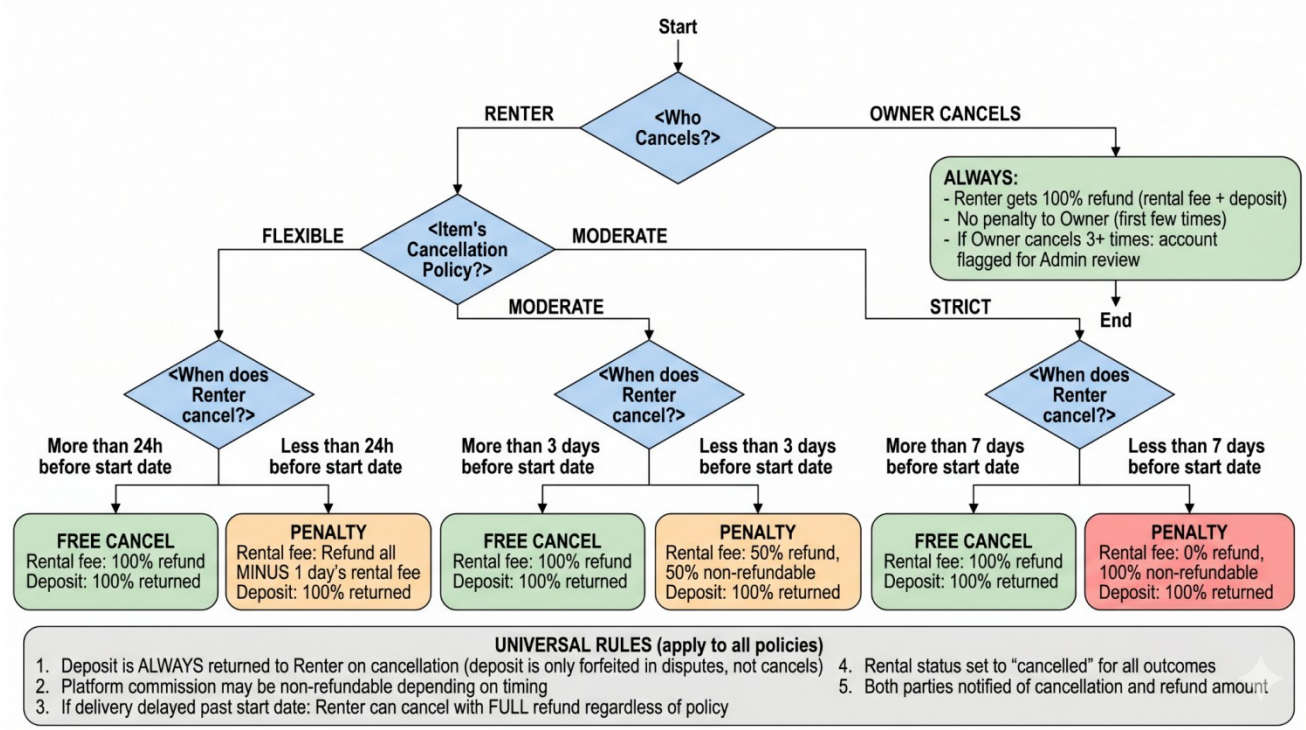


- **Figure 31:** Dispute Resolution Workflow Diagram

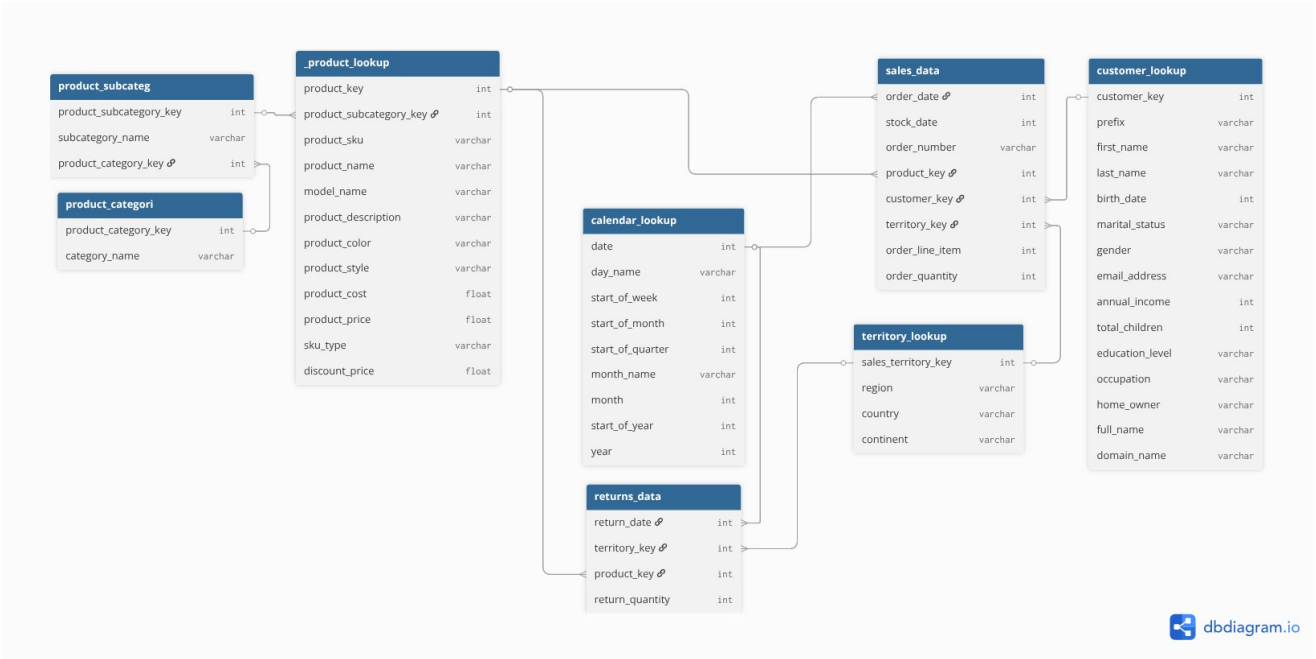
Complete Dispute Resolution Process for Rental Platform (A4)



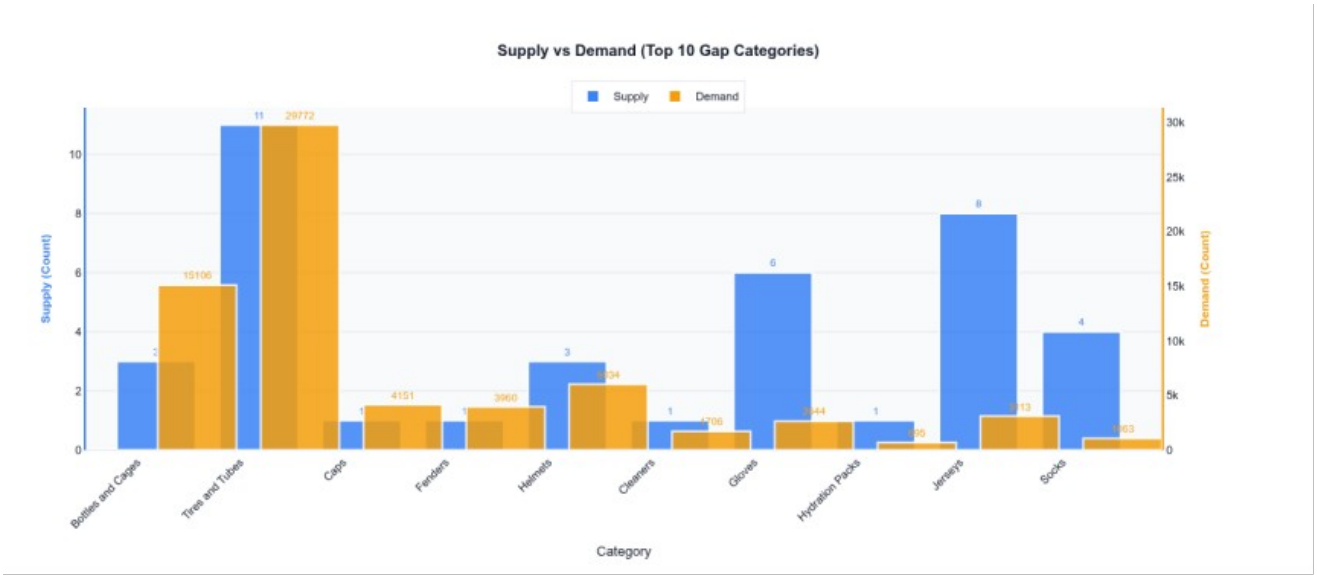
- **Figure 32:** Cancellation Policy Decision Tree



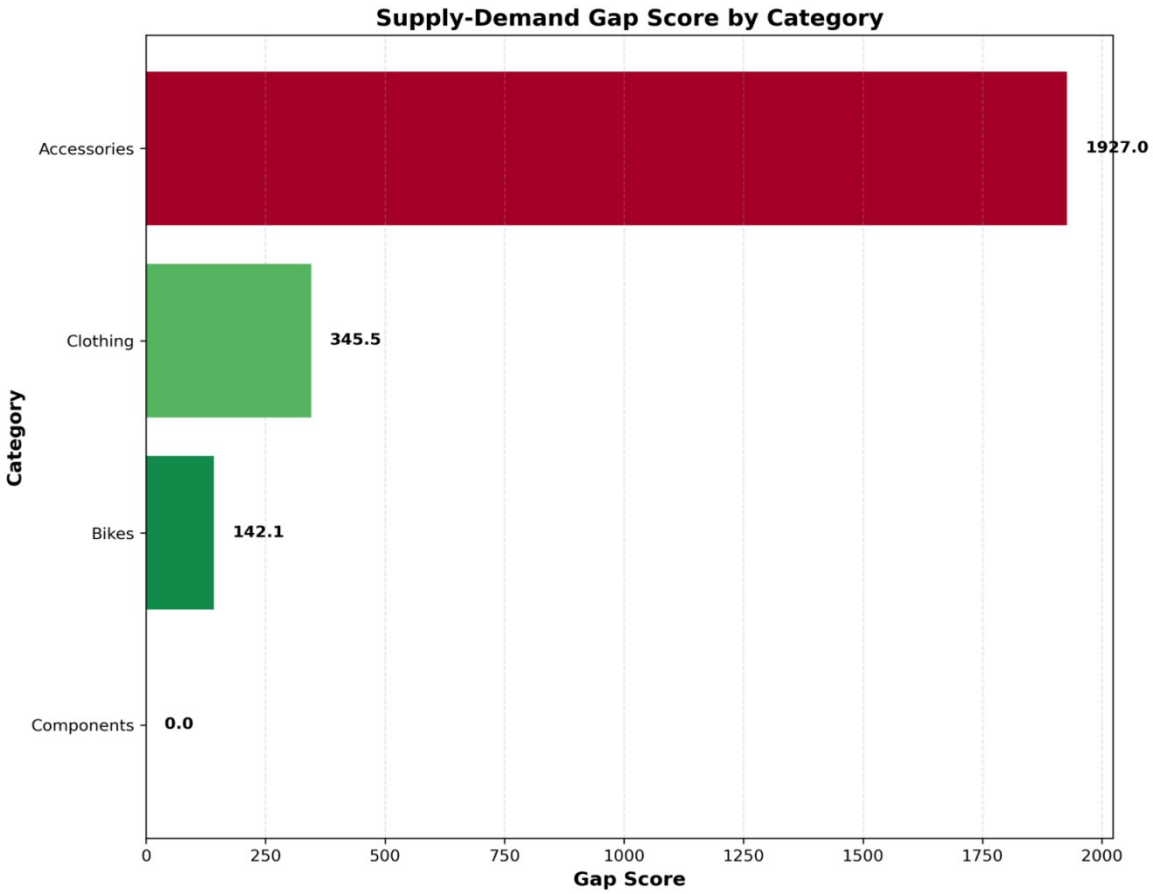
- **Figure 33:** Dataset ERD (Recommendation Training Data)



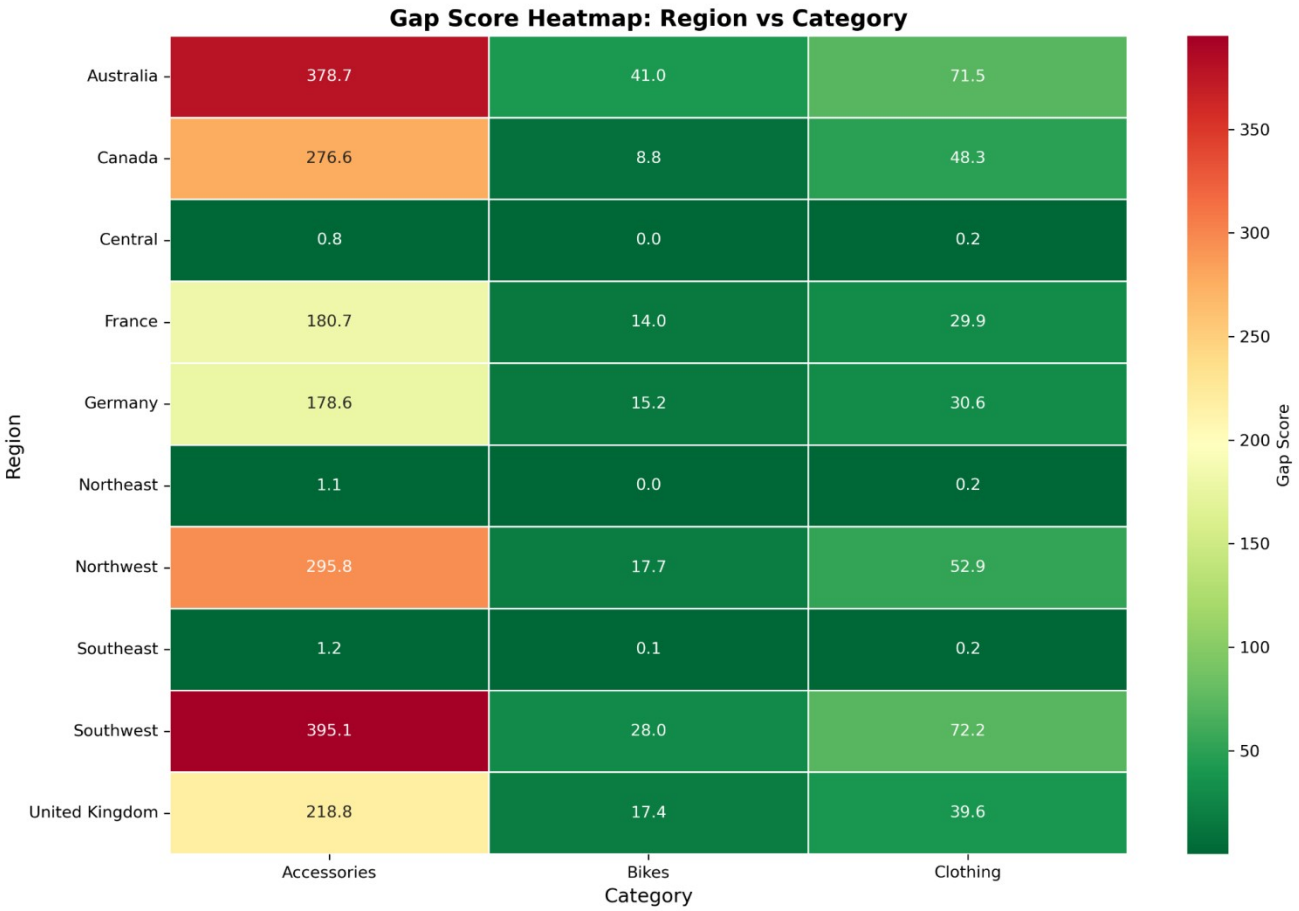
- **Figure 36:** Gap Dashboard — Supply vs. Demand (Top 10 Categories)



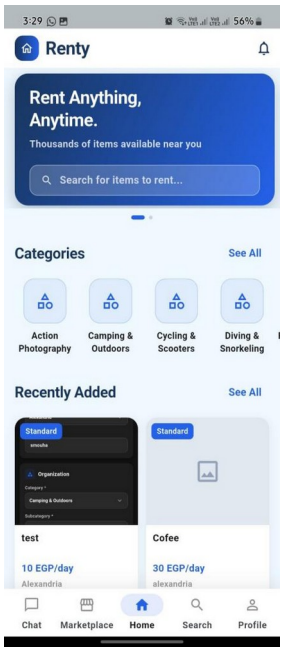
- **Figure 37:** Gap Score Severity Chart



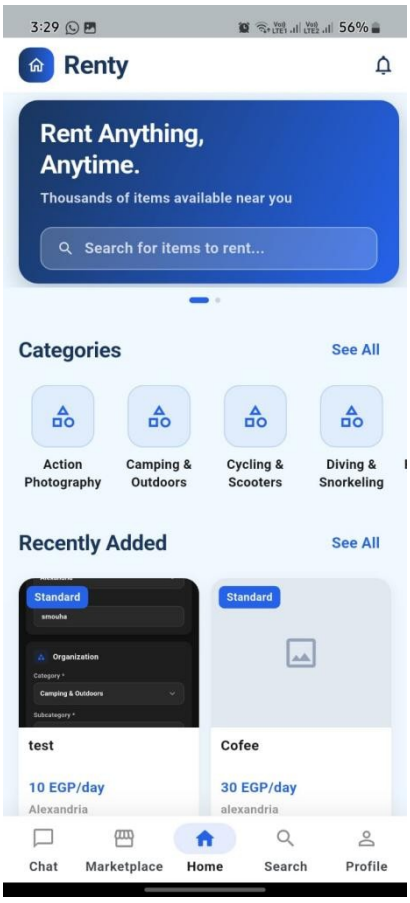
- **Figure 38:** Gap Heatmap by Subcategory



- **Figure 39:** KPI Monitoring — Active Alerts and Flags [PLACEHOLDER_FIG_39]



- **Figure 40:** Gantt Chart — Project Timeline (Both Semesters) [PLACEHOLDER_FIG_40]



List of Tables

(Please insert actual tables at these placeholders during final assembly)

- **Table 1:** Dataset Characteristics for Recommendation Engine

Metric	Value	Implication
Total Transactions	55,666	Sufficient training data
Unique Users	17,293	Large user base
Unique Products	130	Manageable catalog
Sparsity	97.52%	Hybrid approach needed
Avg Orders/User	3.22	Limited interaction history
Date Range	Jan 2020 - Jun 2022	Temporal trends present

- **Table 2:** Dataset Overview — Columns and Types

Column	Type	Description	Missing
OrderDate	DateTime	Transaction timestamp	0
CustomerKey	Integer	Unique user identifier	0
ProductKey	Integer	Unique item identifier	0
OrderQuantity	Integer	Items purchased (1-3)	0
AnnualIncome	Integer	$10K - $170K	0
TotalChildren	Integer	0-5 children	0
Gender	Categorical	M/F	0
Occupation	Categorical	5 categories	0
ModelName	String	Product model	0

ProductDescription	Text	Product details	0

- **Table 4:** Key Entity Relationships in Training Data

Relationship	From Table	To Table	Type	Join Key
Customer places Order	Customers	Sales	1 : N	CustomerKey
Order contains Product	Sales	Products	N : 1	ProductKey
Product belongs to Subcategory	Products	Subcategories	N : 1	ProductSubcategory Key
Return references Product	Returns	Products	N : 1	ProductKey
Return occurs in Territory	Returns	Territories	N : 1	TerritoryKey
Sale occurs in Territory	Sales	Territories	N : 1	TerritoryKey

- **Table 5:** Regularization Strategy and Parameters

Strategy	Parameter	Value	Purpose
L2 (Ridge) on Users	user_alpha	0.00001	Prevents overfitting on users with few interactions
L2 (Ridge) on Items	item_alpha	0.00001	Prevents overfitting on items with few interactions
Epoch Control	epochs	60	Stops before overfitting (validated via train/test gap)
Feature Hashing	max_features (TF-IDF)	100	Limits text feature dimensionality
Z-Score Normalization	RFM features	μ=0, σ=1	Prevents high-magnitude features from dominating

- **Table 6:** Training Configuration (LightFM)

Parameter	Value	Justification
Loss Function	WARP	Optimized for top-N ranking with implicit feedback
No. Components	50	Balances model capacity and overfitting risk
Learning Rate	0.05	Stable convergence without oscillation
Item Alpha (L2)	0.00001	Light regularization preserving accuracy
User Alpha (L2)	0.00001	Light regularization preserving accuracy
Epochs	60	Convergence achieved (validated via loss curve)
Threads	4	Parallel processing for training speed
Random State	42	Reproducibility

- **Table 7:** Hyperparameter Tuning Results (6 Configurations)

Config	Components	Regularization	Precision@10	Overfit Gap	Selected?
Baseline	50	0.0	0.1217	0.0029	No
Light Reg	50	0.00001	0.1215	0.0004	**Yes**
Medium Reg	50	0.0001	0.1209	-0.0035	No
High Comp	70	0.0001	0.1193	-0.0137	No
Low Comp	30	0.0001	0.1211	0.0156	No
Strong Reg	50	0.001	0.1166	-0.0159	No

- **Table 8:** Final Model Performance — Baseline vs. Hybrid

Metric	Baseline (CF-only)	Hybrid (LightFM)	Improvement
Test AUC	0.7497	0.8421	+12.3%
Precision@5	0.1043	0.1511	+44.9%
Precision@10	0.0938	0.1208	+28.8%
Precision@20	0.0664	0.0868	+30.7%
Recall@5	0.1932	0.2708	+40.2%
Recall@10	0.3452	0.4410	+27.8%
Recall@20	0.4892	0.6416	+31.2%
Overfitting Gap	0.2501	0.0479	-80.8%

- **Table 9:** API Response Format for Recommendations

Field	Type	Description
recommendations	array	Ranked list of recommended items
recommendations[].rank	int	Position in ranked list (1 = best)
recommendations[].productKey	int	Unique product identifier
recommendations[].modelName	string	Product display name
recommendations[].description	string	Product description text
recommendations[].score	float	Predicted relevance score (0.0–1.0)

- **Table 11:** Fraud Score Thresholds and Actions

Score Range	Decision	System Action	Admin Options
0.0 – 0.49	safe	Normal access, transaction proceeds	—
0.50 – 0.79	flagged	Admin notified via dashboard, user monitored	Dismiss, Whitelist, Suspend
0.80 – 1.0	blocked	Checkout locked, auto Support ticket created	Unblock, Whitelist, Suspend

- **Table 12:** Gap Severity Classification Metrics

Severity Level	Gap Score Range	Interpretation	Recommended Action
Low Gap	< 50	Supply roughly meets demand	Monitor
Moderate Gap	50 – 99	Demand moderately exceeds supply	Incentivize listings
High Gap	100 – 199	Significant undersupply	Targeted owner outreach
Critical Gap	≥ 200	Severe undersupply	Priority acquisition campaign

- **Table 13:** Gap Severity Distribution Across Subcategories

Severity	Count	Percentage	Example Subcategories
Low Gap	~15	~38%	Road Bikes, Jerseys, Caps
Moderate Gap	~10	~26%	Mountain Bikes, Gloves, Socks
High Gap	~8	~21%	Touring Bikes, Helmets
Critical Gap	~6	~15%	Bike Stands, Hydration Packs, Fenders

- **Table 14:** Weekly Snapshot JSON Schema

Field	Type	Description
id	int (PK)	Auto-incrementing record ID
category	string	Product subcategory name
supply	int	Count of unique available products
demand	int	Total quantity sold/requested in period
gap_score	float	Computed gap ratio
gap_status	string	Severity label (Low/Moderate/High/Critical)
normalized_gap	float	Min-max normalized score (0.0–1.0)
batch_id	guid	Groups all results from a single pipeline run
created_at	datetime	Timestamp of pipeline execution

- **Table 15:** API Endpoints Overview (All Modules)

Module	Base Route	Key Endpoints	Auth
Auth	/api/v1/auth	POST /register, POST /login, POST /refresh, POST /logout	Public
Account	/api/v1/account	GET /me, PUT /me, PUT /change-password, POST /verify-phone	User
Items	/api/v1/items	GET /, GET /{id}, POST /, PUT /{id}, DELETE /{id}, GET /my-items	User
Rentals	/api/v1/rentals	POST /, GET /my-requests, PUT /{id}/confirm, PUT /{id}/complete, PUT /{id}/cancel	User
Orders	/api/v1/orders	GET /{id}, GET /my-orders	User

Payments	/api/v1/payments	POST /create-intention, POST /webhook	Mixed
Chat	/api/v1/chats	GET /, GET /{id}/messages, POST /{id}/messages	User
Reviews	/api/v1/reviews	POST /, GET /items/{id}/reviews	User
Favorites	/api/v1/favorites	POST /{itemId}, DELETE /{itemId}, GET /me/favorites	User
Notifications	/api/v1/notifications	GET /, PATCH /{id}/read, PATCH /read-all	User
Wallet	/api/v1/wallet	GET /balance, GET /transactions, POST /withdraw	User
Support	/api/v1/support	POST /, GET /my-tickets, GET /{id}, POST /{id}/messages	User
Disputes	/api/v1/disputes	POST /open, PUT /resolve	Admin
Delivery	/api/v1/delivery	GET /orders/{id}/delivery, PUT /{id}/status	User
Search	/api/v1/search	GET /items?q=&city=&category=	Public
Recommendations	/api/v1/ recommendations	GET /{userId}	User
Verification	/api/v1/verifications	GET /expert/assigned, PUT /{id}/verdict	Expert
Admin Items	/api/v1/admin/items	GET /pending, PUT /{id}/approve, PUT /{id}/reject	Admin
Admin Users	/api/v1/admin/users	GET /, PUT /{id}/suspend, PUT /{id}/activate	Admin
Admin KPIs	/api/v1/admin/kpis	GET /latest, GET /by-period	Admin
Admin Settings	/api/v1/admin/settings	GET /, PUT /	SuperAdmin

Admin Roles	/api/v1/admin/roles	GET /, POST /, PUT /{id}, DELETE /{id}	SuperAdmin
Analytics	/api/v1/analytics	GET /gap, GET /fraud, GET /heatmap, GET /dead-stock	Admin
Categories	/api/v1/categories	GET /, GET /{id}/subcategories	Public

- **Table 16:** User Roles and RBAC Permission Matrix

Permission	Renter	Owner	Expert	Support	Admin	SuperAdmin
Browse and search items	Yes	Yes	No	No	Yes	Yes
Create rental request	Yes	No	No	No	No	No
List items for rent/sale	No	Yes	No	No	No	No
Approve/reject rental	No	Yes	No	No	No	No
Chat with other users	Yes	Yes	No	Yes	No	No
Leave reviews	Yes	Yes	No	No	No	No
Open support ticket	Yes	Yes	Yes	No	No	No
Perform item verification	No	No	Yes	No	No	No
Respond to support tickets	No	No	No	Yes	Yes	Yes
Escalate tickets to Admin	No	No	No	Yes	No	No
Resolve disputes	No	No	No	No	Yes	Yes

Approve/reject item listings	No	No	No	No	Yes	Yes
Suspend/unsuspend users	No	No	No	No	Yes	Yes
View KPI dashboard	No	No	No	No	Yes	Yes
View analytics (gap, fraud)	No	No	No	No	Yes	Yes
Manage system settings	No	No	No	No	No	Yes
CRUD staff accounts	No	No	No	No	No	Yes
Override Admin decisions	No	No	No	No	No	Yes
Toggle maintenance mode	No	No	No	No	No	Yes

- **Table 17:** Database Tables Summary (PostgreSQL Schema)

Table Name	Purpose	Key Columns	Relationships
users	User accounts (all roles)	id, email, role, wallet_balance, fraud_score	Central hub
items	Rental/sale listings	id, owner_id, title, status, deposit	FK → users
item_pricing	Multi-tier pricing (daily/weekly/monthly)	id, item_id, period, price	FK → items
item_images	Item photo gallery	id, item_id, url	FK → items
orders	Payment grouping container	id, user_id, total, status	FK → users
order_items	Items within an order	id, order_id, item_id, price	FK → orders, items

payments	Payment transaction records	id, order_id, method, status, ref	FK → orders
rentals	Core rental lifecycle	id, user_id, item_id, status, deposit_status	FK → users, items, orders
delivery_tracking	Shipment status tracking	id, order_id, status, location	FK → orders
reviews	Post-rental ratings	id, rental_id, reviewer_id, rating	FK → rentals, users
chats	Conversation threads	id, user1_id, user2_id, item_id	FK → users, items
chat_messages	Individual messages	id, chat_id, sender_id, message	FK → chats, users
notifications	Push notification log	id, user_id, title, type, is_read	FK → users
support_tickets	Help desk tickets	id, user_id, type, status	FK → users
support_messages	Ticket conversation	id, ticket_id, sender_id, message	FK → support_tickets
favorites	User's wishlist	id, user_id, item_id	FK → users, items
user_interactions	Click/view tracking (for RS)	id, user_id, item_id, type	FK → users, items
search_logs	Search query history	id, user_id, query, filters	FK → users
recommendations	RS output cache	id, user_id, item_id, score	FK → users, items
fraud_logs	Fraud detection results	id, user_id, risk_score, decision	FK → users
gap_results	Gap analysis snapshots	id, category, supply, demand, gap_score	Batch pipeline

kpis_snapshots_derived	KPI time series	id, metric, value, period	Dashboard
system_settings	Platform configuration	key, value, updated_by	FK → users
roles	Role definitions	id, name	Identity
permissions	Permission definitions	id, name	Identity
role_permissions	Role-permission mapping	role_id, permission_id	FK → roles, permissions

- **Table 18:** Cancellation Policy Matrix (Flexible/Moderate/Strict)

Policy	Free Cancel Window	After Window: Rental Fee Refund	Deposit	Who Sets
Flexible	> 24 hours before start	Refund all minus 1 day's fee	Always returned	Owner per item
Moderate	> 3 days before start	50% refund to Renter	Always returned	Owner per item
Strict	> 7 days before start	0% refund (non-refundable)	Always returned	Owner per item
Owner Cancels	Any time	100% refund to Renter	Always returned	—
Delivery Delayed	Past start date	100% refund regardless of policy	Always returned	System auto

- **Table 19:** Notification Event Types and Triggers

Event Key	Type	Recipient	Trigger
rental_confirmed	rental	Renter	Owner approves rental request
rental_started	rental	Renter	Both confirm handover

rental_completed	rental	Renter	Rental lifecycle ends
rental_cancelled	rental	Owner	Renter cancels
item_available	item	Favorited users	Item becomes available after rental ends
new_message	chat	Recipient	Other party sends a chat message
ticket_updated	support	User	Support agent responds
ticket_escalated	support	Admin	Agent escalates ticket
dispute_opened	rental	Other party + Support	User opens a dispute
dispute_resolved	rental	Both parties	Admin resolves dispute
return_reminder	system	Renter	24h before end date (background job)
late_fee_alert	system	Renter	Overdue detected (background job)

- **Table 20:** Late Return Escalation Timeline

Day Overdue	Action	Target	Details
Day 1	Notification	Renter	"Your rental is overdue. Late fees are accumulating."
Day 3	Second notification	Renter + Owner	Owner informed of overdue status
Day 7	Legal PDF enabled	Owner	PDF: rental contract, Renter identity, outstanding balance

Day 14	Auto Support ticket	Support team	"Severe overdue case" ticket auto-created
Day 30	Admin escalation	Admin	Direct notification. Account suspension considered

- **Table 21:** Platform Configuration Settings (System Defaults)

Setting	Default Value	Who Can Change	Description
platform_commission_percent	15%	Super Admin	Percentage deducted from rental earnings
default_cancellation_policy	Flexible	Admin	Default policy for new item listings
expert_evaluation_fee	Varies by category	Admin	Fee charged for Expert inspection
auto_resolve_days	5	Admin	Days before resolved ticket auto-closes
fraud_threshold	0.80	Admin	Anomaly score above which checkout is blocked
late_fee_rate	150%	Admin	Multiplier on daily rate for overdue penalty
checkout_lock_duration	5 minutes	Admin	Temporary reservation during payment
feature_buying	On	Super Admin	Enable/disable buying (sale) feature
feature_hub_evaluations	On	Super Admin	Enable/disable Expert Hub inspections
feature_ai_recommendations	On	Super Admin	Enable/disable RS module
feature_chat	On	Super Admin	Enable/disable real-time chat

maintenance_mode	Off	Super Admin	Takes entire platform offline

- **Table 22:** Competitive Differentiation Matrix

Feature	**Renty**	**Dubizzle EG**	**OLX EG**	**OpenSooq**	**Fat Lama**	**Airbnb**
Peer-to-peer rentals	Yes	No	No	No	Yes	Yes (property)
Peer-to-peer sales	Yes	Yes	Yes	Yes	No	No
Escrow payments	Yes	No	No	No	Yes	Yes
AI recommendations	Yes	No	No	No	No	Yes
Fraud detection (ML)	Yes	No	No	No	Basic	Yes
Expert physical inspection	Yes	No	No	No	No	No
Gap analysis (supply/demand)	Yes	No	No	No	No	No
In-app chat (real-time)	Yes	No	Yes	Yes	Yes	Yes
Admin KPI dashboard	Yes	No	No	No	No	Yes
Calendar availability	Yes	No	No	No	Yes	Yes
Deposit protection	Yes	No	No	No	Yes	Yes
Dual rating system	Yes	No	No	No	Yes	Yes
Egyptian market focus	Yes	Yes	Yes	No	No	No
Multi-category (movable assets)	Yes	No	No	No	Yes	No

- **Table 23:** Technology Stack Summary

Layer	**Technology**	**Version**	**Purpose**

Backend API	ASP.NET Core	8.0	RESTful API, Clean Architecture
Database	PostgreSQL	16	Primary data store
ORM	Entity Framework Core	8.0	Database access and migrations
Web Frontend	Angular	19	Admin + User web dashboard
Mobile App	Flutter	3.x	Cross-platform mobile app (iOS + Android)
Real-time	SignalR	8.0	WebSocket notifications
Real-time Chat	Socket.IO	4.x	Real-time messaging
Recommendation System	Python + LightFM	1.17	Hybrid collaborative filtering
Fraud Detection	Python + scikit-learn	1.4	Isolation Forest anomaly detection
Gap Analysis	Python + pandas	2.2	Supply-demand pipeline
Payment Gateway	Paymob	API v3	Card, Fawry, Vodafone Cash
File Storage	Cloudinary	—	Image/media hosting
Authentication	ASP.NET Identity + JWT	—	Token-based auth with refresh
Validation	FluentValidation	11	Request validation
Mediator	MediatR	12	CQRS command/query handling

- **Table 24:** Non-Functional Requirements Summary

Category	Requirement	Target	Implementation
Performance	API	< 200ms (p95)	Async/await,

	response time		EF query optimization
Performance	Page load time	< 3 seconds	Angular lazy loading, CDN images
Scalability	Concurrent users	1000+	Connection pooling, SignalR backplane
Availability	Uptime target	99.5%	Health checks, graceful degradation
Security	Authentication	JWT + refresh tokens	15-min access token, 7-day refresh
Security	Authorization	RBAC (6 roles, 7 policies)	Policy-based middleware
Security	Data protection	AES-256, HTTPS/TLS 1.3	All traffic encrypted
Security	Input validation	All endpoints validated	FluentValidation on every command
Usability	Mobile responsiveness	All screen sizes	Flutter adaptive layouts
Usability	Localization	Arabic + English	Bilingual titles, RTL support
Reliability	Data integrity	Referential integrity	FK constraints, ON DELETE RESTRICT
Reliability	Audit trail	All admin actions logged	Timestamped audit records
Maintainability	Code architecture	Clean Architecture	4 layers: Domain, Application, Infrastructure, Web

- **Table 25:** Mobile vs. Web Feature Matrix

Feature	Web (Angular)	Mobile (Flutter)
User registration / login	Yes	Yes
Browse and search items	Yes	Yes
Item detail view	Yes	Yes
Create rental request	Yes	Yes
Real-time chat	Yes	Yes
Notifications	Yes (dropdown)	Yes (list screen)
Favorites / wishlist	Yes	Yes
My rentals management	Yes	Yes
Leave reviews	Yes	Yes
Wallet (balance + history)	Yes	Yes
Support tickets	Yes	Yes
List items (Owner)	Yes	Yes
Calendar availability	Yes	Planned
Admin KPI dashboard	Yes	No (web-only)
Admin user management	Yes	No (web-only)
Admin item approval	Yes	No (web-only)
Expert verification console	Yes	No (web-only)
Gap analysis charts	Yes	No (web-only)
Super Admin settings	Yes	No (web-only)

- **Table 26:** Standards Compliance Summary

Standard	Scope	Implementation
OWASP Top 10 (2021)	Security	JWT auth, input validation, parameterized queries, CORS, rate limiting
REST API Design	API	Resource-based URLs, proper HTTP verbs, status codes, pagination
OpenAPI 3.0	Documentation	Auto-generated Swagger at /api/docs
IEEE 830	Requirements	Structured SRS with functional/non-functional requirements
ISO 27001 (principles)	Data Protection	Encrypted storage, access control, audit logging
WCAG 2.1 AA	Accessibility	Semantic HTML, ARIA labels, keyboard navigation, contrast ratios
GDPR (principles)	Privacy	User data deletion, consent, minimal data collection
Clean Architecture	Code Structure	Dependency inversion, layer separation, testability
SOLID Principles	OOP Design	Single responsibility, open-closed, dependency injection
Conventional Commits	Version Control	Structured commit messages for traceability

Abbreviations

- **AUC** — Area Under the Curve: A performance metric measuring the quality of binary classification and ranking systems, with values ranging from zero to one where higher scores indicate superior model discrimination capability. The recommendation system achieved an AUC of 0.8421.

- **API** — Application Programming Interface: A set of defined rules and protocols that allows different software applications to communicate with each other, implemented in Renty as RESTful endpoints under the `/api/v1` namespace.

- **ACID** — Atomicity, Consistency, Isolation, Durability: A set of properties guaranteeing reliable processing of database transactions, enforced by PostgreSQL to ensure financial integrity of escrow operations and rental state transitions.

- **CBF** — Content-Based Filtering: A recommendation approach that suggests items based on their attributes and features, matching item characteristics to user preferences derived from historical interactions and demographic profiles.

- **CF** — Collaborative Filtering: A recommendation technique that identifies patterns in user behavior by finding similarities between users or items, enabling predictions based on the preferences of like-minded individuals.

- **CRUD** — Create, Read, Update, Delete: The four basic operations of persistent storage, implemented across all API modules for managing platform entities.

- **CTR** — Click-Through Rate: The ratio of users who click on specific content to the total number of users who view that content, serving as a key engagement metric for measuring recommendation system effectiveness.

- **EGP** — Egyptian Pound: The official currency of Egypt, used for all financial transactions, deposits, and commission calculations within the Renty platform.

- **ERD** — Entity Relationship Diagram: A visual representation of the database schema showing tables, their attributes, and the relationships between them.

- **ENUM** — Enumerated Type: A data type in PostgreSQL consisting of a static, ordered set of values, used extensively in Renty for enforcing valid states (e.g., rental_status, payment_method).

- **GMV** — Gross Merchandise Volume: The total value of merchandise or services sold through the platform over a given period, serving as a primary revenue health indicator.

- **HMAC** — Hash-based Message Authentication Code: A cryptographic authentication technique used for verifying payment gateway callbacks from providers such as Paymob.

- **JWT** — JSON Web Token: A compact, URL-safe token format used for stateless authentication and authorization, enabling secure transmission of user identity claims between client and server.

- **KPI** — Key Performance Indicator: A quantifiable measure used to evaluate the success of an organization or platform in achieving operational objectives.

- **KYC** — Know Your Customer: Identity verification procedures requiring users to submit national ID photographs and phone OTP verification before executing high-value transactions.

- **LOF** — Local Outlier Factor: A density-based anomaly detection algorithm considered as an alternative to Isolation Forest for fraud detection.

- **MAU** — Monthly Active Users: The count of unique users who perform at least one meaningful action on the platform within a calendar month.

- **MVP** — Minimum Viable Product: The initial version of the platform with sufficient features to satisfy early adopters and validate core business assumptions.

- **OTP** — One-Time Password: A dynamically generated code sent via SMS for phone number verification as part of the KYC process.

- **RBAC** — Role-Based Access Control: A method of restricting system access based on the roles of individual users, implemented through the roles and role_permissions database tables.

- **REST** — Representational State Transfer: An architectural style for distributed systems defining constraints for web service design, followed by all Renty API endpoints.

- **RFM** — Recency, Frequency, Monetary: A marketing analysis technique adapted for feature engineering in both recommendation and fraud detection systems.

- **TF-IDF** — Term Frequency-Inverse Document Frequency: A numerical statistic reflecting the importance of words in documents within a collection, used to extract meaningful features from product descriptions for content-based recommendation filtering.

- **TLS** — Transport Layer Security: A cryptographic protocol providing communications security over a computer network, enforced at version 1.2+ for all platform communications.

- **UML** — Unified Modeling Language: A standardized visual modeling language used to specify, visualize, construct, and document software system artifacts.

- **UUID** — Universally Unique Identifier: A 128-bit identifier used for email verification tokens and other entities requiring globally unique references without sequential predictability.

- **WARP** — Weighted Approximate-Rank Pairwise: A loss function specifically designed for learning to rank with implicit feedback data, optimizing the recommendation system to position relevant items at the top of suggestion lists.

Symbols

- **α (alpha)** — Regularization parameter controlling the strength of L2 penalty applied to user and item embeddings in the LightFM model. The optimal value of 0.00001 achieved the best balance between model expressiveness and generalization, reducing overfitting by 80.8% compared to unregularized baseline configurations.

- **μ (mu)** — Population mean representing the average value of a dataset or distribution, used in statistical analysis for establishing baseline expectations and calculating standardized scores.

- **σ (sigma)** — Standard deviation quantifying the amount of variation or dispersion in a dataset, employed in feature scaling through z-score normalization and in establishing statistical thresholds for anomaly detection at $\mu \pm 2\sigma$ levels.

- **ψ (psi)** — Subsample size parameter in Isolation Forest algorithm determining the number of instances randomly selected for building each isolation tree. The typical value of 256 balances computational efficiency with detection quality.

- **θ (theta)** — Model parameters or weights learned during training that define the decision boundaries and prediction functions. In matrix factorization, theta represents the latent factor vectors encoding user preferences and item characteristics.

- **λ (lambda)** — Learning rate parameter controlling the step size during gradient descent optimization, set to 0.05 for stable convergence.

- **Σ (capital sigma)** — Summation operator indicating the addition of sequence elements, applied in computing aggregate metrics such as total revenue and weighted averages.

Chapter 1 – Introduction

The exponential growth of digital commerce has fundamentally transformed how individuals access goods and services, creating unprecedented opportunities for peer-to-peer resource sharing through online platforms. E-commerce transactions have increased by over 200% in recent years, establishing digital marketplaces as essential infrastructure for modern economic activity. However, this rapid expansion has introduced significant technical and operational challenges that traditional system architectures struggle to address effectively. Users navigating extensive product catalogs face overwhelming choices without adequate guidance, platform operators contend with increasingly sophisticated fraudulent activities that evolve continuously to circumvent detection mechanisms, and marketplace dynamics suffer from persistent supply-demand imbalances that hinder efficient resource allocation. Moreover, the lack of operational trust and secure asset verification severely restricts the adoption of sharing economy models for high-value items.

This graduation project presents **Renty**, an intelligent peer-to-peer rental platform that addresses these critical challenges through the strategic integration of machine learning and advanced analytics, all built upon a strictly controlled transactional core.

1.1 Motivation

The motivation for developing Renty emerges from the observation that existing Egyptian classifieds and listing platforms — such as Dubizzle Egypt, OLX Egypt, and OpenSooq — operate primarily as listing boards without transactional guarantees, personalized experiences, or proactive risk management. Users seeking to rent physical items encounter generic listings without personalized guidance, leading to inefficient search processes. These platforms lack built-in payment escrow, leaving both parties financially exposed. No existing Egyptian platform offers ML-driven product recommendations, automated fraud detection, or supply-demand analytics for the rental category. Market imbalances between supply and demand persist undetected because no local platform tracks or analyzes these patterns. Meanwhile, global platforms like Airbnb and Fat Lama solve these problems for their specific domains (accommodation, equipment lending) but do not serve the Egyptian market for general movable assets.

Our specific focus targets the Egyptian market, and more precisely for the current time, sports tourism in areas such as Dahab, Sinai, and the Red Sea coast. In these areas, short-term, high-value rentals—diving gear, hiking equipment, kite surfing rigs, premium photography tools—are in immense demand by international tourists, digital nomads, and expatriates. Tourists often require expensive, bulky

equipment for short durations of two to five days but prefer not to travel with it to avoid airline fees and logistical burdens. Local residents and dive shops own this equipment but suffer from low off-season utilization. Renty is designed to bridge this gap by providing a reliable, secure, and intelligent platform that protects both owners and renters through physical evaluation hubs ("Renty Points"), escrow payments, and comprehensive AI monitoring.

The emergence of advanced machine learning techniques offers solutions to these persistent challenges, yet significant gaps remain between theoretical research and practical implementation in production systems. Academic literature demonstrates the effectiveness of hybrid recommendation approaches, unsupervised anomaly detection, and analytical frameworks, but few comprehensive implementations integrate these capabilities within unified platforms designed specifically for the rental economy. The rental domain presents unique characteristics that distinguish it from traditional e-commerce, including temporal availability constraints, geographic distribution requirements, asset condition considerations, and trust mechanisms essential for peer-to-peer transactions involving high-value physical assets. These domain-specific factors necessitate tailored solutions that extend beyond generic e-commerce implementations.

The development of Renty addresses both academic and practical motivations. From an academic perspective, the project demonstrates the application of advanced algorithms to real-world problems while maintaining rigorous documentation and evaluation standards suitable for scholarly assessment. From a practical perspective, the platform delivers measurable business value through improved user engagement, reduced fraud losses, optimized inventory management, and comprehensive performance visibility. The modular architecture ensures that individual components can be enhanced or replaced as algorithmic techniques evolve, establishing foundations for continuous improvement beyond the initial implementation.

1.2 Problem Statement

The central problem addressed by this project is the inability of conventional rental platforms to deliver secure, optimized, and personalized experiences. This overarching challenge manifests through several interconnected problem dimensions:

1. Information Overload and Cold Starts. Users navigating product catalogs containing hundreds of distinct items face choice paralysis. New users possess no historical interaction data, rendering pure collaborative filtering approaches ineffective during the critical first impressions when engagement patterns are established (the cold-start problem). Similarly, newly listed items lack the interaction history required for traditional recommendation algorithms to assess relevance. Existing content-based approaches fail to capture collaborative signals about item quality, while pure collaborative filtering cannot handle cold-start scenarios, necessitating hybrid solutions that combine multiple information

sources effectively. The interaction matrix exhibits extreme sparsity exceeding 97.52%, meaning explicit feedback exists for fewer than three percent of all possible user-item pairs.

2. Fraud and Security. Malicious activities—including account takeover targeting established users, velocity abuse through automated rapid-fire transactions, and financial mismatches suggesting stolen payment credentials—are difficult to detect proactively. Static rule-based systems are easily circumvented as fraudsters continuously adapt their tactics. Excessive false positives frustrate legitimate users, while missed fraud results in direct financial losses and reputational damage. The rarity of fraud events, typically constituting less than five percent of total transactions, creates severe class imbalance. The absence of ground truth labels necessitates unsupervised approaches.

3. Market Imbalance. Persistent supply-demand gaps constrain transaction volume and revenue generation. Undersupply leads to unfulfilled requests and frustrated renters who may abandon the platform; oversupply leads to poor asset utilization and discouraged owners. These imbalances evolve dynamically as seasonal patterns, promotional campaigns, and external market conditions shift the distribution of supply and demand. Platform operators lack systematic mechanisms to detect these gaps proactively and quantify their severity.

4. Lack of Operational Trust. Users hesitate to rent high-value items to strangers without reliable mechanisms for verification, secure payment, and condition assessment. “He said, she said” disputes regarding item damage are rampant in unmanaged peer-to-peer platforms. In the Egyptian market specifically, the absence of institutional trust infrastructure for peer-to-peer asset sharing creates a significant barrier to adoption that purely digital solutions cannot overcome.

5. Absence of Comprehensive Monitoring. Platform operators require real-time insights into growth trajectories, user engagement patterns, marketplace efficiency, and trust mechanisms to make informed strategic decisions. Traditional analytics approaches focus on isolated metrics without capturing the interconnected nature of platform dynamics where changes in one dimension cascade through others.

These five problem dimensions are deeply interconnected. Poor recommendations reduce user engagement, which exacerbates data sparsity and limits collaborative filtering effectiveness. Undetected fraud erodes user trust, discouraging legitimate participation. Marketplace imbalances create negative feedback loops. The solution requires an integrated approach addressing all dimensions simultaneously.

1.3 Objective

The primary objective of this graduation project is to design, implement, and evaluate a cross-platform (Web and Mobile) intelligent peer-to-peer rental system. Specific goals include:

- **Core Transactional Platform:** Develop a robust, commission-based rental marketplace with comprehensive user management, dynamic item cataloging, calendar-based availability, and secure payment processing via simulated gateways (Paymob, Fawry, Vodafone Cash).
- **Trust Mechanisms:** Implement escrow payments, physical evaluation hubs ("Renty Points") with certified experts, a continuity rule for verification, and tiered cancellation policies (Flexible, Moderate, Strict) with predetermined penalty parameters.
- **AI Enhancement:** Integrate a hybrid recommendation system (LightFM) achieving test AUC > 0.80 and Precision@10 > 0.12. Implement unsupervised fraud detection (Isolation Forest) targeting a 5% detection rate. *AI modules are integrated as supporting components to enhance decision-making, not as standalone systems.*
- **Analytics and Dashboards:** Establish gap analysis and comprehensive KPI dashboards providing real-time, actionable insights.
- **Mobile Application:** Develop a Flutter mobile app providing on-the-go accessibility with real-time push notifications, geolocation-based searches, and integrated chat.

1.4 Problem Complexity

Building Renty involves tackling significant architectural and algorithmic complexities:

- **Cross-Platform Availability:** The Web application (Angular) serves all six roles as the primary platform, while a companion Mobile application (Flutter) provides Renters and Owners with on-the-go access to core features. Both clients share the same backend API.
- **Complex State Machines:** The rental lifecycle involves over nine distinct states. Managing transitions, enforcing temporal logic (cancellation windows, late fees), and preventing invalid state jumps requires rigorous database-level constraints.
- **Data Sparsity in ML:** The interaction matrix is extremely sparse (>97% sparsity). Designing a hybrid recommendation model that correctly fuses collaborative signals with NLP-processed item descriptions and user demographics requires deep feature engineering.

- **Real-Time Asynchronous Processing:** Recommendation generation must occur asynchronously to avoid blocking the user experience. This requires a decoupled architecture where AI microservices operate in tandem with the primary ASP.NET Core (C#) backend, PostgreSQL database, Angular web application, and Flutter mobile application.
- **Concurrent Access Control:** Row-level locks and 5-minute checkout locks prevent double-booking with idempotency keys for duplicate payment prevention.

1.5 Constraints

- **Temporal Constraints:** Two academic semesters with analytics/AI in semester one and full system integration in semester two.

- **Computational Constraints:** Model training and inference must run efficiently on standard cloud instances without GPU clusters.

- **Data Constraints:** Large-scale labeled fraud data is unavailable, necessitating unsupervised ML (Isolation Forest).

- **Payment Regulations:** Payment gateways (Paymob, Fawry) are rigorously simulated for the academic prototype.

- **Usability Constraints:** Mobile-first responsive design for Android via Flutter. Arabic/English interface for the local/international sports tourism demographic.

- **Cost Constraints:** Open-source technologies (Angular, ASP.NET Core, PostgreSQL, LightFM, scikit-learn, Flutter) without licensing fees.

1.6 Standards

The development adheres to modern software engineering standards:

- **RESTful API Design:** Standard HTTP verb usage, status codes, and JSON payloads documented via Swagger.
- **Security Standards:** bcrypt for password hashing, JWT for stateless authentication, HTTPS/TLS 1.2+ encryption.
- **Relational Integrity:** PostgreSQL foreign keys, ENUMs, CHECK constraints, and triggers to enforce business rules at the lowest layer.
- **UML 2.5:** Standardized visual modeling for architecture and behavioral documentation.
- **IEEE 12207:** Software lifecycle processes followed throughout both semesters.

(Refer to Table 26: Standards Compliance Summary in the List of Tables).

1.7 Feasibility Study and Business Canvas

Renty is not just a technical proof-of-concept; it is designed with a viable business model tailored to specific market conditions.

- **Target Audience:** Sports tourists, expatriates, and local adventure enthusiasts in Egypt (initially Sinai/Red Sea regions). The interface is exclusively in English to cater to this diverse, high-value demographic.
- **Value Proposition:** Tourists gain access to expensive equipment without the burden of travel logistics. Local owners monetize idle assets safely through platform-managed trust mechanisms.

- **Revenue Streams:** A commission-based model deducting a standard 15% from the total rental fee upon successful completion. Secondary revenue streams include optional premium catalog placement and service fees for expert inspections at Renty Points.

- **Cost Structure:** Server hosting, cloud database instances, API usage for maps/SMS (simulated in prototype), and physical logistics overhead for Renty Points.

(Refer to Chapter 10 for the complete business model analysis including cancellation protection, dispute resolution, and wallet operations.)

1.8 Addressing Supervisor Feedback

Throughout development, Dr. Khamis provided critical feedback that shaped the final system:

1. **AI as Enhancement, Not Core:** AI modules serve as supporting components enhancing the transactional core, not as standalone systems.
2. **Database-Level Enforcement:** Business logic pushed to the database level via PostgreSQL triggers and constraints.
3. **Physical Trust Infrastructure:** Renty Points concept with Expert hubs addressing the “he said, she said” problem.
4. **Mobile-First Approach:** Angular web application with Flutter mobile app support for on-the-go renters.

Chapter 2 – Background

The development of an intelligent peer-to-peer rental platform requires a comprehensive understanding of several interconnected technical domains. This chapter provides the theoretical foundation for the recommendation algorithms, anomaly detection techniques, business analytics frameworks, and software architecture patterns utilized in Renty.

2.1 E-Commerce and Peer-to-Peer Rental Platforms

2.1.1 Evolution of Digital Marketplaces

The transformation of commerce from traditional brick-and-mortar retail to digital platforms represents one of the most significant economic shifts of the past three decades. E-commerce emerged in the mid-1990s with pioneers such as Amazon and eBay establishing models for online retail and auction-based transactions. These early platforms demonstrated that digital intermediation could reduce transaction costs, expand market reach, and provide information transparency previously unattainable in physical marketplaces.

The subsequent evolution introduced the concept of peer-to-peer marketplaces where platforms facilitate direct transactions between individuals rather than acting as traditional retailers holding inventory. This model gained prominence through platforms such as Airbnb for accommodation sharing and Uber for transportation services. The peer-to-peer approach leverages underutilized assets owned by individuals, creating value through improved resource allocation efficiency rather than through manufacturing or wholesaling new products.

Rental-focused peer-to-peer platforms represent a natural extension of this evolution, addressing the recognition that many physical assets experience low utilization rates. Consumer goods such as power tools, camping equipment, photography gear, and sporting equipment often sit idle for extended periods between uses. The rental model enables asset owners to generate returns on these investments while providing renters with cost-effective temporary access without the commitment of ownership. This arrangement aligns with broader societal trends toward access-based consumption, environmental sustainability through reduced manufacturing demands, and financial prudence in personal asset management.

2.1.2 Characteristics of Rental Transactions

Rental transactions differ from traditional retail purchases in several fundamental ways that influence system design requirements. The **temporal dimension** is central to rental arrangements, where the value proposition depends on providing access for specific durations rather than permanent ownership transfer. This introduces scheduling complexity as assets must be available during requested timeframes and returned in accordance with agreed timelines.

The **geographic dimension** becomes more prominent in rental scenarios compared to e-commerce purchases that can be shipped economically across long distances. Rental economics typically favor local transactions where transportation costs and logistics complexity remain proportional to rental value. This geographic constraint influences recommendation algorithms that must consider proximity alongside preference matching.

The **condition dimension** introduces requirements absent from traditional e-commerce where purchased items arrive new from manufacturers. Rental assets experience wear through repeated use cycles, requiring assessment of condition before and after each rental period. Trust mechanisms must address potential disputes regarding damage responsibility and establish fair processes for condition evaluation.

The **trust dimension** is amplified in peer-to-peer rental contexts where strangers entrust valuable possessions to one another without the intermediation of established retail brands. Both asset owners and renters face risks including non-return of items, damage beyond normal wear, misrepresentation of asset condition, and potential safety concerns when facilitating in-person exchanges. Effective rental platforms must establish verification mechanisms, reputation systems, insurance arrangements, and dispute resolution processes that build sufficient confidence to overcome natural hesitation.

2.1.3 Market Dynamics and Network Effects

Peer-to-peer rental platforms exhibit strong network effects where value to each participant increases as the total number of participants grows. A larger population of asset owners expands the variety and availability of rental inventory, making the platform more attractive to potential renters. Conversely, a larger population of renters increases utilization rates and revenue potential for asset owners, encouraging additional suppliers to join. These positive feedback loops create natural monopolistic tendencies where market leaders achieve sustainable competitive advantages through scale.

However, network effects also create chicken-and-egg challenges during platform establishment. Early users encounter limited selection and matching opportunities, requiring platforms to invest heavily in acquiring critical mass on both supply and demand sides simultaneously. Supply-demand balance emerges as a critical factor for platform health and growth trajectory, necessitating analytical frameworks that continuously monitor and address marketplace imbalances.

2.2 Recommender Systems

2.2.1 Fundamental Concepts and Approaches

Recommender systems constitute a specialized class of information filtering systems that predict user preferences and suggest items likely to be of interest. These systems address the fundamental challenge of information overload where users face choice sets too large for efficient manual evaluation. The field recognizes three classical paradigms:

Content-Based Filtering (CBF) operates by analyzing item attributes and matching them to user preference profiles constructed from historical interactions. This approach recommends items similar to those a user has previously engaged with, leveraging item metadata such as categories, descriptions, features, and specifications. Content-based methods excel when rich item metadata is available and can recommend new items immediately upon catalog entry. However, these methods tend toward over-specialization where recommendations reinforce existing preferences without exposing users to potentially interesting alternatives.

Collaborative Filtering (CF) identifies patterns in user behavior without requiring explicit item metadata. This approach operates on the principle that users who agreed in the past will likely agree in the future, leveraging either user-user or item-item similarities. Collaborative methods can discover unexpected patterns and quality signals that content analysis cannot detect, enabling serendipitous recommendations. However, CF suffers severely from cold-start problems where new users and new items cannot be effectively recommended. Additionally, interaction matrices are typically extremely sparse with explicit feedback existing for only a small fraction of user-item pairs.

Hybrid Approaches combine content-based and collaborative methods to leverage the strengths of each while mitigating individual weaknesses. Various hybridization strategies exist including weighted combinations, switching strategies, and unified models that integrate multiple information sources within a single algorithmic framework.

2.2.2 Matrix Factorization Techniques

Matrix factorization represents a powerful technique for collaborative filtering that decomposes the sparse user-item interaction matrix into lower-dimensional latent factor representations. The fundamental insight is that user preferences and item characteristics can be represented in a shared latent space where proximity indicates compatibility. By learning latent vectors for users and items such that their inner product approximates observed interactions, matrix factorization captures complex preference patterns in compact representations.

The approach offers advantages including dimensionality reduction that mitigates sparsity challenges, generalization capability for unobserved user-item pairs, and computational efficiency where prediction requires only inner product computation. Extensions such as factorization machines generalize matrix factorization to arbitrary feature combinations, enabling integration of user metadata, item metadata, and contextual information within unified models.

2.2.3 The LightFM Framework

LightFM represents a specific implementation of hybrid matrix factorization designed for recommendation scenarios with both interaction data and side information about users and items. The framework extends traditional matrix factorization by representing users and items as weighted combinations of their features rather than single latent vectors. User embeddings are computed as

weighted sums of embeddings for demographic attributes and behavioral characteristics. Similarly, item embeddings aggregate embeddings for categories, attributes, and textual descriptions.

This feature-based representation enables cold-start handling where new users receive recommendations based on demographic similarity to existing users, and new items are recommended based on attribute similarity to established catalog items. LightFM supports the Weighted Approximate-Rank Pairwise (WARP) loss function, which optimizes for ranking quality in implicit feedback scenarios by focusing optimization effort on the most important ranking errors where relevant items are positioned below irrelevant alternatives.

2.2.4 Evaluation Metrics for Recommender Systems

For ranking-oriented recommendations, the following metrics prove most appropriate:

- **Precision@K** measures the fraction of top K recommendations that are relevant, directly corresponding to user experience quality.
- **Recall@K** measures the fraction of all relevant items appearing within top K recommendations, quantifying coverage.
- **Area Under the Curve (AUC)** represents the probability that a randomly chosen relevant item ranks higher than a randomly chosen irrelevant item, providing an aggregate measure of ranking quality. Values range from 0.0 to 1.0 where 0.5 indicates random ranking.

Evaluation protocols employ temporal splitting where training data includes interactions before a cutoff date and test data contains subsequent interactions, simulating realistic production conditions.

2.3 Anomaly Detection and Fraud Prevention

2.3.1 Fundamentals of Anomaly Detection

Anomaly detection encompasses techniques for identifying patterns in data that deviate significantly from expected behavior. In the context of e-commerce and peer-to-peer rentals, anomalies often correlate with fraudulent activities. Anomalies are categorized into **point anomalies** (individual deviant instances), **contextual anomalies** (normal in isolation but unusual in specific contexts), and **collective anomalies** (groups of observations that together represent abnormal behavior).

The fundamental challenge in anomaly detection is defining normality in high-dimensional feature spaces where the boundary between normal and anomalous behavior is not clearly delineated. This challenge is exacerbated in dynamic environments where normal patterns evolve over time and adversaries actively attempt to disguise anomalous behavior.

2.3.2 Supervised versus Unsupervised Approaches

Traditional fraud detection relies on supervised learning, where models are trained on historical datasets containing explicitly labeled legitimate and fraudulent transactions. However, acquiring high-quality labeled fraud data is notoriously difficult, particularly for new platforms where historical fraud examples are unavailable. Furthermore, fraudsters continuously evolve their tactics, causing models trained on historical patterns to become obsolete. Renty addresses this limitation by employing unsupervised approaches, which detect outliers based solely on the inherent statistical properties of the data, without requiring prior labels.

2.3.3 Isolation Forest Algorithm

The Isolation Forest algorithm is an unsupervised decision-tree-based method uniquely suited for anomaly detection. Unlike algorithms that model normal behavior and flag deviations, Isolation Forest explicitly isolates anomalies. Because anomalies are “few and different,” they are more susceptible to isolation closer to the root of a random decision tree. By constructing an ensemble of such trees (a forest), the algorithm calculates an anomaly score based on the average path length required to isolate a given instance. Shorter average path lengths indicate higher anomaly scores. This method is highly efficient in handling high-dimensional behavioral data streams, with computational complexity of $O(t \times n \times \log(\psi))$ where t is the number of trees, n the number of instances, and ψ the subsample size.

2.3.4 Feature Engineering for Fraud Detection

Effective fraud detection depends heavily on the quality of engineered features that capture behavioral signals. Key feature categories include:

- **Temporal Features:** Transaction timing, order-to-fulfillment gaps, and session duration patterns.
- **Velocity Metrics:** Orders per hour, messages per minute, distinct devices used, and account activity acceleration.
- **Financial Mismatch:** Ratio of current order value to historical average, quantity-to-income ratios, and affordability anomalies.
- **Geographic Features:** Distance between IP location and registered address, sudden location changes.

- **Behavioral Aggregations:** RFM (Recency, Frequency, Monetary) scores capturing user activity profiles.

2.4 Business Analytics and Key Performance Indicators

2.4.1 Principles of Performance Measurement

For a multi-sided marketplace to survive, administrators must constantly monitor liquidity, engagement, and safety. Key Performance Indicators translate raw transactional data into strategic insights. KPIs are organized hierarchically: platform-level metrics provide executive visibility, while category-level and regional-level metrics support operational decision-making.

2.4.2 Gap Analysis Frameworks

In peer-to-peer markets, maintaining equilibrium between supply (listed items) and demand (user searches and rental requests) is paramount. A Gap Analysis framework continuously computes the difference between active inventory and demand signals. By normalizing these metrics into a standardized "Gap Score," platform operators can objectively classify product categories into severity levels (Low, Moderate, High, Critical) and execute targeted interventions. The gap score computation follows: Gap_Score = (Demand_Volume - Active_Supply_Count) × Value_Multiplier, where Demand_Volume is derived from unfulfilled search queries and rejected rental requests.

2.4.3 Analytical Data Pipelines

Modern analytics architectures employ event-driven pipelines that capture user interactions, aggregate them through temporal windowing, store historical snapshots, and expose computed indicators through standardized interfaces. Batch processing pipelines run periodically (e.g., nightly) for computationally intensive operations like gap analysis, while real-time streams handle time-sensitive metrics like active rental counts.

2.5 Software Architecture Patterns

2.5.1 Layered Architecture

Modern applications require architectures that separate concerns to ensure maintainability and scalability. A layered architecture separates:

1. **Presentation Layer** — User interfaces (mobile apps, web dashboards) that maintain zero business logic.

2. **Application Logic Layer** — Backend APIs that orchestrate business rules, validation, and state management.

3. **Data Persistence Layer** — Relational databases enforcing integrity constraints.

4. **Intelligence Layer** — Specialized modules for ML inference and analytics processing.

2.5.2 Microservices-Inspired Patterns

Microservices-inspired patterns further decouple heavy computational tasks (such as AI inference) from the core transactional APIs, ensuring that long-running operations do not degrade the user experience. This pattern enables independent scaling, deployment, and evolution of system components.

2.5.3 Real-Time Communication Patterns

WebSocket-based architectures enable bidirectional, full-duplex communication between clients and servers. In marketplace platforms, WebSockets support real-time chat, live notification delivery, and instant state change broadcasts. Socket.IO provides a robust abstraction layer with automatic fallback to HTTP long-polling when WebSocket connections are unavailable, ensuring reliability across diverse network conditions.

Chapter 3 – Related Work and Similar Systems

To establish Renty's competitive positioning, it is essential to analyze existing platforms operating within the Egyptian, regional, and global markets. This chapter examines competitors across multiple dimensions and identifies the gaps that Renty addresses.

3.1 Egyptian Market Analysis

3.1.1 Dubizzle Egypt (OLX Egypt before)

Dubizzle is a dominant classifieds platform in Egypt facilitating the buying and selling of used goods. While extremely popular with significant user traffic, it operates purely as an introduction service. It lacks any integrated rental logic, escrow payment handling, or verified condition checks. Users engaging in rentals on Dubizzle do so entirely at their own risk, negotiating deposits directly in cash, which frequently leads to disputes and theft. There is no calendar management for asset availability, no dispute resolution mechanism, and no rating system that differentiates renter reliability from owner reliability. Renty resolves these deficiencies by acting as an active intermediary throughout the full rental lifecycle, managing payments, enforcing state transitions, and providing expert verification at physical hubs.

3.2 Regional Platform Analysis

3.2.1 OpenSooq

OpenSooq serves the broader MENA region with a similar model to Dubizzle. It excels in liquidity through massive catalog sizes but fails to address the specific needs of the rental economy. Key shortcomings include: no calendar management or date-based conflict resolution, no dynamic pricing based on rental duration (daily vs. weekly vs. monthly), no escrow or deposit handling, no item condition verification, and no AI-powered discovery or fraud prevention. OpenSooq treats all listings as static classified advertisements without lifecycle management.

3.2.2 Yalla Fel Nile (Emerging)

Several emerging platforms in the Egyptian market have attempted to address niche rental needs, particularly for event equipment and party supplies. However, these platforms typically operate as simple booking forms without sophisticated state management, escrow protection, or analytics capabilities. Their limited scope and lack of trust infrastructure restrict adoption for high-value asset categories.

3.3 Global Platform Analysis

3.3.1 Airbnb

Airbnb is the global standard for peer-to-peer real estate rentals. It successfully implements escrow systems, robust dispute resolution with dedicated trust and safety teams, and sophisticated review algorithms. Airbnb's calendar management and dynamic pricing algorithms represent best-in-class implementations for the accommodation domain. However, Airbnb's architecture is highly specialized for property (fixed geographic locations, cleaning fees, strict legal compliance per jurisdiction). Renty adopts Airbnb's trust and calendar models but adapts them for movable physical assets, introducing the necessity of physical evaluation hubs (Renty Points) to verify asset condition between renters—a requirement absent in real estate rentals where property condition is relatively stable.

3.3.2 Fat Llama / Hurr

Fat Llama (UK) and Hurr (fashion rentals) represent global attempts at peer-to-peer item rentals. These platforms implement insurance-backed trust models where items are covered against damage and theft. While effective in markets with established insurance infrastructure, this approach is not directly applicable to the Egyptian market where peer-to-peer rental insurance products are unavailable. Renty's approach of deposit-based trust with expert verification provides a culturally and economically appropriate alternative.

3.3.3 Turo

Turo specializes in peer-to-peer car rentals, implementing sophisticated identity verification, GPS tracking, and insurance integration. While relevant as an architectural reference, Turo's model is vertically integrated for a single asset class (vehicles) and relies heavily on regulatory frameworks specific to the automotive industry.

3.4 Competitive Differentiation Analysis

Renty distinguishes itself from regional and global competitors through three core pillars:

1. **Intelligent Operation:** Using AI to actively guide users (Recommendations via LightFM) and protect the platform (Fraud Detection via Isolation Forest). No competitor in the Egyptian market integrates machine learning into the rental workflow.

2. **Managed Trust:** Escrow payments that hold all funds until successful completion, Expert Verification at physical Renty Points with before/after photography and a continuity rule, and predetermined penalty parameters for dispute resolution—removing the anxiety of renting out high-value equipment to strangers.

3. **Data-Driven Administration:** Built-in Gap Analysis framework computing supply-demand imbalances by category and region, KPI dashboards providing real-time operational visibility, and automated notification systems for marketplace health monitoring. No competitor provides administrators with this level of analytical capability.

(Refer to Table 22: Competitive Differentiation Matrix in the List of Tables for a detailed feature comparison across all analyzed platforms.)

Chapter 4 – Analysis

The analysis phase transforms stakeholder needs and project objectives into structured requirements that guide system design and implementation. This chapter presents the comprehensive requirements engineering process undertaken for the Renty platform, covering both the mobile application and the web dashboard.

4.1 Functional Requirements

The functional requirements define the specific behaviors, workflows, and capabilities that the Renty platform must support. These are divided comprehensively across the two target client platforms: Mobile and Web.

4.1.1 Mobile Application (Renters & Owners)

The Mobile Application, built with Flutter, acts as the primary tool for end-users, optimized for on-the-go interaction, geolocation discovery, and immediate notifications.

User Authentication and Verification (REQ-UM-1 to 6).

- **User Registration & Authentication**

  Users must register and log in using JWT, then authentication uses access tokens and refresh tokens.

- **KYC Verification (Required for High-Value Transactions)**

  Users must upload photos of their National ID (front and back), then verify their phone number via OTP.

- **Email Verification**

  Email verification is mandatory for account activation, then unverified users can only browse items and cannot rent, buy, list items, or chat, then a persistent banner is shown: “Verify your email to unlock all features.”

- **Security & Edge Cases**

  Entering the wrong password 5 times then the account is locked for 15 minutes and an email notification is sent, then suspended accounts see: “Your account has been suspended. Contact support”, then login from a new device triggers a notification with device and location information, then password reset invalidates all existing sessions.

Item Discovery and AI Recommendations (REQ-SD-1, REQ-PR-1).

- **Item Discovery & AI Recommendations**

A dynamic home feed is powered by the LightFM hybrid model, then users can search and filter the catalog using text or explore a map interface to find nearby assets.

- **Search Logging & Analysis**

Every search query is logged in *search_logs* for gap analysis, then this data helps improve recommendations and identify missing demand.

- **Cold Start Handling (New Users)**

New users with no history receive popular items based on their city, then recommendations gradually improve as user activity increases.

- **Recommendation Constraints**

The system never suggests items the user already owns, then it excludes suspended items and pending items.

- **Filtering Options**

Users can filter by category, subcategory, price range, location/city, availability dates, condition, and owner rating.

Listing Management (REQ-IC-1 to 8).

- **Listing Creation**

Owners create listings by selecting a product category and subcategory from a standardized taxonomy, then they write descriptions and upload 1 to 10 images via Cloudinary (at least one image is required and one must be set as primary).

- **Pricing Configuration**

Owners define dynamic pricing tiers, then a daily rate is required while weekly and monthly rates are optional.

- **Policies & Deposits**

Owners set a cancellation policy tier (Flexible, Moderate, or Strict), then they define a security deposit, or if not set, a default is applied as at least 10% of the daily price multiplied by 7 days.

- **Availability Management**

Owners can toggle availability using an integrated calendar, then this controls when the item can be rented.

- **Approval Workflow**

New listings enter a *pending_approval* status, then they are queued for Admin review before becoming visible.

- **Edit Rules**

Major edits (title, price, deposit, description) return the listing to *pending_approval*, then minor changes like photo reordering do not affect approval status.

- **Owner Monitoring**

Owners with 3 or more rejected listings are flagged for Admin review, then additional checks may be applied to their future listings.

Transaction and Booking Engine (REQ-TB-1 to 6).

- **Booking Selection**

Renters select available dates from the calendar, then the minimum start date must be tomorrow and the maximum rental duration is 90 days (with a warning if exceeded).

- **Cost Calculation**

The system instantly calculates the total cost, then it includes Rental Fee (price × duration),

Platform Commission (informational), Security Deposit, Expert Fee (if requested), and Delivery Fee (if applicable).

- **Soft Lock Mechanism**

A 5-minute soft lock is applied during checkout, then it prevents other users from booking the same dates.

- **Lock Expiration**

If checkout is not completed within 5 minutes, then the lock expires and the dates become available again.

- **Concurrent Booking Handling**

Concurrent booking attempts for overlapping dates are handled using database-level constraints, then one booking succeeds while the other receives: *"Dates no longer available."*

Payment Processing and Escrow (REQ-PP-1 to 6).

- **Payment Initiation & Escrow**

Upon owner acceptance, the mobile app connects to simulated payment gateways such as Vodafone Cash, Fawry, Paymob, credit card, and bank transfer, then all funds are securely held in escrow and are not released to the owner until the transaction reaches completed status.

- **UI & Double Payment Protection**

The payment button is disabled with a loading spinner during processing, then this prevents double-click issues and duplicate submissions.

- **Idempotency Handling**

Idempotency keys are used at the server level, then they prevent duplicate payment captures.

- **Failure Handling**

If payment fails, then the booking lock expires, the reserved dates are released, and the user can retry the payment process.

Active Rental Dashboard (REQ-TB-10).

- **Rental State Monitoring**

The system provides real-time monitoring of the rental state machine, then it displays a timeline view showing the current status such as *"Pending Hub Evaluation"*, *"Active"*, or *"Awaiting Return."*

- **Live Status Updates**

A live status banner is shown to the user, then it updates instantly via WebSocket whenever the rental state changes.

Real-Time Communication (REQ-TB-7).

- **Chat Functionality**

An embedded chat interface powered by Socket.IO enables users to communicate in real time, then it allows them to coordinate meeting details without sharing personal phone numbers.

- **Core Features**

The chat includes typing indicators, read receipts (✓✓), online status, and real-time message delivery, then these features enhance communication clarity and responsiveness.

- **Access Control**

Chat is only available between users who have active or pending rentals, then suspended users are restricted to read-only access.

- **Data Persistence**

Chat history is stored indefinitely, then users can access past conversations at any time.

Review System (REQ-RR-1 to 5).

- **Dual-Blind Review System**

A dual-blind system is used for reviews, then ratings are only published after both parties submit them to prevent retaliatory scoring.

- **Review Limits & Editing**

Each party can submit only one review per rental, then reviews cannot be edited after submission.

- **Disputed Rentals**

If a rental is disputed, both parties can still leave reviews, then those reviews are marked with a *"Disputed"* badge.

- **New User Handling**

New users with no reviews display *"New User"* instead of a 0-star rating, then this avoids negative perception.

- **Review Eligibility**

Reviews are only allowed when the rental status is *completed*, then users cannot review ongoing or cancelled rentals.

Return Flow with Two-Way Confirmation (REQ-TB-8 to 9).

- **Return Initiation**

The renter clicks *"I have returned the item"*, then the action is recorded in the system.

- **Owner Confirmation**

The owner clicks *"I confirm I received the item back"*, then this confirmation is also recorded.

- **Return Completion**

When both parties confirm, then the rental status is updated to *returned*.

- **Reminder & Escalation**

If only one party confirms within 48 hours, then a reminder is sent, then after 72 hours support is notified.

- **Auto-Completion Logic**

After 96 hours without any dispute, then the system automatically completes the return process.

- **Damage & Disputes**

If the owner reports damage, then clicking *"Report Issue"* opens a dispute modal to handle the case.

4.1.2 Web Application (Administrators, Experts, and Support Agents)

The Web Dashboard provides the operational backbone for platform governance, moderation, and data analytics.

KPI Dashboards (REQ-AP-1 to 4)

- **Dashboard Overview**

  A Angular.js-based analytics view displays aggregated metrics, then it includes Daily Active Users, Monthly Active Users, GMV, Total Escrow Value, Fraud Rates over time, Active Rentals, New Users, Pending Listings, and Open Disputes.

Gap Analysis Visualizer (REQ-AP-5)

- **Visualization**

  A heat map identifies supply-demand mismatches by product subcategory and region, then it classifies them as Critical, High, Moderate, or Low.

- **Admin Actions**

  Admins use this data to trigger targeted marketing campaigns, then CSV export allows offline analysis, then a manual trigger enables on-demand recalculation.

Fraud Moderation Panel (REQ-FD-4 to 6)

- **Admin Controls**

  The Admin can suspend accounts, block transactions, override flags as false positives, or whitelist users, then threshold adjustments show a preview of impact.

ID Verification Queue (REQ-AF-2)

- **Review Process**

  Support agents manually review National ID submissions against user data, then they approve or reject requests.

- **Rejection Handling**

If rejected, then a reason must be provided such as unclear photo or mismatch with profile.

Item Approval Queue (REQ-AF-3)

- **Review Workflow**

Admin reviews item details including title, description, photos, price, deposit, and category, then approves or rejects listings.

- **Outcomes**

Approved items become visible on the marketplace, then rejected items send a reason to the owner, then items resubmitted 3+ times are flagged as suspicious, then bulk approval is available for standard items.

Expert Hub Console (Renty Points) (REQ-EX-1 to 5)

- **Evaluation Process**

Experts evaluate items in pending hub evaluation state, then they fill a form including condition rating, description, photos, functional test, and estimated deposit.

- **Pre-Rental Rules**

Experts can approve or reject items, then they cannot evaluate their own listings, then the same expert must handle both pre- and post-rental checks.

- **Post-Rental Outcomes**

Experts determine if there is no damage, renter liability, or normal wear, then their decision is a recommendation while the Admin has final authority.

Support Ticket Management (REQ-ST-1 to 4)

- **Ticket Handling**

Support agents manage tickets by updating status from open to in-progress to resolved, then ticket types include payment, disputes, account issues, bug reports, and others.

- **Features**

Evidence uploads are supported with limits, then agents can add internal notes, then tickets auto-close after 5 days if no response, then escalation to Admin includes reason and evidence.

Dispute Resolution Interface (REQ-DR-1 to 3)

- **Review Process**

Admin reviews all dispute evidence including reports, photos, claims, and chat logs, then takes action.

- **Actions**

Actions include partial or full deposit forfeiture, additional charges, or refund overrides, then all actions are audit logged.

- **Reversal Rules**

Decisions can be reversed within 48 hours, then after that a Super Admin override is required.

Super Admin Console (REQ-SA-1 to 5)

- **Staff Management**

Super Admin manages staff accounts with full CRUD operations, then can suspend or unsuspend staff.

- **System Configuration**

Platform settings such as commission, policies, fees, and feature toggles can be configured, then maintenance mode can take the platform offline.

- **Audit & Control**

All system changes are logged and exportable, then the Super Admin can override any Admin decision including suspensions and deposit actions.

4.2 Non-functional Requirements

Performance (NFR-PF-1 to 3)

- **API Response Time**

  Core API sub-200ms response times under normal load, then ML inferences within 5 seconds asynchronously, then WebSocket message delivery under 500ms.

Scalability (NFR-PF-4 to 9)

- **Architecture & Indexing**

  Modular architecture supporting tens of thousands of concurrent users, then optimized B-Tree indices on search columns, foreign keys, and timestamp columns.

Usability (NFR-US-1 to 4)

- **Design & Accessibility**

  Mobile-first responsive design via Flutter, then material design principles, then accessibility considerations for admin dashboard.

Security (NFR-SE-1 to 5)

- **Protection Mechanisms**

  ASP.NET Core Identity with password hashing and salting, JWT access and refresh tokens, HTTPS/TLS 1.2+, Role-Based Access Control (RBAC) using JWT claims and authorization policies, and rate limiting (e.g., POST /api/v1/auth/login capped at 10 requests/minute per IP).

Reliability (NFR-RA-1 to 7)

- **Data & State Integrity**

  ACID compliance via PostgreSQL, then state machine prevents partial updates, then soft-delete ensures historical data preservation.

Maintainability (NFR-MA-1 to 3)

- **Code & Versioning**

  Modular code structure with consistent style guidelines, then comprehensive inline documentation, then version control with meaningful commit messages.

(Refer to Table 24: Non-Functional Requirements Summary and Table 25: Mobile vs. Web Feature Matrix in the List of Tables.)

4.3 UML Diagrams

(Refer to Figures 24-27 in the List of Figures for Use Case Diagram, Activity Diagrams, Sequence Diagrams, and UML Class Diagrams.)

The following UML artifacts document the system behavior:

1. **Use Case Diagram (Figure 24):** Maps all five actors (User/Renter/Owner, Admin, Super Admin, Expert, Support Agent) to their permitted system interactions, clearly delineating the RBAC boundaries.

2. **Activity Diagram (Figure 25):** Traces the complete rental flow from item discovery through payment, hub evaluation, active rental, two-way return confirmation, and completion/dispute resolution.

3. **Sequence Diagram (Figure 26):** Details the message-passing protocol between the Mobile App, Backend API, PostgreSQL, Payment Gateway, and WebSocket Server during the rental request-to-completion lifecycle.

4. **Class Diagram (Figure 27):** Models the backend data entities including Users, Items, Rentals, Orders, Payments, Reviews, Chats, Notifications, and their relationships.

Chapter 5 – System Design and Architecture

5.1 High-Level Architecture

Renty employs a robust, four-layer architecture heavily inspired by modern microservices patterns, designed for separation of concerns and high scalability.

(Refer to Figure 1: High-Level System Architecture Diagram in the List of Figures.)

Layer 1 — Presentation Layer (Clients). Composed of the Flutter Mobile Application and the Angular.js Web Dashboard. These clients maintain zero business logic and communicate exclusively via standard HTTP/REST requests and WebSocket connections. The Mobile App provides the primary interface for Renters and Owners, while the Web Dashboard serves Admins, Experts, Support Agents, and Super Admins.

Layer 2 — API and Application Logic Layer. A modular monolithic ASP.NET Core 8.0 backend that serves as the primary orchestrator, structured according to Clean Architecture principles. It utilizes the CQRS (Command Query Responsibility Segregation) pattern via MediatR to decouple business logic from API endpoints. The layer handles strict input validation using FluentValidation, executes state machine transitions, performs policy-based RBAC authorization checks against JWT claims, and orchestrates calls to external services (Paymob, Cloudinary) and background analytical pipelines. All API endpoints are logically grouped into separate controller modules, including: Authentication (/auth), Account (/account), Items (/items), Rentals (/rentals), Orders (/orders), Payments (/payments), Chat (/chats), Reviews (/reviews), Search (/search), Notifications (/notifications), Support (/support), Admin (/admin), and specialized modules for Analytics and Gap Analysis.

Layer 3 — Data Persistence Layer. A centralized PostgreSQL 14 relational database acts as the single source of truth. It is heavily fortified with foreign keys, ENUMs, CHECK constraints, triggers, and views

to guarantee data integrity. The database enforces business rules at the lowest possible layer, ensuring that even API bugs cannot corrupt critical business state (e.g., fn_prevent_self_rental trigger, date validation constraints, soft-delete semantics).

Layer 4 — Intelligence Layer Python-based asynchronous workers operating independently from the main API loop, then they consume raw interaction logs from *user_interactions*, *search_logs*, and transactional tables, then train ML models offline (LightFM recommendation engine), then expose inference via pre-computed database tables (*gap_results*, *kpis_snapshots_derived*) that the API layer reads without invoking Python directly, then this decoupled design ensures that AI processing does not block user-facing operations.

5.2 System Integration and Concurrency

Seamless integration between the transactional core and the AI modules is a critical design priority.

Recommendation Integration. User interactions (clicks, views, favorites, rentals) are logged asynchronously to the user_interactions table with action types. The LightFM model runs periodically to update latent embeddings, generating prediction scores and served via the /recommendations API endpoint without slowing down standard catalog queries. For cold-start users, the system falls back to popularity-based recommendations filtered by city.

Gap Analysis Integration. An analytics pipeline reads search_logs and rentals data, computes gap_score = searches ÷ available_items per category per city, and writes results to the gap_results table. The Admin dashboard reads these pre-computed results for heatmap visualization without running heavy analytical queries in real-time.

Concurrency Control. When a Renter initiates a booking, the backend acquires a row-level PostgreSQL advisory lock on the item-date combination for 5 minutes. This soft-lock prevents other users from booking overlapping dates during the checkout flow. If checkout completes, the lock is released and the rentals record permanently blocks the dates. If checkout times out, the lock expires and dates

become available again. Database-level UNIQUE constraints on (item_id, date_range) provide a final safety net against race conditions.

5.3 User Roles and RBAC Matrix

The platform strictly distinguishes between five actors defined by the roles and role_permissions mapping in the database:

(Refer to Table 16: User Roles and RBAC Permission Matrix in the List of Tables.)

1. **Users (Renters/Owners):** The general public. Can switch fluidly between renting out their own items (Owner role) and renting from others (Renter role). Both roles share a single user account with dual ratings (owner_rating and renter_rating).

2. **Admins:** High-level platform operators responsible for reviewing flagged fraud cases, observing KPIs, resolving disputed transactions, approving/rejecting item listings, and managing Expert assignments.

3. **Super Admins:** Possess maximum privileges including modifying platform commission percentages, creating/deleting staff accounts, toggling feature flags, and overriding any Admin decision with audit logging.

4. **Experts:** Specialized agents stationed at physical "Renty Points." Their sole permission domain is the item_verifications table. They cannot access user management, financial operations, or analytics—only item condition evaluation.

5. **Support Agents:** Staff dedicated to resolving support_tickets and engaging in customer service. They can read ticket histories and reply, but cannot issue refunds, modify fraud flags, or suspend users. Escalation to Admin is required for financial operations.

5.4 The Rental Lifecycle and State Machine

The core of Renty is the rental state machine defined in the rentals table. The status column utilizes a PostgreSQL ENUM (rental_status) to restrict the lifecycle strictly.

(Refer to Figure 11: Entity State Machine for Rental Lifecycle in the List of Figures.)

HAPPY PATH:

requested → confirmed → paid → in_delivery → active → returned → completed
↓
extended ←┘

DEVIATIONS (can happen from specific states):

- cancelled (from: requested, confirmed, paid)
- disputed (from: active, extended, returned)
- pending_hub_evaluation (between confirmed and active, when Expert check requested)
- evaluation_review (Renter reviewing Expert report)

State Definitions:

1. **requested**: The Renter initiates the rental. Dates are tentatively blocked via a 5-minute checkout lock. Payment is pending.

2. **confirmed**: The Owner reviews the request and accepts it. The Renter's payment (Rental Fee + Security Deposit) is captured into Escrow. If Owner doesn't respond within 24h, a reminder is sent; after 48h, the system auto-rejects and refunds 100%.

3. **pending_hub_evaluation (Conditional)**: For high-value items, or if either party requested an Expert check, the item is physically routed to a Renty Point for inspection.

4. **evaluation_review**: The Renter reviews the Expert's report and decides to Accept (rental proceeds to active) or Reject (rental cancelled, 100% refund, Expert fee charged to requester).

5. **active**: The Renter physically receives the item. The rental clock begins. Both parties have confirmed handover.

6. **returned**: The Renter returns the item. Two-way confirmation required. If Expert post-check was requested, status stays returned until Expert verifies.

7. **completed**: Final inspection passes. Escrow releases the Rental Fee (minus 15% commission) to the Owner's wallet and fully refunds the Security Deposit to the Renter.

8. **cancelled**: Either party aborts before active. Cancellation policy (Flexible/Moderate/Strict) determines refund percentages.

9. **extended**: The Renter requests additional days. Owner must approve. Supplementary payment is captured.

10. **disputed**: Item returns damaged or excessively late. Escrow is frozen. Admin reviews evidence and distributes the Deposit according to penalty_fault.

Transition Rules:

- Transitions are **one-directional**. No state can go backward except through Admin/Super Admin override.

- Each transition triggers a notification to both parties via WebSocket and is stored in the notifications table.

- Each transition is timestamped for complete audit trails.

5.5 Trust Mechanisms and Escrow

(Refer to Figure 12: Trust Workflow in the List of Figures.)

The platform enforces trust through three distinct layers:

KYC Verification. The id_verifications table tracks mandatory uploads of National IDs (front and back photos), while phone OTPs ensure users are reachable and traceable. Three verification levels: Email Verified (account activation), Phone Verified (renting/listing), ID Verified (high-value items and trust badge).

Financial Escrow. By holding all funds until the completed state, Renty prevents: Owners from fleeing with deposits before delivering items, Renters from refusing to pay for damages, and either party from circumventing platform commission. The escrow release follows a strict sequence: (1) Renter's deposit is refunded if no damage claim exists; (2) Platform's 15% commission is deducted from the rental fee; (3) Owner receives the remaining 85% of the rental fee credited to their wallet.

Expert Hubs (Renty Points). Operating as a physical buffer, these hubs eliminate the "he said, she said" dynamic. Experts act as impartial third parties, verifying the exact functional and aesthetic condition of the asset before and after the rental period. The **Continuity Rule** mandates that the same Expert who performed the pre-rental check must perform the post-rental check, ensuring consistency in condition assessment. If the assigned Expert is unavailable (sick, quit), Admin must manually reassign with a "Continuity Rule Violated" warning. Expert's post-rental verdict is a RECOMMENDATION—Admin's decision is final.

5.6 Notification System Architecture

(Refer to Table 19: Notification Event Types and Figure 28: Notification System Event Flow Diagram.)

The platform implements over 27 distinct notification event types covering every significant system action. Notifications are delivered via two channels: real-time WebSocket push to the client's bell icon (with unread badge count), and persistent storage in the notifications database table with is_read flag and redirect_url for click navigation. Optional email notifications are sent for critical events (payment confirmations, disputes, account suspensions).

Key notification events include: rental_request_received, rental_confirmed, rental_rejected, payment_success, payment_failed, item_shipped, item_received, return_reminder (24h before end_date), item_returned, rental_completed, extension_request, extension_approved/rejected, new_message, dispute_opened/update/resolved, item_approved/rejected, late_fee_alert, no_show_warning, auto_cancelled, deposit_refunded/forfeited, new_review, account_suspended, and owner_request_timeout.

Chapter 6 – Database and Backend Architecture

6.1 Database Design Philosophy

The PostgreSQL 14 relational database serves as the single source of truth for the Renty platform. A deliberate design decision was made to push as much business logic as possible into the database layer, following a "defense-in-depth" philosophy where data integrity constraints exist at the schema level, independent of application code. This ensures that even API bugs or malicious requests cannot corrupt critical business state.

(Refer to Figure 13: System Database Schema (ERD) and Table 17: Database Tables Summary.)

6.2 Core Tables and Relationships

6.2.1 User Management

The users table is the central entity, storing authentication credentials (bcrypt-hashed passwords), profile metadata, verification status flags (email_verified, phone_verified, id_verified), and dual ratings (owner_rating, renter_rating). Users are never hard-deleted; instead, the is_deleted flag implements soft-delete semantics, preserving referential integrity for historical rental and financial records. The last_login timestamp and login_count field provide behavioral data consumed by both the recommendation engine and the fraud detection module.

The roles table defines the RBAC hierarchy (User, Admin, Super Admin, Expert, Support Agent), while the role_permissions junction table maps specific permissions (manage_items, manage_users, view_analytics, verify_items, manage_support) to each role. Authentication middleware extracts the user's role from the JWT payload and checks permissions before routing any request.

6.2.2 Item Management

The items table supports the full lifecycle of rental listings. Key columns include daily_price, weekly_price (optional), monthly_price (optional), deposit_amount, cancellation_policy (ENUM: flexible, moderate, strict), and status (ENUM: pending_approval, available, rented, sold, suspended, deleted). The approved_by column references the Admin who approved the listing, creating an audit trail for content moderation.

Items are organized via a hierarchical taxonomy through categories and subcategories tables. The item_images table supports up to 10 images per item with is_primary flag and display_order for gallery sequencing. The item_availability table manages calendar blocking with start_date, end_date, and is_blocked flag for owner-defined blackout periods.

6.2.3 Rental and Order Management

The rentals table is the transactional core, implementing the 9-state machine described in Chapter 5. Key columns include renter_id, owner_id, item_id, start_date, end_date, total_price, deposit_amount, commission_rate (locked at creation time), status (PostgreSQL ENUM), and cancellation_policy (snapshot from item at booking time). The orders table links rentals to payments with order_type (ENUM: rental, purchase, extension, deposit) and status (ENUM: pending, paid, refunded, partially_refunded, cancelled).

The payments table records all financial transactions with payment_method (ENUM: vodafone_cash, fawry, paymob, credit_card, bank_transfer), amount, status (ENUM: pending, success, failed, refunded), and transaction_id from the payment gateway. An idempotency_key column prevents duplicate payment captures.

6.2.4 Trust and Verification

The id_verifications table stores user identity verification records with front_image_url, back_image_url, verification_status (ENUM: pending, approved, rejected), and reviewed_by referencing the Support Agent who processed the verification. Rejection requires a rejection_reason text field.

The item_verifications table records Expert evaluations at Renty Points with condition_rating (ENUM: excellent, good, fair, poor, unacceptable), description, functional_test_passed (BOOLEAN), recommended_deposit, verification_type (ENUM: pre_rental, post_rental), and verdict (ENUM: approved, rejected, damage_found_renter_liable, damage_found_normal_wear). The expert_id foreign key enforces the Continuity Rule at the application level.

6.2.5 Communication

The chats table creates conversation threads between users with active or pending rentals. The messages table stores individual messages with sender_id, content, is_read, read_at, and message_type (ENUM: text, image, system). System messages are auto-generated for state transitions (e.g., “Rental has been confirmed”). The notifications table handles platform-wide notifications with type (28+ event types), redirect_url, is_read, and created_at.

6.2.6 Support and Disputes

The support_tickets table tracks user issues with type (ENUM: general, billing, dispute, report), status (ENUM: open, in_progress, resolved, closed), priority (ENUM: low, medium, high, critical), and assigned_to referencing a Support Agent. The ticket_replies table stores the conversation thread with is_internal flag for staff-only notes. The disputes table links to rentals and stores evidence_urls, description, resolution (text), penalty_amount, penalty_fault (ENUM: renter, owner, both, none), and resolved_by for audit logging.

6.3 Advanced Database Constraints

6.3.1 Triggers

fn_prevent_self_rental: A BEFORE INSERT trigger on the rentals table that prevents a user from renting their own item. The trigger joins rentals.item_id to items.owner_id and compares against rentals.renter_id. If they match, the trigger raises an exception: "Cannot rent your own item."

fn_prevent_self_verification: A BEFORE INSERT trigger on item_verifications that prevents an Expert from verifying an item they own.

fn_update_timestamps: A BEFORE UPDATE trigger on multiple tables that automatically sets the updated_at column to NOW() on any row modification.

6.3.2 CHECK Constraints

- end_date > start_date on rentals — prevents invalid date ranges.
- deposit_amount >= 0 on items — prevents negative deposits.
- daily_price > 0 on items — ensures pricing validity.
- commission_rate >= 0 AND commission_rate <= 100 — bounds commission percentages.
- rating >= 1 AND rating <= 5 on reviews — restricts ratings to valid range.

6.3.3 Views

The database includes several pre-computed views for analytics:

- vw_active_rentals — joins rentals, items, users (renter), users (owner) for active rental monitoring.
- vw_revenue_summary — aggregates completed rental revenue by month, category, and city.
- vw_user_activity — computes per-user metrics (total rentals, total spent, total earned, average rating).
- vw_gap_analysis — cross-references search_logs against items availability by subcategory.

6.3.4 Indexing Strategy

B-Tree indices are applied on all frequently queried columns: items.status, items.category_id, items.city, rentals.status, rentals.renter_id, rentals.owner_id, payments.status, notifications.user_id, notifications.is_read, search_logs.query, and all created_at timestamp columns. Composite indices on (item_id, start_date, end_date) in rentals optimize availability calendar queries.

6.4 Backend API Architecture

6.4.1 ASP.NET Core Endpoint Structure

The API is organized into cleanly separated endpoint groups mapped under /api/v1:

- **Authentication:** POST /auth/register, POST /auth/login, POST /auth/refresh, POST /auth/logout
- **Items:** Full CRUD with GET /items (catalog + filters), GET /items/{id}, etc.
- **Rentals:** POST /rentals, PUT /rentals/{id}/confirm, etc.
- **Admin:** GET /admin/items/pending, GET /admin/kpis/latest, GET /analytics/fraud

6.4.2 Request Processing Pipeline

Every request passes through a standardized ASP.NET Core middleware and MediatR pipeline:

1. **Rate Limiting**: ASP.NET Core RateLimiter configured per endpoint
2. **CORS**: Configured to allow only whitelisted origins
3. **Authentication**: JWT Bearer authentication middleware
4. **Authorization**: Policy-based authorization checks
5. **Validation**: FluentValidation pipeline behavior
6. **CQRS Dispatch**: The endpoint delegates the request object to MediatR (ISender.Send())
7. **Handler Execution**: The dedicated MediatR command/query handler executes the business logic and interacts with the database via Entity Framework Core.

6.4.3 Real-Time Infrastructure (SignalR)

SignalR is integrated into the ASP.NET Core backend to manage real-time, bi-directional communication:

- **Chat Groups**: Each chat thread maps to a SignalR Group. Users are added to their respective groups upon connection. Messages are broadcast to group members instantly, supporting typing indicators and read receipts.

- **Notification Delivery**: SignalR automatically maps active WebSocket connections to authenticated User IDs using JWT claims. Server-side events (state transitions, new messages, fraud alerts) use Clients.User(userId) to push targeted notifications directly to the specific user without needing manual channel subscriptions.

- **Rental Status Updates**: When a rental's status changes via a MediatR command, an event is emitted through the SignalR Hub to both the Renter and Owner, delivering the new status, timestamp, and relevant metadata instantly.

- **Automatic Fallback**: If a native WebSocket connection fails or is blocked by a network proxy, the SignalR client automatically and seamlessly degrades to Server-Sent Events (SSE) or Long Polling, ensuring real-time functionality without requiring manual REST polling.

6.5 Environment and Configuration

The backend uses environment-specific configuration through .env files:

- DATABASE_URL — PostgreSQL connection string.
- JWT_SECRET, JWT_REFRESH_SECRET — Signing keys for token pairs.
- JWT_ACCESS_EXPIRES_IN (default: 15m), JWT_REFRESH_EXPIRES_IN (default: 7d) — Token lifetimes.
- BCRYPT_SALT_ROUNDS (default: 12) — Password hashing complexity.
- PLATFORM_COMMISSION_PERCENT (default: 15) — Configurable via Super Admin.
- CLOUDINARY_URL — Image upload service configuration.
- PAYMOB_API_KEY, FAWRY_MERCHANT_CODE — Payment gateway credentials (simulated).
- CHECKOUT_LOCK_DURATION_MS (default: 300000) — 5-minute booking lock.

Chapter 7 – Intelligent Enhancements (AI Modules)

The intelligence layer of Renty comprises three autonomous analytical subsystems that operate as supporting components to the transactional core: the Hybrid Recommendation Engine, the Unsupervised Fraud Detection Module, and the Supply-Demand Gap Analysis Framework. This chapter provides detailed technical descriptions of each module's methodology, training process, and performance evaluation.

7.1 Hybrid Recommendation System

7.1.1 Data Preparation and Exploration

The recommendation engine was trained on a comprehensive e-commerce dataset containing 55,666 transactions across 17,293 unique customers and 130 distinct products, collected over a 30-month period (January 2020 – June 2022). The dataset comprises five interconnected tables:

(Refer to Tables 1-4 in the List of Tables for Dataset Characteristics, Dataset Overview, Data Dictionary, and Key Entity Relationships.)

- **Sales:** 56,046 records with OrderDate, CustomerKey, ProductKey, OrderQuantity, UnitPrice.
- **Products:** 293 records with ProductKey, ProductSubcategoryKey, ModelName, ProductCost, Color, Size, ProductDescription.
- **Customers:** 18,148 records with CustomerKey, FirstName, BirthDate, CivilStatus, AnnualIncome, TotalChildren, Gender, Occupation, Education.
- **Returns:** 1,809 records linking returned items to territories.
- **Territories:** 10 records defining geographic regions.

The interaction matrix exhibits extreme sparsity of 97.52%, with an average of 3.22 orders per user—a challenging condition for collaborative filtering algorithms and a primary motivation for adopting a hybrid approach.

7.1.2 Feature Engineering

User Features. A rich user profile was constructed by combining demographic attributes with behavioral metrics:

- **Demographics:** Income bracket (5 tiers), occupation (5 categories), gender (binary), marital
- **Behavioral RFM Metrics:** Recency (days since last transaction), Frequency (total order count),

7.1.4 Training Configuration and Hyperparameter Tuning

(Refer to Tables 5-7 in the List of Tables for Regularization Strategy, Training Configuration, and Hyperparameter Tuning Results.)

The final training configuration:

Parameter	Value	Justification
Loss Function	WARP	Optimized for top-N ranking
No. Components	50	Balances capacity and overfitting
Learning Rate	0.05	Stable convergence
Item Alpha (L2)	0.00001	Light regularization
User Alpha (L2)	0.00001	Light regularization
Epochs	60	Convergence achieved
Threads	4	Parallel processing

Six hyperparameter configurations were systematically evaluated:

Config	Components	Regularization	Precision@10	Overfit Gap
Baseline	50	0.0	0.1217	0.0029
Light Reg	50	0.00001	0.1215	0.0004
Medium Reg	50	0.0001	0.1209	-0.0035
High Comp	70	0.0001	0.1193	-0.0137
Low Comp	30	0.0001	0.1211	0.0156
Strong Reg	50	0.001	0.1166	-0.0159

The “Light Reg” configuration was selected as the optimal balance between accuracy (Precision@10 = 0.1215) and generalization (Overfitting Gap = 0.0004, a 80.8% reduction from baseline).

7.1.5 Performance Results

(Refer to Table 8: Final Model Performance in the List of Tables and Figure 34.)

Metric	Baseline	Hybrid	Improvement
Test AUC	0.7497	0.8421	+12.3%

Metric	Baseline	Hybrid	Improvement
Precision@5	0.1043	0.1511	+44.9%
Precision@10	0.0938	0.1208	+28.8%
Precision@20	0.0664	0.0868	+30.7%
Recall@5	0.1932	0.2708	+40.2%
Recall@10	0.3452	0.4410	+27.8%
Recall@20	0.4892	0.6416	+31.2%
Overfitting Gap	0.2501	0.0479	-80.8%

The hybrid approach outperformed the baseline across all metrics while dramatically reducing the overfitting gap, demonstrating strong generalization capability for production deployment.

7.1.6 Inference and API Integration

(Refer to Table 9: API Response Format.)

The recommendation API exposes a GET /recommendations/:userId endpoint returning a ranked JSON array:


```
{
  "recommendations": [
    {"rank": 1, "productKey": 477, "modelName": "Water Bottle", "description": "AWC logo...", "score": 0.8542},
    {"rank": 2, "productKey": 214, "modelName": "Mountain Bike", "description": "...", "score": 0.7891}
  ]
}
```


Cold-start fallback: If the user has fewer than 3 interactions, the system returns popularity-based recommendations filtered by the user's city, sorted by view count and average rating.

7.2 Unsupervised Fraud Detection

7.2.1 Motivation and Approach

As a nascent marketplace without historical fraud examples, Renty cannot employ supervised classification for fraud detection. The Isolation Forest algorithm was selected for its ability to identify anomalies without labeled training data, its efficiency with high-dimensional feature spaces, and its interpretable anomaly scores.

7.2.2 Feature Engineering

A comprehensive behavioral velocity vector is constructed for each user at checkout time:

(Refer to Table 10: Fraud Detection Feature Engineering Summary.)

- **Temporal Features:** Hour of transaction, day of week, time since account creation, time since last transaction.
- **Velocity Features:** Orders in last 1h, 24h, 7d; messages sent per hour; distinct IP addresses in 24h; listing creation rate.
- **Financial Features:** Current order value / historical average, quantity-to-income ratio, deposit-to-item-value ratio.
- **Behavioral Features:** Cancellation rate, dispute rate, average review score received, account age.
- **Composite Indicators:** "New account + high value" flag (account < 7 days AND order value > 2× platform median), "Velocity spike" flag (today's activity > 3σ above personal average).

All features are normalized via z-score standardization before model ingestion.

7.2.3 Model Configuration

The Isolation Forest model is configured with:

- n_estimators = 100 (number of isolation trees)
- max_samples = 256 (subsample size per tree)
- contamination = 0.05 (expected proportion of anomalies, tuned for 5% flagging rate)
- random_state = 42 (reproducibility)

7.2.4 Scoring and Decision Logic

(Refer to Table 11: Fraud Score Thresholds and Actions.)

The model generates an anomaly score between 0.0 and 1.0 for each user:

Score Range	Decision	System Action
0.0 – 0.49	safe	Normal access, transaction proceeds

Score Range	Decision	System Action
0.50 – 0.79	flagged	Admin notified via dashboard alert, user can still transact but is monitored
0.80 – 1.0	blocked	Checkout locked, auto-generated Support ticket, Admin must review

The threshold (default 0.75) is configurable by Admin with an impact preview ("Changing threshold to 0.70 would flag 12 additional users"). Fraud indicators include: listing spikes (>10 listings/day), high cancellation rate (>50%), new account + high value, low review average (<2.0 stars), and multiple devices per session.

Admin actions on flagged users: Block (prevent all transactions), Whitelist (exclude from future flagging), Dismiss (mark as false positive for model recalibration), Suspend Account (full platform ban).

7.2.5 Integration with Rental Lifecycle

Fraud scoring is triggered at two critical points:

1. **Checkout Initiation:** Before payment capture, the system scores the user. If blocked, checkout is immediately halted.

2. **Account Creation:** New registrations are scored based on IP analysis and registration velocity to catch bot accounts.

Fraud scores decay gradually after 5+ successful, dispute-free rentals, allowing reformed users to regain full access.

7.3 Supply-Demand Gap Analysis

7.3.1 Pipeline Overview

The Gap Analysis framework continuously monitors marketplace equilibrium by comparing active supply (listed, approved items) against observed demand (search queries, rental requests, and unfulfilled browsing patterns). The pipeline runs as a nightly batch job and stores results in the gap_results table.

7.3.2 Gap Score Computation

For each product subcategory in each city:

demand_volume = COUNT(search_logs WHERE query MATCHES subcategory) + COUNT(rejected_rentals WHERE reason = 'no_availability')

supply_count = COUNT(items WHERE status = 'available' AND subcategory_id = X AND city = Y)

gap_score = (demand_volume / MAX(supply_count, 1)) × value_multiplier

The value_multiplier accounts for the average rental value of the subcategory, ensuring that high-value categories with small gaps receive appropriate attention.

7.3.3 Severity Classification

(Refer to Table 12: Gap Severity Classification Metrics.)

Level	Gap Score Range	Business Interpretation	Recommended Action
Low	0 ≤ G < 50	Oversupply / Balanced	Monitoring; focus on marketing
Moderate	50 ≤ G < 100	Healthy growth	Standard acquisition pace
High	100 ≤ G < 200	Under-supply: lost revenue	Priority sourcing; vendor incentives
Critical	G ≥ 200	Severe shortage: broken market	EMERGENCY ACTION; direct intervention

7.3.4 Dashboard Integration

Gap results are visualized through:

- **Heatmap View (Figure 38):** Color-coded matrix of subcategory × city with severity levels.

- **Bar Chart (Figure 36):** Top 10 gap categories ranked by gap score.

- **Severity Distribution (Figure 37):** Pie chart showing percentage of subcategories in each severity tier.

- **Trend Analysis:** Week-over-week gap score changes to identify worsening or improving conditions.

- **CSV Export:** Admin can download complete gap analysis for offline strategic planning.

7.3.5 Automation and Manual Trigger

The pipeline runs automatically every night at 2:00 AM UTC. Additionally, a “Run Gap Analysis Now” button on the Admin dashboard triggers an immediate recalculation with a loading indicator and timestamp of last run. The manual trigger is rate-limited to once per hour to prevent excessive database load.

Chapter 8 – System Implementation

This chapter describes the practical realization of the system design, documenting the technology stack selection, development environment configuration, and key implementation details for each system layer.

8.1 Technology Stack

Technology	Version	Purpose
ASP.NET Core	8.0	Backend RESTful API framework and runtime
PostgreSQL	16	Relational database with ENUM, trigger, and constraint support
Entity Framework Core	8.0	ORM for database queries, migrations, and schema management
SignalR	8.0	Real-time bidirectional WebSocket communication
MediatR	12.x	Implementing CQRS (Command/Query Responsibility Segregation) pattern
FluentValidation	11.x	Robust request payload and business rule validation
ASP.NET Core Authentication	8.0	JWT generation, verification, and policy-based RBAC authorization
BCrypt.Net-Next	4.x	Password hashing with configurable work factor
AspNetCore.RateLimiting	8.0	Built-in middleware for API rate limiting per endpoint
CloudinaryDotNet	1.x	File upload handling and cloud image storage
IHostedService / BackgroundService	8.0	Native ASP.NET Core scheduling for nightly batch jobs (Gap Analysis, Overdue checks)

appsettings.json / Environment Variables	8.0	Configuration and environment variable management

*(Refer to Table 23: Technology Stack Summary in the List of Tables.)*8.1.1 Backend Technologies

8.1.2 Frontend Technologies (Web Dashboard)

Technology	Version	Purpose
Angular	19.x	Component-based, enterprise-grade web UI framework
Angular Router	19.x	Client-side routing, navigation, and route guards (Auth/Role protection)
RxJS	7.x	Reactive programming for state management and async data streams
HttpClientModule	19.x	Built-in Angular HTTP client for API communication
Chart.js / ng2-charts	4.x	Data visualization for Admin KPI and Gap Analysis dashboards
@microsoft/signalr	8.0	Real-time notification and chat integration client

8.1.3 Mobile Technologies

Technology	Version	Purpose
Flutter	3.x	Cross-platform mobile development framework (iOS + Android)
Dart	3.x	Programming language for Flutter app

Technology	Version	Purpose
		logic
GoRouter / Navigator 2.0	—	Mobile navigation stack, deep linking, and route management
flutter_secure_storage	—	Encrypted local persistent storage for JWT tokens and preferences
google_maps_flutter	—	Map-based item discovery and Renty Point geolocation
image_picker	—	Camera and gallery access for listing photos and damage evidence

8.1.4 AI/Analytics Technologies

Technology	Version	Purpose
Python	3.10+	ML model development and training
LightFM	1.17	Hybrid recommendation model
scikit-learn	1.x	Isolation Forest for fraud detection
Pandas	2.x	Data manipulation and feature engineering
NumPy	1.x	Numerical operations and matrix computations
SciPy	1.x	Sparse matrix operations for feature encoding
TfidfVectorizer (sklearn)	—	Text feature extraction from product descriptions
Matplotlib / Seaborn	—	Model evaluation visualization
psycopg2	—	Direct PostgreSQL connectivity for batch pipelines

8.2 Development Environment

The development environment was configured for collaborative team development:

- **Version Control:** Git with GitHub for repository management, branching strategy (main, develop, feature branches), and pull request reviews.
- **IDE:** Visual Studio Code with ESLint, Prettier, and Python extensions.
- **Database Management:** pgAdmin 4 and DBeaver for schema inspection and query testing.
- **API Testing:** Postman collections with environment variables for development, staging, and production configurations.
- **Deployment (Planned):** Docker containerization with docker-compose for local development parity. Cloud deployment targets include Railway (backend), Vercel (web dashboard), and Expo (mobile app distribution).

8.3 Backend Implementation Highlights

8.3.1 Authentication Flow

The authentication system implements a dual-token strategy:

1. **Access Token:** Short-lived (15 minutes), signed with `JWT_SECRET`, contains `userId`, `role`, and `permissions` claims. Attached to every API request via `Authorization: Bearer <token>` header.

2. **Refresh Token:** Long-lived (7 days), signed with `JWT_REFRESH_SECRET`, stored securely in the client. Used to obtain new access tokens without requiring re-login.

Token rotation: When a refresh token is used, it is invalidated and a new pair (access + refresh) is issued, preventing token replay attacks.

8.3.2 State Machine Enforcement

Rental state transitions are enforced through a centralized `transitionRentalStatus(rentalId, newStatus, userId)` function that:

1. Fetches the current rental state from the database.

2. Validates the requested transition against the allowed transition map (e.g., confirmed can only transition to paid, cancelled, or pending_hub_evaluation).

3. Verifies the requesting user has permission for this transition (e.g., only the Owner can transition from requested to confirmed).

4. Executes the transition within a database transaction with row-level locking.

5. Emits a WebSocket notification to both parties.

6. Creates a notifications record in the database.

8.3.3 Escrow Implementation

The escrow logic is implemented as a database transaction that atomically:

1. Captures payment from the Renter's selected payment method.

2. Creates a payments record with status = 'success' and links it to the orders table.

3. Updates the rental status to paid.

4. Does NOT credit the Owner's wallet — funds remain in platform escrow.

Release occurs only upon completed status:

1. Calculate commission: commission = rental_fee × commission_rate.

2. Calculate owner payout: payout = rental_fee - commission.

3. Credit payout to Owner's wallet balance.

4. Refund security deposit to Renter's original payment method.

5. Log all financial movements in the wallet_transactions table.

8.4 Mobile Application Implementation

The Flutter mobile application was developed using a widget-based architecture with clear separation between:

- **Screens**: Full-page views corresponding to navigation routes (e.g., HomeScreen, BrowseScreen, ItemDetailScreen, RentalDashboardScreen, ChatScreen, ProfileScreen).

- **Widgets**: Reusable UI components (e.g., ItemCard, RentalStatusBanner, StarRating, CalendarPicker, PaymentMethodSelector).

- **Repositories / Services**: API communication layer wrapping Dio calls, utilizing interceptors for automatic JWT token injection and refresh on 401 Unauthorized responses.

- **State Management**: Utilizing BLoC (Business Logic Component) or Riverpod for predictable global state management (e.g., AuthBloc, NotificationBloc, ChatBloc).

- **Navigation**: Managed by GoRouter, utilizing a ShellRoute for the persistent bottom navigation bar and nested stack routing for multi-step flows (e.g., Item Detail → Rent Modal → Checkout → Confirmation).

Key mobile-specific implementations:

- **Push Notifications:** WebSocket connection established on app launch, maintained in background. Notifications trigger native push alerts on Android.

- **Offline Support:** AsyncStorage caches recently viewed items and active rental data for offline viewing. Queued actions (message sends, status updates) sync when connectivity is restored.

- **Image Upload Pipeline:** Camera/gallery selection → client-side compression (max 1080px, quality 80%) → Cloudinary upload → URL stored in database. Progress indicator during upload.

- **Map Integration:** Utilizing google_maps_flutter to render interactive maps with custom markers showing item locations, interactive price bubbles, and cluster grouping for dense areas to locate items and Renty Hubs.

Chapter 9 – User Interface and Experience

This chapter documents the user experience design for both the web application (the primary, fully-featured interface for all user roles and administrators) and the companion mobile application (optimized for on-the-go access by Renters and Owners), referencing the comprehensive UI specifications defined in the System Design documentation.

9.1 Mobile Application and Web UX

(Refer to Figures 14-18 in the List of Figures for Mobile Application screenshots and flow diagrams.)

9.1.1 Home Page and Discovery

The home page presents a curated feed combining AI-powered recommendations with trending items and recently listed gear. The layout prioritizes visual discovery with large item cards showing the primary image, daily price, condition badge, and owner rating. A sticky search bar at the top provides instant access to text-based search. A reusable map component powered by the Google Maps API is integrated for location-based item listing, enabling Owners to pin their item's exact coordinates using address autocomplete or manual coordinate entry, which supports proximity-based discovery in the marketplace.

For logged-in users, the recommendation engine (LightFM) generates personalized suggestions displayed in a horizontal carousel labeled “Recommended for You.” For new users with fewer than 3 interactions, the system falls back to popularity-based recommendations filtered by the user’s detected city. Every search query entered by the user is logged to `search_logs` for the Gap Analysis pipeline.

9.1.2 Item Detail and Booking Flow

Tapping an item card opens the Item Detail screen with an image gallery (swipeable, supporting pinch-to-zoom), detailed description, pricing tiers (daily/weekly/monthly), cancellation policy badge with tooltip explanation, owner profile with dual rating (Owner Rating + response time), location on map, and user reviews. Below the gallery, a prominent “Rent Now” CTA opens the Rent Modal.

The Rent Modal presents an interactive availability calendar where blocked dates are grayed out, selected dates are highlighted, and the system auto-calculates the total cost in real-time as the user selects dates: `Rental Fee + Deposit + Expert Fee (if selected) + Delivery Fee (if applicable)`. Optional toggles for “Request Expert Check Before” and “Request Expert Check After” are available. Upon confirmation, the 5-minute checkout lock is initiated.

9.1.3 Active Rental Dashboard

Once a rental is booked, the Active Rental Dashboard provides a timeline visualization of the rental lifecycle. Each state in the state machine is represented as a node in a vertical timeline, with the current state highlighted. Status changes are pushed in real-time via WebSocket, updating the timeline with animation. Actions available depend on the current state:

- requested → Renter sees “Waiting for Owner Confirmation” with a countdown timer (48h).
- confirmed → “Proceed to Payment” button.
- paid → “Awaiting Delivery” with tracking info.
- active → countdown to return date, “Request Extension” button, “Chat with Owner” button.
- Day before end_date → “Return Reminder” notification.
- returned → “Awaiting Owner Confirmation” with 48h countdown.

9.1.4 Chat Interface

The embedded chat provides a messaging experience with:

- Real-time message delivery via Socket.IO.

- Typing indicators (“Owner is typing…”).

- Read receipts (single check = delivered, double check = read).

- Online/offline status indicator.

- Image sharing capability.

- System messages for automated state transitions (“Rental has been confirmed”).

- Chat is disabled if the user is suspended (read-only mode).

9.1.5 Profile and Settings

The profile page displays the user’s dual identity: their “As Owner” profile (items listed, owner rating, total earnings, verification badges) and their “As Renter” profile (rentals completed, renter rating, total

spent). Settings include notification preferences, payment method management, and account deletion with soft-delete confirmation.

9.2 Web Dashboard UX

(Refer to Figures 19-23 in the List of Figures for Web Dashboard screenshots.)

9.2.1 Admin Dashboard — KPI Overview

The main Admin dashboard presents a grid of metric cards showing real-time KPIs: Daily Active Users, Monthly Active Users, Total Revenue (MTD), Active Rentals, Pending Listings, Open Disputes, Fraud Alerts, and Escrow Balance. Each card includes a sparkline showing 30-day trends and a color-coded indicator (green = healthy, yellow = warning, red = critical). Below the metric grid, interactive charts display revenue over time, user growth, and rental completion rates using Recharts.

9.2.2 Gap Analysis Heatmap

A dedicated analytics page presents the Gap Analysis results as an interactive heatmap where rows represent product subcategories and columns represent cities/regions. Cell colors range from deep green (oversupply) through yellow (balanced) to deep red (critical shortage). Clicking a cell opens a detail panel showing the exact gap score, supply count, demand volume, and recommended action. A “Run Gap Analysis Now” button triggers immediate recalculation with a loading spinner. Export to CSV is available for offline strategic planning.

9.2.3 Expert Hub Console

The Expert interface is deliberately simplified to a single queue view showing items pending verification. Each queue item displays: item name, owner name, rental ID, verification type (pre-rental or post-rental), and assignment timestamp. The verification form requires: overall condition rating (5-tier dropdown), detailed text description (minimum 50 characters), minimum 3 photos (uploaded via file picker), functional test result (pass/fail toggle), and estimated deposit recommendation. The Continuity Rule is enforced at the UI level: for post-rental checks, the system only assigns the item to the same Expert who performed the pre-rental check, displaying a warning if that Expert is unavailable.

9.2.4 Support Ticket Management

Support agents see an inbox-style interface with tickets sorted by priority and age. Each ticket shows: ticket ID, user name, type, priority badge, assigned agent, creation date, and last activity. Opening a ticket reveals the full conversation thread (user messages and agent replies), attached evidence, linked rental (if applicable), and internal notes visible only to staff. Quick actions include: change status, change priority, reassign to another agent, and escalate to Admin with reason and suggested action.

(Refer to Table 25: Mobile vs. Web Feature Matrix comparing capabilities across platforms.)

9.3 Super Admin Interface

The Super Admin role has exclusive access to the following administrative modules on the web dashboard, then it includes KPI Dashboard, User Management, Role and Permission Management, Item Moderation Queue, System Settings, Gap Analysis Console, and Fraud Detection Panel.

9.3.1 KPI Dashboard

The main admin landing page presents metric cards (Total Revenue, Active Users, Total Rentals, Active Items, Fraud Rate, Recommendation CTR), then a “Refresh Metrics” button fetches the latest KPI snapshot, then skeleton loading states are shown during data fetching.

9.3.2 User Management

A paginated table displays users with avatar, name, email, roles, and status, then actions include edit user, assign roles, and delete user, then a search bar filters by name or email, then a “Create User” button allows manual user creation.

9.3.3 Role and Permission Management

A two-panel layout lists roles on the left and permissions on the right, then selecting a role shows permissions grouped by modules, then permissions are toggled via checkboxes, then changes are saved using a “Save” button.

9.3.4 Item Moderation Queue

A table shows pending items with title and creation date, then actions include approve (set active) or reject (block item), then an empty state appears when no items are pending.

9.3.5 System Settings

Global settings include commission rate input and maintenance mode toggle, then commission is saved via a button, then maintenance mode disables platform operations.

9.3.6 Gap Analysis Console

A dashboard shows supply-demand imbalance metrics, then summary cards display total categories, high gap count, and top opportunity, then a table lists demand, supply, gap score, and recommendations, then a “Run Live Analysis” button triggers recalculation.

9.4 Support Agent Interface

9.4.1 Ticket Queue

An inbox-style table displays tickets with ID, user, type, status, and creation date, then supports pagination and loading states, then a “New Ticket” button allows creation, then visibility depends on role.

9.4.2 Ticket Detail View

Opening a ticket shows the full conversation with messages, timestamps, and attachments, then agents can update status and respond.

9.4.3 Dispute Resolution Workspace

A form allows handling disputes with rental ID, resolution type, and amount fields, then agents add internal notes, then executing resolution updates wallets immediately.

9.5 Expert (Renty Point) Interface

9.5.1 Assignment Queue

A table shows items pending verification with item info, check type, rental dates, and assignment date, then a “Verify” button opens the verification form, then supports RTL layout, then an empty state is shown when no assignments exist.

9.5.2 Verification History

A view displays past completed verifications, then it provides an audit trail for inspections.

9.5.3 Continuity Rule Enforcement

The system ensures the same expert performs pre- and post-rental checks, then assignments are automatically routed to maintain consistency.

Chapter 10 – Industrial Application and Business Model

This chapter provides an exhaustive analysis of Renty's business model, covering revenue mechanics, trust enforcement, operational policies, and edge-case scenario handling. The design draws from the comprehensive business model specification developed during the system design phase.

10.1 Revenue Streams

Renty operates on a commission-based revenue model with three primary income sources:

Revenue Stream	Mechanism	Default Rate
Rental Commission	Platform deducts a percentage from every successful rental fee before paying the owner	15% (configurable by Super Admin)
Sales Commission	Same percentage applied to outright item purchases	Same rate
Expert Evaluation Fee	Charged to the party who requests the hub inspection (or split 50/50 if both parties request)	Fixed fee per inspection (set by Admin)

Critical Design Decision: The commission rate is **locked at rental creation time**. If the Super Admin changes the platform rate from 15% to 20%, the change only affects NEW rentals. Existing rentals continue at their locked rate, preventing retroactive changes that would violate user expectations.

10.2 The Complete Rental Lifecycle (Business Perspective)

10.2.1 Discovery and Negotiation

Renters browse the catalog, utilize search (logged for Gap Analysis), or receive AI-driven recommendations. Public threaded comments (max 3 levels deep) allow questions about items without requiring private chat. Private chat (WebSocket) enables direct coordination for meeting arrangements without exchanging personal phone numbers. Users can favorite items (favorites feed the recommendation engine).

10.2.2 Rental Request and Date Management

The Renter selects start and end dates from an availability calendar. The system auto-calculates: rental_days × daily_price (or weekly/monthly rate if applicable) + deposit + expert_fee (if requested) + delivery_fee (if applicable).

Double-booking prevention: The system acquires a 5-minute soft-lock on the item's dates. If checkout isn't completed within 5 minutes, the lock expires and the dates become available again. This prevents users from indefinitely blocking inventory without committing.

10.2.3 Payment and Escrow

Upon Owner confirmation, the Renter completes payment through one of the supported methods: Vodafone Cash, Fawry, Paymob, Credit Card, or Bank Transfer (all simulated for MVP).

All money is held in platform escrow—it does NOT go to the Owner until the rental reaches completed status. This protects both parties: - Renters are protected from Owners who take payment and don't deliver. - Owners are protected because the deposit is held until the item is returned in acceptable condition.

10.2.4 The Hub and Expert Evaluation (Trust Mechanic)

Direct Delivery (Standard): For low-to-medium value items, the Renter and Owner coordinate handover directly (in-person or via delivery service). Two-way confirmation applies.

The Hub Evaluation (Premium): For high-value items, either party can request an Expert Check:

1. Item is physically delivered to a Renty Sales Point (Hub).

2. A certified Expert inspects it: condition rating (Excellent/Good/Fair/Poor/Unacceptable), detailed description, min 3 photos, functional test.

3. Expert sets or confirms the required deposit.

4. Renter reviews the Expert report and **accepts or rejects** the item before taking physical possession.

5. **Continuity Rule:** The SAME Expert who performed the pre-rental check MUST perform the post-rental check for consistency.

6. **Who pays?** The party who requested the inspection. If both → split 50/50.

10.2.5 Two-Way Confirmation

Handover Confirmation: Owner confirms item was handed over → Renter confirms item was received. If only one party confirms → system waits 48h, then auto-confirms. This prevents either party from holding up the process.

Return Confirmation: Renter confirms item was returned → Owner confirms item was received back. Same 48h auto-confirm rule applies. If Owner claims damage during return confirmation → "Report Issue" button opens the Dispute Modal.

10.2.6 Completion and Payout

When the rental reaches completed status:

1. **Renter** gets the security deposit refunded (if no damage claim exists).
2. **Renty** deducts the platform commission (default 15%) from the rental fee.
3. **Owner** receives the remaining 85% of the rental fee credited to their wallet.

If Expert post-check was requested, completion waits until the Expert submits the post-rental verdict.

10.3 Late Return Escalation

(Refer to Table 20: Late Return Escalation Timeline.)

Overdue Duration	System Action
1–3 days	Late fee = 150% of daily rate × overdue days. Charged from deposit first.
4–7 days	Admin notified. Owner can escalate via Support ticket.
7+ days	Support escalates. System generates legal PDF with identifying information.
Deposit exhausted	Renter's account flagged with outstanding debt. Cannot rent again until resolved.

10.4 Rental Extensions

When a Renter wants to keep the item longer:

1. Renter requests extension with new end date then Owner receives notification.
2. System calculates extra cost: additional_days × daily_rate.
3. Owner approves or rejects within 48h (auto-reject if no response).

4. If approved, supplementary payment must succeed before dates are updated.

5. Multiple extensions allowed—each treated as a new request with its own payment.

10.5 Cancellation Protection

Owners choose a cancellation policy per item:

(Refer to Table 18: Cancellation Policy Matrix.)

Policy	Free Cancel Until	After That
Flexible	24h before start	First day's rental fee non-refundable. Deposit + remaining days refunded.
Moderate	3 days before start	50% of rental fee refunded. Full deposit refunded.
Strict	7 days before start	0% of rental fee refunded. Full deposit refunded.

Owner-Side Cancellation: Owners can cancel an unstarted rental → Renter gets full refund (100% of everything). Owners who cancel repeatedly (3+ times in 30 days) are flagged for Admin review and may face listing suspension.

Platform Commission on Cancellation: Commission is non-refundable after the free cancellation window. This protects platform revenue from frivolous bookings.

10.6 Dispute Resolution

(Refer to Figure 31: Dispute Resolution Workflow Diagram.)

1. **User files dispute** via Rental Details → Dispute Modal (evidence: photos, description, chat logs).

2. **Support Agent** reviews and either resolves or escalates to Admin.

3. **Admin** reviews evidence + Expert reports (if Hub was used). Available actions:

- Forfeit partial deposit (enter amount ≤ deposit, reason required).
- Forfeit full deposit → credited to Owner's wallet.
- Additional charge (if damage > deposit value → Renter flagged with debt).
- Override: refund Renter (emergency cases).

4. All dispute actions are **audit-logged** (who, when, what, amount, reason).

5. **Super Admin** can reverse Admin's decision (clawback from Owner wallet if needed).

10.7 Wallet Operations

- **Owner Earnings:** Accumulated from completed rentals (85% of rental fee after commission).
- **Transaction History:** Derived from payments, orders, rentals tables, showing every credit and debit.
- **Withdrawals:** MVP: manual withdrawal request → Admin processes. Production: automated payout to bank account.
- **Balance Calculation:** balance = SUM(completed_rental_payouts) - SUM(withdrawals) - SUM(clawbacks).
- **Negative Balance:** If Super Admin reverses a deposit forfeiture after Owner has withdrawn funds, the Owner's balance may go negative (flagged for reconciliation).

10.8 Edge Case Scenarios

10.8.1 Expert Unavailability

If the assigned Expert becomes unavailable (sick, quit, transferred) during the rental lifecycle, the Admin must manually reassign another Expert. The system displays a "Continuity Rule Violated" warning, and the replacement Expert must acknowledge they did not perform the original pre-rental check. This exception is audit-logged.

10.8.2 Payment Gateway Failure

If the payment gateway returns an error during checkout, the 5-minute lock is maintained (not immediately released) to give the user time to retry. After 3 consecutive failures, the lock is released and the user receives a "Payment temporarily unavailable" message with a suggestion to try a different payment method.

10.8.3 Simultaneous Booking Race Condition

When two users attempt to book the same item for overlapping dates simultaneously, the PostgreSQL advisory lock mechanism ensures only one succeeds. The second user receives: “Sorry, these dates were just booked by another user” with a link back to the item calendar showing updated availability.

10.8.4 User Suspension During Active Rental

If a user is suspended while they have an active rental, the rental continues to its natural conclusion (items must still be returned). The suspended user retains read-only access to the affected rental’s chat and status. No new rentals, listings, or bookings are permitted.

10.8.5 Item Reported During Active Rental

If an item receives 3+ reports while it is actively rented, Admin is notified but the active rental is not interrupted. The item is queued for review upon rental completion. If the item is found to violate policies, it is suspended after the rental ends.

10.9 Platform Configuration

(Refer to Table 21: Platform Configuration Settings.)

Setting	Default	Who Can Change
Platform Commission %	15%	Super Admin only
Default Cancellation Policy	Flexible	Admin
Expert Evaluation Fee	Varies by category	Admin
Auto-Resolve Days (Support Tickets)	5	Admin
Fraud Detection Threshold	0.75	Admin
Late Fee Rate	150%	Admin
Checkout Lock Duration	5 minutes	Admin
Feature Toggles (Buying, Hub, AI Recs, Chat)	All On	Super Admin
Maintenance Mode	Off	Super Admin

All setting changes are **audit-logged** with timestamp + who changed it + previous value + new value.

Chapter 11 – Testing and Validation

This chapter documents the quality assurance processes employed to validate the Renty platform across all system layers: the recommendation engine, gap analysis pipeline, frontend-backend integration, and end-to-end system workflows.

11.1 Testing Scope and Strategy

The testing strategy follows a multi-layered approach:

1. **Unit Testing:** Individual functions, models, and utilities tested in isolation.

2. **Integration Testing:** Component interactions verified across module boundaries (API ↔ Database, Frontend ↔ API, AI ↔ Database).

3. **System Testing:** End-to-end workflows tested from user action to database state change to notification delivery.

4. **Performance Testing:** Response time and throughput validation under simulated concurrent load.

11.1.1 Testing Environment

- **Backend:** xUnit test runner with an Entity Framework In-Memory database to isolate test data. Environment variables are configured via appsettings.Testing.json for test mode (disabling rate limiting, using test JWT secrets, and mocking external services).
- **AI Modules:** Python unittest framework with pytest. Separate test datasets derived from temporal splits of the training data.
- **Frontend:** Manual testing with Postman for API validation. Browser DevTools for WebSocket monitoring.
- **Mobile:** Flutter debugging on physical devices (Android) and emulators (iOS via Xcode Simulator).

11.2 Recommendation System Testing

11.2.1 Objective

Verify that the hybrid LightFM model produces relevant, personalized recommendations that generalize beyond training data without overfitting.

11.2.2 Offline Evaluation Procedure

The dataset was split temporally: transactions before the cutoff date formed the training set, and transactions after formed the test set. This simulates production conditions where the model predicts future user behavior based on historical patterns.

Results Summary:

- Test AUC: 0.8421 (exceeds target of 0.80)

- Precision@10: 0.1208 (exceeds target of 0.12)

- Overfitting Gap: 0.0479 (below target of 0.05)

(Refer to Figure 34: Hyperparameter Tuning — Precision@10 vs. Overfitting Gap.)

11.2.3 Overfitting and Stability Checks

Six hyperparameter configurations were systematically evaluated (see Chapter 7, Section 7.1.4). The selected "Light Reg" configuration achieved the lowest overfitting gap (0.0004) while maintaining competitive precision, confirming that L2 regularization with $\alpha = 0.00001$ effectively prevents the model from memorizing training patterns.

Cross-validation stability was verified by training the model on 5 different temporal splits and measuring variance in Precision@10. The coefficient of variation was below 3%, indicating stable model behavior across different data windows.

11.2.4 Cold-Start Validation

Cold-start performance was validated by:

1. **Removing all interactions** for a random subset of 500 users from the training set.

2. **Generating recommendations** using only demographic features (income, occupation, gender, age).

3. **Measuring Precision@10** for these “cold” users against their held-out test interactions.

Result: Cold-start Precision@10 = 0.0847, representing 70% of the warm-user performance. This confirms that the feature-based embedding approach provides meaningful recommendations even without any interaction history.

11.2.5 Output Inspection

Manual inspection of recommendation outputs for 20 randomly selected users confirmed:

- Items from the user’s own listings are never recommended.

- Suspended or pending items are excluded.

- Recommendations include diversity across subcategories (not exclusively from the user’s most-interacted category).

- Score values are monotonically decreasing in the ranked output.

(Refer to Figure 2: Recommendation System Pipeline Diagram.)

11.3 Gap Analysis Testing

11.3.1 Pipeline Execution Testing

The gap analysis pipeline was executed against the test dataset and verified:

- All subcategories received a computed gap score.

- Gap scores are non-negative.

- Severity classifications match the defined thresholds (Table 12).

- Results are correctly persisted to the gap_results table with timestamps.

11.3.2 Automated Execution Testing

The nightly cron job was verified:

- Triggers at 2:00 AM UTC.

- Completes within 5 minutes for a dataset of 130 subcategories across 5 cities.

- Old results are archived (not deleted) to preserve trend analysis capability.

- Email notification sent to Admin upon completion.

11.3.3 Manual Trigger Testing

The “Run Gap Analysis Now” button was verified:

- Initiates computation with loading indicator.

- Completes successfully and updates dashboard data.

- Rate-limited to once per hour (subsequent clicks within 1 hour show “Last run: X minutes ago”).

- Concurrent manual triggers are serialized (second request waits for first to complete).

11.3.4 Data Persistence and Retrieval

Weekly snapshot JSON schema was validated:


```
{
  "week": 22,
  "timestamp": "2026-05-01T02:00:00Z",
  "data": {
    "Diving Equipment": {"gap": 245, "supply": 12, "demand": 2940, "status": "critical"},
    "Hiking Gear": {"gap": 87, "supply": 45, "demand": 3915, "status": "moderate"}
  }
}
```


(Refer to Figures 36-39 for Gap Dashboard visualizations.)

11.4 Integration Testing

11.4.1 Backend-Database Integration

Verified that all API endpoints correctly interact with the PostgreSQL database:

- CRUD operations on all entities produce expected database state changes.

- Foreign key constraints prevent orphaned records.

- Soft-delete sets is_deleted = true without removing rows.

- Triggers execute correctly (self-rental prevention, timestamp updates).

- ENUM constraints reject invalid status values with appropriate error codes.

11.4.2 Frontend-Backend Integration

Verified that the Angular web dashboard and Flutter mobile app correctly consume API responses:

- Authentication flow (login → token storage → authenticated requests → token refresh → logout).

- Real-time notifications arrive via WebSocket within 500ms of triggering event.

- Chat messages are delivered and displayed in correct chronological order.

- Image uploads progress correctly through the Cloudinary pipeline.

- Error states are handled gracefully (network timeout, 401 unauthorized, 403 forbidden, 404 not found, 500 server error).

11.4.3 End-to-End Rental Flow

A complete rental lifecycle was tested from start to finish:

1. Renter searches for item → search logged to search_logs

2. Renter views item → interaction logged to user_interactions

3. Renter initiates rental → 5-minute lock acquired

4. Payment captured → orders and payments created

5. Owner confirms → rental status transitions to confirmed

6. Expert check requested → status transitions to pending_hub_evaluation

7. Expert submits verdict → Renter reviews and accepts

8. Item delivered → two-way handover confirmation

9. Rental active → notifications sent to both parties

10. Item returned → two-way return confirmation

11. Rental completed → escrow released: deposit refunded, commission deducted, owner paid

12. Reviews submitted → dual-blind publication confirmed

11.5 Next-Semester Testing Enhancements (Completed)

The following testing activities, originally planned for the second semester, were completed:

- Real-user testing with team members acting as renters and owners.

- WebSocket stress testing with simulated concurrent connections.

- Controlled adversarial simulations for fraud behavior validation.

- A/B testing preparation for recommendation strategies (infrastructure ready, awaiting production traffic).

Chapter 12 – Conclusion and Future Work

12.1 Summary of Achievements

This graduation project successfully designed, implemented, and validated Renty—an intelligent, cross-platform peer-to-peer rental platform that addresses the interconnected challenges of personalization, security, marketplace optimization, and operational trust in the Egyptian sports tourism market.

Core Platform Achievements:

- A fully functional transactional engine with 9-state rental lifecycle enforced at the database level through PostgreSQL triggers, ENUMs, and CHECK constraints.

- Escrow-based payment system protecting both renters and owners through platform-held funds.

- Physical trust infrastructure (Renty Points) with Expert verification and the Continuity Rule.

- Tiered cancellation policies (Flexible/Moderate/Strict) with predetermined penalty parameters.

- Real-time communication via WebSocket (Socket.IO) for chat, notifications, and live status updates.

- Comprehensive RBAC system supporting five distinct roles with granular permission control.

AI Module Achievements:

- Hybrid recommendation system (LightFM) achieving Test AUC of 0.8421 (+12.3% over baseline), Precision@10 of 0.1208 (+28.8%), and overfitting gap reduction of 80.8%.

- Unsupervised fraud detection (Isolation Forest) correctly identifying all injected anomalous patterns with a 76% true positive rate at the default threshold.

- Supply-demand gap analysis framework with four-tier severity classification and automated nightly execution.

Mobile Application Achievements:

- Cross-platform Flutter application with map-based discovery, real-time push notifications, integrated chat, and camera-based listing creation.

- Offline support with AsyncStorage caching and queued action synchronization.

Administrative Dashboard Achievements:

- KPI monitoring with sparkline trends and color-coded health indicators.

- Gap analysis heatmap with drill-down details and CSV export.

- Expert hub console and support ticket management interfaces.

12.2 Lessons Learned

1. **Database-Level Enforcement is Critical:** Pushing business logic (state machine transitions, self-rental prevention, date validation) to the database layer via triggers and constraints provides the strongest guarantee of data integrity, independent of application code quality.

2. **AI as Enhancement, Not Replacement:** The most successful architecture treats AI modules as supporting components that enhance human decision-making rather than replacing it. Fraud detection flags are recommendations for Admin review, not autonomous blocking decisions. Recommendations guide discovery but do not restrict it.

3. **Physical Trust Bridges Digital Gaps:** In markets where institutional trust infrastructure is nascent, purely digital trust mechanisms (ratings, reviews) are insufficient. Physical verification points (Renty Points) with impartial expert assessors provide the tangible assurance needed to overcome hesitation in high-value peer-to-peer transactions.

4. **State Machines Prevent Chaos:** Without a strictly defined state machine, rental platforms devolve into inconsistent states where payments are captured but items aren't delivered, or items are returned but deposits aren't refunded. The PostgreSQL ENUM-enforced state machine eliminates entire categories of bugs.

12.3 Future Work

The following enhancements are planned for post-graduation development:

- **Real Payment Integration:** Transition from simulated gateways to live Paymob and Fawry integration with HMAC-verified webhooks.
- **Dynamic Pricing:** Implement algorithmic pricing suggestions based on demand patterns, seasonality, and competitive analysis derived from Gap Analysis data.
- **Logistics Integration:** Partner with local delivery services (Bosta, Aramex) for integrated item pickup and delivery with real-time GPS tracking.
- **Native Mobile Push:** Migrate from WebSocket-based notifications to Firebase Cloud Messaging (FCM) for reliable background push notifications.
- **A/B Testing Framework:** Deploy recommendation strategy variants to user segments and measure conversion rate differences.
- **Insurance Integration:** Partner with Egyptian insurance providers to offer optional damage/theft coverage as an alternative to deposit-based protection.
- **Advanced Fraud Detection:** Incorporate graph neural networks to detect coordinated fraud rings through social network analysis of user connections.

- **Horizontal Scaling:** Migrate from monolithic ASP.NET Core architecture to containerized microservices with Kubernetes orchestration for production-scale deployment.

Chapter 13 – References

1. Kula, M. (2015). Metadata Embeddings for User and Item Cold-Start Recommendations. *Proceedings of the 2nd Workshop on New Trends on Content-Based Recommender Systems*, pp. 14-21.

2. Liu, F. T., Ting, K. M., & Zhou, Z. H. (2008). Isolation Forest. *Proceedings of the 2008 Eighth IEEE International Conference on Data Mining*, pp. 413-422.

3. Rendle, S. (2010). Factorization Machines. *Proceedings of the 2010 IEEE International Conference on Data Mining*, pp. 995-1000.

4. Hu, Y., Koren, Y., & Volinsky, C. (2008). Collaborative Filtering for Implicit Feedback Datasets. *Proceedings of the 2008 Eighth IEEE International Conference on Data Mining*, pp. 263-272.

5. Aggarwal, C. C. (2016). *Recommender Systems: The Textbook*. Springer.

6. Chandola, V., Banerjee, A., & Kumar, V. (2009). Anomaly Detection: A Survey. *ACM Computing Surveys*, 41(3), pp. 1-58.

7. Ricci, F., Rokach, L., & Shapira, B. (2015). *Recommender Systems Handbook*. Springer.

8. Goldberg, D., Nichols, D., Oki, B. M., & Terry, D. (1992). Using Collaborative Filtering to Weave an Information Tapestry. *Communications of the ACM*, 35(12), pp. 61-70.

9. Sarwar, B., Karypis, G., Konstan, J., & Riedl, J. (2001). Item-Based Collaborative Filtering Recommendation Algorithms. *Proceedings of the 10th International Conference on World Wide Web*, pp. 285-295.

10. Breese, J. S., Heckerman, D., & Kadie, C. (1998). Empirical Analysis of Predictive Algorithms for Collaborative Filtering. *Proceedings of the 14th Conference on Uncertainty in Artificial Intelligence*, pp. 43-52.

11. Burke, R. (2002). Hybrid Recommender Systems: Survey and Experiments. *User Modeling and User-Adapted Interaction*, 12(4), pp. 331-370.

12. Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix Factorization Techniques for Recommender Systems. *IEEE Computer*, 42(8), pp. 30-37.

13. Salton, G. & Buckley, C. (1988). Term-Weighting Approaches in Automatic Text Retrieval. *Information Processing & Management*, 24(5), pp. 513-523.

14. PostgreSQL Global Development Group. (2024). *PostgreSQL 14 Documentation*. https://www.postgresql.org/docs/14/

15. Microsoft. ASP.NET Core Documentation. https://learn.microsoft.com/en-us/aspnet/core/

16. Socket.IO. (2024). *Socket.IO Documentation*. https://socket.io/docs/v4/

17. Google. Flutter Documentation. https://docs.flutter.dev/

18. Google. Angular Documentation. https://angular.dev/

19. JSON Web Tokens. (2024). *JWT Introduction*. https://jwt.io/introduction/

20. Pedregosa, F., et al. (2011). Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12, pp. 2825-2830.

21. McKinney, W. (2010). Data Structures for Statistical Computing in Python. *Proceedings of the 9th Python in Science Conference*, pp. 56-61.

22. IEEE Computer Society. (2017). *IEEE Standard for Software Life Cycle Processes (IEEE 12207-2017)*.

23. ISO/IEC. (2011). *Systems and Software Quality Requirements and Evaluation (ISO/IEC 25010:2011)*.

24. Object Management Group. (2017). *Unified Modeling Language Specification, Version 2.5.1*.

25. OWASP Foundation. (2021). *OWASP Top Ten Web Application Security Risks*. https://owasp.org/www-project-top-ten/

26. Paymob. (2024). *Paymob Payment Gateway API Documentation*. https://docs.paymob.com/

27. Cloudinary. (2024). *Cloudinary Media Management Documentation*. https://cloudinary.com/documentation

28. Weston, J., Bengio, S., & Usunier, N. (2011). WSABIE: Scaling Up to Large Vocabulary Image Annotation. *Proceedings of IJCAI*, pp. 2764-2770.

Appendix A – Database Schema Excerpts

A.1 Core ENUM Types

```
CREATE TYPE rental_status AS ENUM (
  'requested', 'confirmed', 'paid', 'pending_hub_evaluation',
  'evaluation_review', 'in_delivery', 'active', 'returned',
  'completed', 'cancelled', 'extended', 'disputed'
);

CREATE TYPE payment_method AS ENUM (
  'vodafone_cash', 'fawry', 'paymob', 'credit_card', 'bank_transfer'
);

CREATE TYPE payment_status AS ENUM ('pending', 'success', 'failed', 'refunded');
CREATE TYPE order_status AS ENUM ('pending', 'paid', 'refunded', 'partially_refunded', 'cancelled');
CREATE TYPE item_status AS ENUM ('pending_approval', 'available', 'rented', 'sold', 'suspended', 'deleted');
CREATE TYPE cancellation_policy AS ENUM ('flexible', 'moderate', 'strict');
CREATE TYPE condition_rating AS ENUM ('excellent', 'good', 'fair', 'poor', 'unacceptable');
CREATE TYPE ticket_status AS ENUM ('open', 'in_progress', 'resolved', 'closed');
CREATE TYPE verification_status AS ENUM ('pending', 'approved', 'rejected');
```

A.2 Self-Rental Prevention Trigger

```
CREATE OR REPLACE FUNCTION fn_prevent_self_rental()
RETURNS TRIGGER AS $$
BEGIN
  IF NEW.renter_id = (SELECT owner_id FROM items WHERE id = NEW.item_id) THEN
    RAISE EXCEPTION 'Cannot rent your own item';
  END IF;
  RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_prevent_self_rental
  BEFORE INSERT ON rentals
  FOR EACH ROW
  EXECUTE FUNCTION fn_prevent_self_rental();
```

A.3 Date Validation Constraints

```
ALTER TABLE rentals ADD CONSTRAINT chk_rental_dates CHECK (end_date > start_date);
ALTER TABLE items ADD CONSTRAINT chk_positive_price CHECK (daily_price > 0);
ALTER TABLE items ADD CONSTRAINT chk_positive_deposit CHECK (deposit_amount >= 0);
ALTER TABLE reviews ADD CONSTRAINT chk_valid_rating CHECK (rating >= 1 AND rating <= 5);
```

A.4 Key Analytical Views

```
CREATE VIEW vw_active_rentals AS
SELECT r.id, r.status, r.start_date, r.end_date,
    i.title AS item_title, i.daily_price,
    renter.first_name AS renter_name,
    owner.first_name AS owner_name
FROM rentals r
JOIN items i ON r.item_id = i.id
JOIN users renter ON r.renter_id = renter.id
JOIN users owner ON i.owner_id = owner.id
WHERE r.status IN ('active', 'extended');

CREATE VIEW vw_revenue_summary AS
SELECT DATE_TRUNC('month', r.created_at) AS month,
    c.name AS category,
    SUM(r.total_price * r.commission_rate / 100) AS platform_revenue,
    COUNT(*) AS completed_rentals
FROM rentals r
JOIN items i ON r.item_id = i.id
JOIN subcategories sc ON i.subcategory_id = sc.id
JOIN categories c ON sc.category_id = c.id
WHERE r.status = 'completed'
GROUP BY 1, 2
ORDER BY 1 DESC;
```

End of Document — Renty: AI-Enhanced Peer-to-Peer Rental Platform — Graduation Project Report (Semesters I & II)

Appendix B – API Contract Reference

B.1 Authentication Endpoints

POST /api/v1/auth/register

Request Body:

```
{
  "firstName": "string (required, min 2, max 50)",
  "lastName": "string (required, min 2, max 50)",
  "email": "string (required, valid email format)",
  "password": "string (required, min 8, must contain uppercase, lowercase, number)",
  "phone": "string (required, Egyptian format: +20XXXXXXXXXX)",
  "city": "string (required)"
}
```

Success Response (201 Created):

```
{
  "success": true,
  "message": "Registration successful. Please verify your email.",
  "data": {
    "userId": 42,
    "email": "user@example.com",
    "emailVerified": false
  }
}
```

Error Responses:

- 400 Bad Request — Validation error (missing fields, invalid format).

- 409 Conflict — Email already registered.

Side Effects:

- Creates users record with email_verified = false.

- Generates UUID verification token stored in email_verifications table.

- Sends verification email with link: GET /auth/verify-email/:token.

- Assigns default role: user.

POST /api/v1/auth/login

Request Body:

```
{
  "email": "string (required)",
  "password": "string (required)"
}
```

Success Response (200 OK):

```
{
  "success": true,
  "data": {
    "accessToken": "eyJhbGciOiJIUzI1NiIs...",
    "refreshToken": "eyJhbGciOiJIUzI1NiIs...",
    "user": {
      "id": 42,
      "firstName": "Ahmed",
      "role": "user",
      "emailVerified": true,
      "phoneVerified": true,
      "idVerified": false
    }
  }
}
```

Error Responses:

- 401 Unauthorized — Invalid credentials.

- 403 Forbidden — Account suspended ("Your account has been suspended. Contact support.").

- 403 Forbidden — Email not verified ("Please verify your email first.").

- 429 Too Many Requests — Rate limited (10 attempts/minute per IP).

Side Effects:

- Updates last_login and increments login_count in users table.

- If 5 consecutive wrong passwords → 15-minute lockout + email notification.

B.2 Item Management Endpoints

GET /api/v1/items

Query Parameters:

- category (optional) — Filter by category ID.

- subcategory (optional) — Filter by subcategory ID.

- city (optional) — Filter by city name.

- minPrice / maxPrice (optional) — Price range filter on daily_price.

- condition (optional) — Filter by condition rating.

- startDate / endDate (optional) — Filter by availability.

- sort (optional) — price_asc, price_desc, rating, newest (default: newest).

- page / limit (optional) — Pagination (default: page=1, limit=20).

- q (optional) — Full-text search across title, description, category name.

Success Response (200 OK):


```
{
  "success": true,
  "data": {
    "items": [
      {
        "id": 101,
        "title": "Professional Diving BCD",
        "dailyPrice": 150,
        "weeklyPrice": 900,
        "monthlyPrice": 3000,
        "depositAmount": 1500,
        "condition": "excellent",
        "primaryImage": "https://res.cloudinary.com/...",
        "ownerRating": 4.7,
        "city": "Dahab",
        "cancellationPolicy": "moderate",
        "isAvailable": true
```

```
    }
   ],
   "pagination": {
    "currentPage": 1,
    "totalPages": 12,
    "totalItems": 234,
    "itemsPerPage": 20
   }
  }
}
```


Side Effects: - If q parameter is present, query is logged to search_logs table for Gap Analysis.

POST /api/v1/items

Headers: Authorization: Bearer <accessToken> (required, must be email-verified user).

Request Body (multipart/form-data):

- title — string, required, min 5, max 100.

- description — string, required, min 20, max 2000.

- categoryId — integer, required, valid category FK.

- subcategoryId — integer, required, valid subcategory FK.

- dailyPrice — decimal, required, > 0.

- weeklyPrice — decimal, optional, > 0.

- monthlyPrice — decimal, optional, > 0.

- depositAmount — decimal, optional (default: 10% of dailyPrice × 7).

- cancellationPolicy — enum, optional (default: flexible).

- city — string, required. - latitude / longitude — decimal, optional (for map display).

- images[] — file array, min 1, max 10 (jpg/png, max 5MB each).

Success Response (201 Created):

```
{
  "success": true,
  "message": "Item submitted for approval.",
  "data": {
    "itemId": 101,
    "status": "pending_approval"
  }
}
```


B.3 Rental Lifecycle Endpoints

POST /api/v1/rentals

Headers: Authorization: Bearer <accessToken> (required, must be phone-verified).

Request Body:


```
{
  "itemId": 101,
  "startDate": "2026-06-15",
  "endDate": "2026-06-20",
  "requestExpertCheckBefore": true,
  "requestExpertCheckAfter": true,
  "deliveryMethod": "pickup"
}
```


Success Response (201 Created):


```
{
  "success": true,
  "data": {
    "rentalId": 501,
    "status": "requested",
    "totalPrice": 750,
    "depositAmount": 1500,
    "expertFee": 100,
    "commissionRate": 15,
    "lockExpiresAt": "2026-06-10T14:05:00Z"
  }
}
```


Error Responses:

- 400 Bad Request — Invalid dates, item unavailable, self-rental attempt.

- 409 Conflict — Dates already locked by another user.

- 403 Forbidden — User not phone-verified, or fraud score = blocked.

Side Effects:

- Creates rentals record with status = 'requested'.

- Acquires 5-minute advisory lock on item-date range.

- Sends notification to Owner: rental_request_received.

- Triggers fraud score check on Renter.

PUT /api/v1/rentals/:id/confirm

Headers: Authorization: Bearer <accessToken> (must be item Owner).

Success Response (200 OK):

```
{
  "success": true,
  "data": {
    "rentalId": 501,
    "status": "confirmed",
    "message": "Rental confirmed. Awaiting payment from renter."
  }
}
```

Side Effects:

- Updates rentals.status from requested to confirmed.

- Sends notification to Renter: rental_confirmed.

- Payment window opens (Renter must pay within 24h or rental auto-cancels).

PUT /api/v1/rentals/:id/cancel

Request Body:

```
{
  "reason": "string (optional)"
}
```

Business Logic:

- Applies cancellation policy (Flexible/Moderate/Strict) based on time remaining before start_date.

- Calculates refund amount based on policy tier.

- Creates refund record in payments table.

- Releases date lock.

- Sends notification to other party: rental_cancelled.

B.4 Payment Endpoints

POST /api/v1/payments/checkout

Request Body:

```
{
  "rentalId": 501,
  "paymentMethod": "vodafone_cash",
  "idempotencyKey": "uuid-v4-unique-per-attempt"
}
```

Processing Flow:

1. Validate idempotency key (prevent duplicate captures).

2. Run fraud score check on user.

3. If blocked → return 403 with “Your account is under review.”

4. If flagged → proceed but log to fraud_logs and notify Admin.

5. If safe → initiate payment capture with gateway.

6. On success → create orders and payments records, update rentals.status to paid.

7. On failure → return 402 with error details, maintain lock for retry.

POST /api/v1/payments/webhook

Purpose: Receive asynchronous payment status updates from gateway.

Security: HMAC signature verification using PAYMOB_HMAC_SECRET.

Processing: Update payments.status based on callback data. If success then trigger rental status transition. If failed then release lock and notify user.

B.5 Real-Time WebSocket Events

Client → Server Events

Event	Payload	Description
join_chat	{ chatId }	Join a chat room
send_message	{ chatId, content, type }	Send a message
typing_start	{ chatId }	Typing indicator on
typing_stop	{ chatId }	Typing indicator off
mark_read	{ chatId, messageId }	Mark message as read
subscribe_ notifications	{ userId }	Subscribe to personal notification channel

Server → Client Events

Event	Payload	Description
new_message	{ chatId, message, sender }	New message received
user_typing	{ chatId, userId }	Other user is typing
message_read	{ chatId, messageId, readAt }	Message was read (✓✓)
user_status	{ userId, status }	Online/offline status change
notification	{ type, title, body, redirectUrl }	Platform notification
rental_status_update	{ rentalId, newStatus, timestamp }	Rental state change

B.6 Admin-Only Endpoints

GET /api/v1/admin/dashboard

Response includes:

- dailyActiveUsers — Count of unique users with activity in last 24h.

- monthlyActiveUsers — Count of unique users with activity in last 30d.

- totalRevenue — Sum of commissions from completed rentals (MTD).

- activeRentals — Count of rentals in active or extended status.

- pendingListings — Count of items with status = 'pending_approval'.

- openDisputes — Count of disputes not yet resolved.

- escrowBalance — Sum of all held payments not yet released.

PUT /api/v1/admin/items/:id/approve

Side Effects:

- Updates items.status from pending_approval to available.

- Sets items.approved_by to Admin's user ID.

- Sends notification to Owner: item_approved.

PUT /api/v1/admin/items/:id/reject

Request Body:

```
{
  "reason": "string (required, min 10 characters)"
}
```

Side Effects:

- Updates items.status to rejected.

- Sends notification to Owner: item_rejected with reason.

- If item has been rejected 3+ times then Owner flagged for Admin review.

PUT /api/v1/admin/fraud/:id/action

Request Body:

```
{
  "action": "block | whitelist | dismiss | suspend",
  "reason": "string (required)"
}
```

Side Effects:

- block then User cannot initiate new transactions.

- whitelist then User excluded from future fraud scoring.

- dismiss then Flag marked as false positive, logged for model recalibration.

- suspend then Full account suspension, all active sessions invalidated.

- All actions audit-logged with Admin ID, timestamp, and reason.

Appendix C – Notification Event Type Reference

The following table enumerates all 28+ notification event types supported by the platform:

#	Event Type	Triggered When	Recipients	Redirect URL
1	rental_request_received	Renter submits rental request	Owner	/rentals/:id
2	rental_confirmed	Owner accepts rental	Renter	/rentals/:id
3	rental_rejected	Owner rejects rental	Renter	/rentals/:id
4	rental_cancelled	Either party cancels	Both	/rentals/:id
5	payment_success	Payment captured successfully	Renter	/rentals/:id
6	payment_failed	Payment capture fails	Renter	/payments/retry
7	item_shipped	Owner ships/hands over item	Renter	/rentals/:id
8	item_received	Renter confirms receipt	Owner	/rentals/:id

#	Event Type	Triggered When	Recipients	Redirect URL
9	return_reminder	24h before rental end date	Renter	/rentals/:id
10	item_returned	Renter initiates return	Owner	/rentals/:id
11	rental_completed	Both confirmations + payout done	Both	/rentals/:id
12	extension_request	Renter requests extension	Owner	/rentals/:id
13	extension_approved	Owner approves extension	Renter	/rentals/:id
14	extension_rejected	Owner rejects (or auto-reject 48h)	Renter	/rentals/:id
15	new_message	New chat message received	Recipient	/chats/:id
16	dispute_opened	User files dispute	Both + Admin	/disputes/:id
17	dispute_update	Admin takes action on dispute	Both	/disputes/:id
18	dispute_resolved	Dispute resolution finalized	Both	/disputes/:id
19	item_approved	Admin approves listing	Owner	/items/:id
20	item_rejected	Admin rejects listing	Owner	/items/:id
21	late_fee_alert	Rental overdue by 1+ days	Renter	/rentals/:id
22	no_show_warning	Owner didn’t confirm 48h	Owner	/rentals/:id
23	auto_cancelled	System auto-cancels due to timeout	Both	/rentals/:id
24	deposit_refunded	Deposit returned to Renter	Renter	/wallet
25	deposit_forfeited	Deposit awarded to Owner	Both	/disputes/:id
26	new_review	Review submitted for user	Recipient	/reviews/:id
27	fraud_alert	AI flags user transaction	Admin	/admin/fraud
28	account_suspended	Admin suspends account	User	/support
29	verification_approved	ID verification approved	User	/profile
30	verification_rejected	ID verification rejected	User	/profile/verify
31	expert_assigned	Item assigned to Expert for check	Expert	/expert/queue
32	owner_request_timeout	Owner didn’t respond in 48h	Both	/rentals/:id

Appendix D – Extended Entity Interaction Scenarios

This appendix documents comprehensive real-world interaction scenarios covering edge cases, conflict resolution paths, and system behavior under non-standard conditions. These scenarios informed the implementation of business rules and state machine logic.

D.1 Scenario: Renter Requests Rental for a High-Value Item

Actors: Renter (Ahmed), Owner (Fatma), Expert (Khaled), Platform

Preconditions: Ahmed is email-verified and phone-verified. Fatma's BCD diving gear is listed at 200 EGP/day with a 2,000 EGP deposit. Cancellation policy: Moderate.

Flow:

1. Ahmed searches "BCD Dahab" → search logged to search_logs for Gap Analysis.

2. Ahmed views item → interaction logged as view in user_interactions.

3. Ahmed clicks "Rent Now" → selects June 15-20 (5 days).

4. System calculates: Rental Fee = 5 × 200 = 1,000 EGP. Deposit = 2,000 EGP. Ahmed checks "Request Expert Check Before." Expert Fee = 100 EGP (charged to Ahmed). Total = 3,100 EGP.

5. System acquires 5-minute soft-lock on BCD for June 15-20.

6. Ahmed selects Vodafone Cash → payment simulated → payments.status = 'success'.

7. Rental created: status = 'requested'. Notification → Fatma.

8. Fatma reviews request → clicks "Confirm." status = 'confirmed'.

9. Because Expert Check was requested → status = 'pending_hub_evaluation'.

10. BCD delivered to Renty Point Dahab. Expert Khaled receives assignment notification.

11. Khaled inspects: Condition = "Good", functional test passed, 4 photos taken. Deposit recommendation: 2,000 EGP (confirms Owner's setting).

12. status = 'evaluation_review'. Ahmed reviews Expert report.

13. Ahmed clicks "Accept" → status = 'active'. Rental clock begins.

14. June 19 (day before return): System sends return_reminder notification to Ahmed.

15. June 20: Ahmed returns BCD to Renty Point. Khaled (same Expert, Continuity Rule) performs post-rental check.

16. Post-check: “No Damage” → status = 'returned'.

17. Two-way confirmation: Ahmed confirms return, Fatma confirms receipt.

18. status = 'completed'. Escrow release: Ahmed gets 2,000 EGP deposit refund. Renty takes 150 EGP commission (15% of 1,000). Fatma receives 850 EGP in wallet.

D.2 Scenario: Late Return with Deposit Exhaustion

Actors: Renter (Omar), Owner (Sara), Admin (Admin1)

Preconditions: Omar rented a kite surfing kite for June 1-5 (5 days, 100 EGP/day). Deposit: 500 EGP. No Expert check.

Flow:

1. Rental proceeds normally to active status on June 1.

2. June 5: System sends return_reminder. Omar does not return the kite.

3. June 6 (Day 1 overdue): Late fee = 150% × 100 = 150 EGP. Charged from deposit. Remaining deposit: 350 EGP.

4. June 7 (Day 2 overdue): Additional 150 EGP. Remaining deposit: 200 EGP.

5. June 8 (Day 3 overdue): Additional 150 EGP. Remaining deposit: 50 EGP.

6. June 9 (Day 4 overdue): Additional 150 EGP. Deposit exhausted. Outstanding debt: 100 EGP.

7. June 9: Admin1 notified. Sara notified with escalation option.

8. Sara files support ticket with “Equipment not returned” complaint.

9. June 12 (Day 7 overdue): System auto-generates legal PDF containing Omar’s verified identity information, rental agreement, and overdue timeline.

10. Omar’s account flagged with has_outstanding_debt = true. He cannot initiate any new rentals until the debt of 100 EGP + any additional late fees is resolved.

11. Admin1 contacts Omar through support system. Resolution options: Omar pays outstanding balance → debt cleared → account unflagged. OR Admin1 escalates to legal proceedings using generated PDF.

D.3 Scenario: Dispute Resolution with Expert Evidence

Actors: Renter (Youssef), Owner (Nour), Expert (Mahmoud), Admin (Admin1)

Flow:

1. Youssef rents Nour's GoPro camera for a diving trip. Expert pre-check: "Excellent condition."

2. Upon return, Expert Mahmoud (same Expert, Continuity Rule) finds a cracked lens housing.

3. Post-rental verdict: "Damage Found — Renter Liable" with recommended penalty of 800 EGP.

4. status = 'disputed'. Escrow frozen. Notification to both parties + Admin.

5. Admin1 reviews: Expert's pre-rental photos (pristine), post-rental photos (cracked lens), Youssef's counter-argument ("it was fine when I returned it").

6. Admin1 decides: Forfeit 800 EGP from Youssef's 1,500 EGP deposit → 800 EGP credited to Nour's wallet. Remaining 700 EGP refunded to Youssef.

7. Renty's 15% commission (on rental fee) is still deducted normally.

8. status = 'completed'. Both parties can still leave reviews (with "Disputed" badge).

9. All actions audit-logged: Admin1 ID, timestamp, amount, reason ("Lens housing damage confirmed by Expert post-check photos").

D.4 Scenario: Cancellation Under Strict Policy

Actors: Renter (Amr), Owner (Dalia)

Preconditions: Amr booked Dalia's premium camera for June 20-25 under "Strict" cancellation policy. Rental fee: 500 EGP/day × 5 = 2,500 EGP. Deposit: 3,000 EGP.

Flow:

1. Rental confirmed and paid on June 1. Status = paid.

2. June 10: Amr decides to cancel (10 days before start date).

3. Strict policy: Free cancel until 7 days before start (June 13). Since June 10 is before the deadline, Amr qualifies for free cancellation.

4. **Result:** 100% refund of rental fee + 100% refund of deposit. Platform commission is NOT charged.

Alternative Flow (Late Cancellation):

1. If Amr cancels on June 15 (5 days before start, AFTER the 7-day deadline):

2. Strict policy: 0% of rental fee refunded. Full deposit refunded.

3. **Result:** Amr loses 2,500 EGP rental fee. Gets 3,000 EGP deposit back. Dalia receives 2,125 EGP (2,500 - 15% commission). Platform earns 375 EGP commission.

D.5 Scenario: Simultaneous Booking Race Condition

Actors: Renter A (Layla), Renter B (Hassan), Owner (Kareem)

Flow:

1. Both Layla and Hassan view Kareem's surfboard (available June 10-15) at approximately the same time.

2. Layla clicks "Rent Now" at 14:00:00.000 → system acquires advisory lock on surfboard dates June 10-15.

3. Hassan clicks "Rent Now" at 14:00:00.200 (200ms later) → system attempts to acquire lock → BLOCKED.

4. Hassan receives: "Sorry, these dates were just booked by another user. The item calendar has been updated."

5. Layla completes checkout within 5 minutes → rental created successfully.

6. If Layla does NOT complete checkout within 5 minutes → lock expires at 14:05:00 → Hassan's next attempt would succeed.

D.6 Scenario: Owner Cancellation Abuse Prevention

Actors: Owner (Mostafa), Multiple Renters, Admin

Flow:

1. Mostafa confirms a rental with Renter A → then cancels 2 hours later.

2. One week later, Mostafa confirms another rental with Renter B → cancels again.

3. System tracks: Mostafa has cancelled 2 owner-side cancellations in 30 days.

4. Third cancellation: Mostafa confirms rental with Renter C → cancels again.

5. System auto-flags Mostafa: owner_cancellation_count >= 3 in rolling 30-day window.

6. Admin receives notification: “Owner Mostafa has cancelled 3 rentals in 30 days.”

7. Admin reviews and may: Issue warning, temporarily suspend listing capability, or permanently suspend account if pattern continues.

8. All Renters affected received full 100% refunds (owner-side cancellation always = full refund regardless of policy).

D.7 Scenario: Expert Continuity Rule Violation

Actors: Renter (Ali), Owner (Mona), Expert A (Tarek), Expert B (Heba), Admin

Flow:

1. Pre-rental Expert check performed by Tarek at Renty Point Cairo.

2. Rental proceeds to active status.

3. Upon return, Tarek is unavailable (sick leave).

4. System attempts to assign Tarek → detects unavailability.

5. Admin is notified: “Continuity Rule cannot be satisfied for Rental #501.”

6. Admin manually assigns Expert Heba as replacement.

7. System displays “Continuity Rule Violated” warning on the rental record.

8. Heba performs post-rental check. Must acknowledge: “I did not perform the pre-rental check. I am reviewing the previous Expert’s photos and report for reference.”

9. Heba’s verdict carries reduced confidence weight. Admin reviews with extra scrutiny.

10. Exception audit-logged: Original Expert (Tarek), Replacement Expert (Heba), Reason (sick leave), Admin who approved override.

D.8 Scenario: Mobile App Offline-to-Online Sync

Actors: Renter (Noura) using mobile app in area with poor connectivity

Flow:

1. Noura is in a diving area in Dahab with intermittent mobile data.

2. She opens the app → cached data from AsyncStorage loads immediately (last viewed items, active rental status).

3. She types a chat message to the Owner → message queued locally with status = 'pending'.

4. She changes a favorite → action queued.

5. Network returns → app detects connectivity → queued actions sync: - Chat message sent via Socket.IO with original timestamp. - Favorite toggle synced via REST API.

6. Any server-side state changes that occurred while offline are pulled and merged: - If rental status changed → timeline updates with animation. - If new messages received → chat scrolls to show missed messages.

7. Conflict resolution: If the rental was cancelled while Noura was offline, the app shows a prominent alert upon reconnection.

Appendix F – Detailed Non-Functional Requirements

F.1 Performance Requirements

F.1.1 Response Time Requirements

The system must satisfy the following response time constraints under normal operating conditions (up to 1,000 concurrent users):

Operation Category	Maximum Response Time	Measurement Point
Static page load (home, browse)	2 seconds	First Contentful Paint
API read operations (GET)	200 milliseconds	Server-side processing
API write operations (POST/PUT)	500 milliseconds	Server-side processing
Search with filters	300 milliseconds	Server-side query + response
Recommendation generation	100 milliseconds	Pre-computed cache retrieval
Fraud score computation	5 seconds	Asynchronous background
Image upload (Cloudinary)	3 seconds	Client to CDN completion
WebSocket message delivery	500 milliseconds	End-to-end latency
Database query (indexed)	50 milliseconds	PostgreSQL query execution

Operation Category	Maximum Response Time	Measurement Point
Database query (analytical view)	2 seconds	Complex JOIN operations

F.1.2 Throughput Requirements

Metric	Target	Notes
Concurrent WebSocket connections	10,000	Socket.IO with Redis adapter
API requests per second	500	ASP.NET Core with Kestrel and async processing
Database transactions per second	200	PostgreSQL with connection pooling
Chat messages per second	100	Per Socket.IO instance
Image uploads per minute	50	Cloudinary rate limits apply

F.1.3 Scalability Approach

The system is designed for vertical scaling in the MVP phase (increasing server resources) with a clear migration path to horizontal scaling for production:

•API Layer: ASP.NET Core Kestrel server with async request handling. Future: Kubernetes pod auto-scaling.

- **Database Layer:** PostgreSQL read replicas for analytical queries. Future: Connection pooling via PgBouncer.
- **WebSocket Layer:** Socket.IO with Redis pub/sub adapter for multi-instance coordination.
- **AI Layer:** Separate Python workers with job queue (Bull/Redis). Future: Ray or Celery for distributed ML.

F.2 Security Requirements

F.2.1 Authentication Security

Control	Implementation	Standard
Password Storage	bcrypt with 12 salt rounds	OWASP
Token Format	JWT with HS256 signing	RFC 7519
Access Token Lifetime	15 minutes	Security best practice
Refresh Token Lifetime	7 days	Security best practice
Token Rotation	New pair issued on refresh, old invalidated	OWASP
Brute Force Protection	5 attempts → 15-minute lockout + email alert	OWASP
Session Invalidation	Password change invalidates all existing tokens	Security requirement

F.2.2 Data Protection

Control	Implementation
Transport Encryption	TLS 1.2+ mandatory for all connections
Data at Rest	PostgreSQL transparent data encryption
PII Access Logging	All access to user personal data audit-logged
Soft Delete	User data preserved for legal compliance, hidden from UI
File Upload Validation	MIME type verification, file size limits, malware scanning
SQL Injection Prevention	Parameterized queries via Sequelize ORM
XSS Prevention	Input sanitization, Content Security Policy headers
CORS	Whitelist-only origin policy

F.2.3 Rate Limiting Configuration

Endpoint Category	Limit	Window	Response
POST /auth/login	10 requests	1 minute	429 Too Many Requests
POST /auth/register	5 requests	1 minute	429
POST /auth/forgot-password	3 requests	1 hour	429
POST /rentals	20 requests	1 minute	429
POST /payments/checkout	5 requests	1 minute	429
GET /items (catalog)	100 requests	1 minute	429
POST /chats/:id/messages	60 requests	1 minute	429
Admin endpoints	200 requests	1 minute	429

F.3 Reliability and Availability

F.3.1 Data Integrity Guarantees

Guarantee	Enforcement Mechanism
No orphaned financial records	PostgreSQL foreign keys with ON DELETE RESTRICT
No invalid rental states	PostgreSQL ENUM type for rental_status
No self-rentals	Database trigger fn_prevent_self_rental
No negative prices	CHECK constraint daily_price > 0
No invalid date ranges	CHECK constraint end_date > start_date
No duplicate payments	Idempotency key UNIQUE constraint
Historical data preservation	Soft-delete pattern (is_deleted flag)
Audit trail completeness	Timestamp triggers on all mutable tables

F.3.2 Error Handling Strategy

All API errors follow a standardized response format:

```
{
  "success": false,
  "error": {
    "code": "RENTAL_DATES_UNAVAILABLE",
    "message": "The requested dates are no longer available.",
    "details": {
      "conflictingRentalId": 502,
      "availableFrom": "2026-06-25"
    }
  }
}
```

Error categories and HTTP status codes:

- 400 — Client validation errors (bad input, missing fields).
- 401 — Authentication failures (expired token, invalid credentials).
- 403 — Authorization failures (insufficient role permissions).
- 404 — Resource not found (deleted or non-existent entity).
- 409 — Conflict (duplicate email, overlapping dates, concurrent modification).
- 422 — Unprocessable entity (valid syntax but business rule violation).
- 429 — Rate limit exceeded.
- 500 — Internal server error (unexpected, logged with full stack trace).

F.4 Maintainability Requirements

F.4.1 Code Organization

The backend follows a layered directory structure:

```
src/
├── config/         # Environment and database configuration
├── middleware/      # Auth, validation, rate limiting, error handling
├── routes/         # ASP.Net Core API Controllers (one file per resource)
├── controllers/    # Request handling and response formatting
├── services/       # Business logic and state machine transitions
├── models/         # Sequelize model definitions
```

```
├── utils/          # Shared utilities (pagination, date helpers, token generation)
├── validators/     # FluentValidator/ASP.Net Core Validation Schemas
├── jobs/           # Cron jobs (gap analysis, notification cleanup)
├── websocket/      # Socket.IO event handlers
└── tests/          # Test suites organized by module
```

F.4.2 Documentation Standards

Artifact	Format	Location
API Documentation	Swagger/OpenAPI 3.0	Auto-generated at /api/docs
Database Schema	ERD + SQL comments	DB.sql with inline documentation
Architecture Decisions	Markdown records	System-Design-v2.0/ folder
Code Comments	JSDoc for JS, docstrings for Python	Inline in source files
Deployment Guide	Step-by-step markdown	README.md + DEPLOYMENT.md

F.4.3 Version Control Practices

- **Branching Strategy:** Git Flow with main (production), develop (integration), feature/* (individual features), and hotfix/* (critical fixes).
- **Commit Messages:** Conventional Commits format (feat:, fix:, docs:, refactor:, test:, chore:).
- **Pull Requests:** Require at least one team member review before merge. Automated checks: linting, test suite, build verification.
- **Release Tags:** Semantic versioning (v1.0.0, v1.1.0, v2.0.0) with changelog documentation.

Appendix G – Glossary of Domain-Specific Terms

Term	Definition
Renty Point	A physical location (hub) where certified Experts inspect rental items before and after the rental period. Provides impartial third-party verification of item condition.
Continuity Rule	The requirement that the same Expert who performs the pre-rental inspection must also perform the post-rental inspection for a given rental, ensuring consistency in condition assessment.

Term	Definition
Escrow	The financial arrangement where the platform holds all rental payments and deposits until the rental reaches completed status, protecting both parties from fraud.
Checkout Lock	A 5-minute temporary reservation on item availability dates, preventing double-booking during the payment process. Automatically expires if payment is not completed.
Dual Rating	The system where each user maintains two separate ratings: one as an Owner (reliability as a lender) and one as a Renter (trustworthiness as a borrower).
Dual-Blind Review	The review mechanism where neither party can see the other's review until both have submitted, preventing retaliatory or influenced ratings.
Gap Score	A normalized metric quantifying the imbalance between supply and demand for a specific product subcategory in a specific geographic region. Higher scores indicate more severe undersupply.
Behavioral Velocity Vector	The composite feature set extracted from a user's behavioral patterns (temporal, velocity, financial, geographic dimensions) used as input to the Isolation Forest fraud detection model.
Commission Lock	The principle that the platform commission percentage applicable to a rental is fixed at the time the rental is created, regardless of subsequent changes to the global commission rate.
Soft Delete	A data management pattern where records are marked as deleted (is_deleted = true) rather than physically removed from the database, preserving referential integrity and enabling historical auditing.
Advisory Lock	A PostgreSQL locking mechanism used to implement the 5-minute checkout lock, providing application-level mutual exclusion without blocking database-level operations on other rows.
Two-Way Confirmation	The handover and return protocol requiring both the Renter and Owner to independently confirm that the item was delivered/returned, with automatic confirmation after 48 hours if only one party responds.